

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN
MÔN HỌC CÁC HỆ THỐNG PHÂN TÁN

ĐỀ TÀI:

PHÁT TRIỂN HỆ THỐNG CHAT NGANG HÀNG P2P

Giảng viên hướng dẫn: TS. Kim Ngọc Bách

Nhóm lớp: 01

Nhóm bài tập lớn: 13

Danh sách thành viên:

Phạm Văn Đức B22DCCN245

Phan Văn Hoàn B22DCCN329

Nguyễn Văn Hoàng B22DCCN341

Hà Nội, 2026

PHÂN CHIA CÔNG VIỆC

Thành viên	Nhiệm vụ chính	Nội dung thực hiện	Chức năng nâng cao phụ trách
Nguyễn Văn Hoàng	Quản lý mạng P2P và Peer Discovery	Xây dựng Bootstrap Server, xử lý peer join/leave, quản lý danh sách peer online, peer discovery, heartbeat và cập nhật trạng thái online/offline	Mô phỏng churn (peer tham gia/rời mạng liên tục)
Phạm Văn Đức	Hệ thống truyền tin và giao tiếp	Xây dựng chat trực tiếp giữa các peer, chat nhóm, broadcast message, xử lý socket gửi/nhận dữ liệu và quản lý kết nối đồng thời	Broadcast toàn mạng, File Transfer, GUI/Web Interface
Phan Văn Hoàn	Đảm bảo độ tin cậy và bảo mật	Xây dựng kết nối mạng đa thiết bị qua Tailscale. Xử lý ACK, retry khi gửi lỗi, timeout detection, phát hiện mất kết nối, fault tolerance và quản lý lỗi hệ thống	Store-and-forward messaging, Encryption (mã hóa tin nhắn)

MỤC LỤC

PHÂN CHIA CÔNG VIỆC	2
MỤC LỤC	3
DANH MỤC HÌNH ẢNH	6
DANH MỤC BẢNG BIỂU.....	8
DANH SÁCH TỪ VIẾT TẮT	9
CHƯƠNG I. GIỚI THIỆU VỀ HỆ THỐNG CHAT P2P	11
1.1 Đặt vấn đề.....	11
1.2 Mục tiêu đề tài	11
1.2.1 Mục tiêu tổng quát.....	11
1.2.2 Mục tiêu cụ thể.....	12
1.3 Phạm vi và phương pháp thực hiện	12
1.3.1 Phạm vi đề tài.....	12
1.3.2 Phương pháp thực hiện.....	12
CHƯƠNG II. CƠ SỞ LÝ THUYẾT CỦA HỆ THỐNG CHAT P2P	14
2.1 Tổng quan hệ thống phân tán và mô hình P2P.....	14
2.1.1 Hệ thống phân tán	14
2.1.2 Mô hình Peer-to-Peer	14
2.1.3 So sánh P2P và Client–Server.....	16
2.2 TCP Socket và đa luồng	16
2.2.1 TCP Socket.....	16
2.2.2 Giao tiếp giữa các tiến trình	17
2.2.3 Xử lý đa kết nối và đa luồng	17
2.3 Các vấn đề kỹ thuật nền tảng.....	17
2.3.1 Bootstrap Server.....	17
2.3.2 Election Algorithm.....	18
2.3.3 Store-and-Forward	19
2.3.4 Mã hóa đầu cuối	20
CHƯƠNG III. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG CHAT P2P	23
3.1 Kiến trúc tổng thể	23
3.1.1 Mô hình Hybrid P2P với Bootstrap Server	23
3.1.2 Các thành phần và vai trò.....	24

3.2 Giao thức trao đổi thông điệp.....	24
3.2.1 Cấu trúc thông điệp	24
3.2.2 Các loại thông điệp và ý nghĩa	27
3.2.3 Quy trình trao đổi	32
3.3 Cơ chế Peer Discovery	35
3.3.1 Mô hình Khám phá tập trung	35
3.3.2 Lan truyền cấu trúc mạng kiểu đẩy	36
3.3.3 Bộ đệm định tuyến và Khả năng tự phục hồi.....	36
3.4 Cơ chế chat nhóm	36
3.4.1 Bầu cử phi tập trung bằng HRW	36
3.4.2 Giao thức Gossip và Giải quyết xung đột	37
3.4.3 Cơ chế Cập nhật Thành viên và Vai trò của COORD	37
3.4.5 Nhất quán cuối trong Control Plane.....	39
3.5 Cơ chế truyền file	40
3.5.1 Giao thức Thỏa thuận kênh truyền	40
3.5.2 Khởi tạo Luồng nhị phân và Cắt mảnh	41
3.5.3 Cơ chế Phục hồi đứt gãy	42
3.5.4 Xác thực Toàn vẹn và Đóng kết nối.....	43
3.6 Cơ chế store-and-forward& Mail box	44
3.6.1 Mục đích.....	44
3.6.2 Thành phần tham gia	44
3.6.3 Luồng gửi tin nhắn trực tiếp.....	45
3.6.4 Chuyển sang Store-and-Forward khi gửi trực tiếp thất bại	45
3.6.5 Lưu trữ tin nhắn trong Mailbox.....	46
3.6.6 Peer nhận lấy tin nhắn từ Mailbox	47
3.6.7 Outbox cục bộ tại Peer gửi.....	47
3.6.8 Retry và cập nhật trạng thái giao nhận.....	48
3.6.9 Hỗ trợ tin nhắn nhóm	48
3.6.10 Cơ Chế Store-and-Forward Qua Peer Trung Gian.....	48
3.6.11 Kết luận	50
3.7 Cơ chế mã hóa đầu cuối	50
3.8 Mô phỏng churn	51

3.8.1 Giới thiệu.....	51
3.8.2 Mô hình mô phỏng	52
3.8.3 Các thành phần chính	52
3.8.4 Mạng P2P và Topology.....	53
3.8.5 Các tham số mô phỏng.....	53
3.8.6 Các loại sự kiện	54
3.8.7 Các độ đo đầu ra.....	55
CHƯƠNG IV. TRIỂN KHAI HỆ THỐNG	57
4.1 Môi trường và công nghệ sử dụng.....	57
4.2 Cấu trúc mã nguồn và phân chia module	58
4.3 Bootstrap Server	59
4.4 Peer Node	61
4.4.1 Khởi tạo Peer.....	61
4.4.2. Giao diện Chat chính.....	63
4.5 Chat trực tiếp	64
4.5.1 Chat khi hai peer đều online.....	64
4.5.2 Chat khi một peer offline	65
4.6 Chat nhóm	66
4.7 Truyền file	69
4.8 Mailbox & Outbox	71
CHƯƠNG V. THỬ NGHIỆM HỆ THỐNG & ĐÁNH GIÁ.....	75
5.1 Kịch bản thử nghiệm	75
5.1.1 Kịch bản 1:	75
5.1.2 Kịch bản 2.....	77
5.1.3 Kịch bản 3.....	80
5.1.4 Kịch bản 4.....	83
5.2 Đánh giá.....	89
5.3 Kết luận và hướng phát triển	90
TÀI LIỆU THAM KHẢO.....	92
PHỤ LỤC	93

DANH MỤC HÌNH ẢNH

Hình 3.1 Sơ đồ kiến trúc tổng thể.....	24
Hình 3.2 Luồng đăng ký mạng	33
Hình 3.3 Luồng nhắn tin và lưu trữ tin nhắn	34
Hình 3.4 Luồng rời mạng và đào thải.....	35
Hình 3.5 Cơ chế Peer Discovery và FallBack	35
Hình 3.6 Giao thức Gossip và giải quyết xung đột	37
Hình 3.7 Cơ chế cập nhập thành viên trong nhóm	39
Hình 3.8 Luồng hoạt động của đuổi thành viên khỏi nhóm khi offline	40
Hình 3.9 Luồng Thỏa thuận trước khi truyền file	41
Hình 3.10 Khởi tạo luồng nhị phân và cắt mảnh.....	42
Hình 3.11 Cơ chế xác thực toàn vẹn khi gửi file.....	43
Hình 3.12 Luồng gửi tin nhắn trực tiếp	45
Hình 3.13 Luồng đóng gói và gửi lên mailbox khi gửi tin trực tiếp thất bại	46
Hình 3.14 Luồng peer lấy tin nhắn bị lỡ từ mailbox	47
Hình 3.15 Sơ đồ tuần tự trường hợp dùng relay.....	49
Hình 3.16 Cơ chế mã hóa đầu cuối	51
Hình 4.1 Giao diện dashboard của Bootstrap Server	60
Hình 4.2 Trạng thái của các Peer có thể theo dõi trên Dashboard của Bootstrap Server	60
Hình 4.3 Giao diện trang đăng ký peer.....	61
Hình 4.4 Khi đã đăng ký có thể vào lại nhanh bằng nút Open.....	62
Hình 4.5 Giao diện chat chính	63
Hình 4.6 Gửi tin nhắn khi 2 peer đều online	64
Hình 4.7 Chat cho một peer offline	65
Hình 4.8 Nhận được tin nhắn sau khi peer online lại	66
Hình 4.8a Tạo nhóm	67
Hình 4.8b Thêm thành viên vào nhóm	67
Hình 4.9 Tạo nhóm khi thành viên hoan đang offline.....	67
Hình 4.10 Khi hoan online thì đã vào được nhóm	68
Hình 4.11 Gửi file qua tin nhắn riêng.....	69

Hình 4.12 Bên nhận đã nhận được tin nhắn file gửi.....	70
Hình 4.13 Khi ấn nút Download thì sẽ tiến hành tải file về máy	71
Hình 4.14 Trạng thái tin nhắn khi gửi cho người offline	72
Hình 4.15 Hệ thống ghi nhận Log đã lưu tin nhắn tại Mailbox Server.....	72
Hình 4.16 Peer nhận được tin nhắn sau khi online trở lại	73
Hình 4.17: Hệ thống in ra Log đã gửi tin nhắn.	73
Hình 4.18 Tin nhắn đã được mã hóa nằm trong Database của Mailbox	74
Hình 5.1 Giao diện đăng ký peer	75
Hình 5.2 Giao diện peer khởi tạo khi Bootstrap Server bị sập.....	76
Hình 5.3 Các peer giao tiếp khi Bootstrap Server sập.....	76
Hình 5.4 Giao diện sau khi Bootstrap Server khởi động lại.....	77
Hình 5.5 VanDuc đuổi hoan ra khỏi nhóm.....	78
Hình 5.6 Hoan bị xóa khỏi nhóm và thành viên bầu lại COORD	79
Hình 5.7 Hoan online và không còn thấy nhóm nữa	80
Hình 5.8. Peer gửi (hoan) chuyển tiếp tin nhắn qua Peer trung gian (VanDuc) do Peer nhận (Hoang) và Mailbox đều offline	82
Hình 5.9. Peer nhận (Hoang) nhận tin nhắn thành công từ Peer trung gian (VanDuc) dù Peer gửi (hoan) đã offline	83
Hình 5.10 Khởi tạo P2P gồm 15 peer.....	85
Hình 5.11 Log vòng đời churn của 15 peer trong 100 giây mô phỏng	87
Hình 5.12 Thống kê mô phỏng churn.....	87
Hình 5.13 Kết quả 3 chỉ số	88

DANH MỤC BẢNG BIỂU

Bảng 2.1 Các mô hình P2P thông dụng.....	15
Bảng 2.2 So sánh giữa mô hình P2P và mô hình Client-Server.....	16
Bảng 2.3 Ưu điểm và nhược điểm của Bootstrap Server	18
Bảng 2.4 So sánh thuật toán Bully Algorithm và Ring Algorithm	18
Bảng 2.5 So sánh thuật toán mã hóa AES và RSA	22
Bảng 3.1 Các yêu cầu và giải pháp cho mô hình Hybrid P2P.....	23
Bảng 3.2 Các thành phần và vai trò của chúng trong P2PChat.....	24
Bảng 3.3 Các thông điệp trong nhóm Quản trị hạ tầng & Khám phá mạng	27
Bảng 3.4 Các thông điệp trong nhóm Truyền tải dữ liệu	29
Bảng 3.5 Các thông điệp trong nhóm Quản lý Nhóm Serverless.....	30
Bảng 3.6 Các thông điệp trong Nhóm Truyền tải File luồng riêng.....	31
Bảng 3.7 Thành phần tham gia quá trình store-and-forward	44
Bảng 3.8 Các thành phần.....	52
Bảng 3.9 Các loại Topology	53
Bảng 3.10 Các tham số mô phỏng churn.....	53
Bảng 3.11 Các sự kiện trong mô phỏng churn	54
Bảng 3.12 Đánh giá ngưỡng của chỉ số MDR.....	55
Bảng 3.13 Đánh giá ngưỡng của chỉ số CT.....	56
Bảng 3.14 Kích thước các loại gói tin	56
Bảng 3.15 Đánh giá ngưỡng của chỉ số CT.....	56
Bảng 4.1 Môi trường và công nghệ sử dụng	57
Bảng 5.1 Hướng phát triển của hệ thống trong tương lai.....	91

DANH SÁCH TỪ VIẾT TẮT

Từ viết tắt	Từ đầy đủ	Ý nghĩa
ACK	Acknowledgement	Tín hiệu xác nhận một gói dữ liệu P2P đã đến đích an toàn.
AES	Advanced Encryption Standard	Thuật toán mã hóa đối xứng, tốc độ nhanh, phù hợp để mã hóa nội dung tin nhắn có kích thước lớn.
CAP	Consistency, Availability, Partition tolerance	Định lý giới hạn: Không thể đồng thời đảm bảo cả tính Nhất quán, Sẵn sàng, và Chịu phân tách.
COORD	Coordinator	Node điều phối (máy chủ ảo) được bầu ra để quản lý trạng thái nhóm trong mạng Serverless.
DHT	Distributed Hash Table	Bảng băm phân tán, dùng để tổ chức và tìm kiếm peer có hệ thống.
DTO	Data Transfer Object	Đối tượng chuyển giao dữ liệu, hệ thống sử dụng pattern "Fat DTO" cho cấu trúc thông điệp.
E2EE	End-to-End Encryption	Mã hóa đầu cuối: Tin nhắn được mã hóa tại người gửi và chỉ giải mã tại người nhận hợp lệ.
HRW	Highest Random Weight (Rendezvous Hashing)	Thuật toán Băm trọng số ngẫu nhiên, được dùng để bầu cử phi tập trung Coordinator nhóm.
IPC	Inter-Process Communication	Giao tiếp giữa các tiến trình trên cùng một máy (thường qua Shared Memory)
P2P	Peer-to-Peer	Mô hình kiến trúc phân tán trong đó mỗi nút mạng vừa là client vừa là server
RSA	Rivest–Shamir–Adleman (Algorithm)	Thuật toán mã hóa bất đối xứng, dùng để mã hóa khóa đối xứng (như khóa AES)

TCP	Transmission Control Protocol	Giao thức truyền tải hướng kết nối, đáng tin cậy, đảm bảo thứ tự và toàn vẹn dữ liệu
TTL	Time To Live	Thời gian sống (hết hạn) tối đa, áp dụng cho tin nhắn chờ giao tại Mailbox Server.
WAL	Write-Ahead Logging	Chế độ ghi nhật ký trước của SQLite, được dùng để đảm bảo thread safety

CHƯƠNG I. GIỚI THIỆU VỀ HỆ THỐNG CHAT P2P

1.1 Đặt vấn đề

Trong những năm gần đây, nhu cầu giao tiếp trực tuyến ngày càng tăng cao, đặc biệt với sự bùng nổ của các ứng dụng nhắn tin tức thì. Hầu hết các ứng dụng chat hiện tại như Facebook Messenger, Zalo, Telegram đều sử dụng mô hình Client–Server, tức là mọi tin nhắn đều phải đi qua một server trung tâm. Mô hình này có những hạn chế đáng kể:

- Khi server trung tâm gặp sự cố, toàn bộ hệ thống ngừng hoạt động. Tất cả người dùng đều không thể giao tiếp.
- Tin nhắn của người dùng được lưu trữ và xử lý tại server trung tâm, tiềm ẩn rủi ro về bảo mật và quyền riêng tư.
- Server trung tâm cần được duy trì liên tục với hạ tầng mạnh mẽ, tốn kém chi phí băng thông và năng lượng.
- Khi số lượng người dùng tăng đột biến, server trung tâm phải xử lý khối lượng lớn kết nối đồng thời, dẫn đến nghẽn cổ chai.

Xuất phát từ thực tiễn đó, mô hình P2P với mỗi nút mạng vừa là client vừa là server ra đời như một giải pháp thay thế. Mô hình P2P cho phép các peer trao đổi thông điệp trực tiếp với nhau mà không cần thông qua server trung tâm, khắc phục được nhiều nhược điểm của mô hình Client–Server truyền thống.

1.2 Mục tiêu đề tài

1.2.1 Mục tiêu tổng quát

Xây dựng một hệ thống chat ngang hàng (P2P Chat) hoàn chỉnh, cho phép nhiều người dùng trao đổi tin nhắn trực tiếp qua mạng mà không phụ thuộc hoàn toàn vào server trung tâm. Hệ thống cần đảm bảo khả năng giao tiếp tin nhắn trực tiếp, nhóm, broadcast, truyền tệp tin, đồng thời cung cấp giao diện web trực quan và khả năng chịu lỗi khi peer offline hoặc crash.

1.2.2 Mục tiêu cụ thể

Đề tài hướng tới việc xây dựng một hệ thống chat ngang hàng (P2P Chat) cho phép các peer tham gia và giao tiếp trực tiếp với nhau thông qua mạng máy tính mà không phụ thuộc hoàn toàn vào server trung tâm. Hệ thống hỗ trợ cơ chế peer discovery để các peer có thể tìm kiếm và kết nối với nhau khi tham gia mạng.

Hệ thống được xây dựng theo mô hình P2P, trong đó mỗi peer vừa đóng vai trò client vừa đóng vai trò server. Các peer có khả năng gửi và nhận dữ liệu đồng thời thông qua giao tiếp mạng bằng TCP Socket hoặc giao thức tương đương.

Đề tài triển khai các chức năng chính như chat trực tiếp giữa các peer, chat nhóm, broadcast message và truyền tệp tin giữa các peer nhằm đáp ứng nhu cầu trao đổi thông tin trong mạng phân tán.

Ngoài ra, hệ thống cần quản lý trạng thái online/offline của các peer, hỗ trợ xử lý lỗi kết nối và đảm bảo việc truyền tin đáng tin cậy khi peer mất kết nối hoặc rời mạng đột ngột.

Bên cạnh đó, đề tài hướng tới việc xây dựng giao diện web hoặc GUI trực quan, đồng thời áp dụng các kiến thức về hệ thống phân tán như peer discovery, quản lý trạng thái phân tán và khả năng chịu lỗi cơ bản trong mạng P2P.

1.3 Phạm vi và phương pháp thực hiện

1.3.1 Phạm vi đề tài

Phạm vi triển khai của hệ thống được thực hiện trong phạm vi môi trường mạng LAN, có thể mở rộng ra mạng Internet thông qua Tailscale VPN. Hệ thống được triển khai và thử nghiệm trên các máy tính cá nhân sử dụng hệ điều hành Windows và Linux, sử dụng Docker để container hóa các thành phần. Các peer có thể hoạt động trên cùng một mạng LAN hoặc ở các mạng LAN khác nhau thông qua mạng riêng ảo Tailscale.

1.3.2 Phương pháp thực hiện

- Nghiên cứu tài liệu: Tìm hiểu về mô hình P2P, các giao thức truyền thông trong hệ thống phân tán và các mô hình chat nhóm phân tán.
- Phân tích yêu cầu: Xác định các yêu cầu chức năng và phi chức năng, xây dựng biểu đồ use-case và biểu đồ hoạt động.

- Thiết kế hệ thống: Thiết kế kiến trúc tổng thể, giao thức truyền thông, cấu trúc cơ sở dữ liệu, và các cơ chế xử lý lỗi.
- Cài đặt: Triển khai Bootstrap Server, Peer Node, Web UI.
- Kiểm thử: Thử nghiệm các kịch bản: peer tham gia/rời mạng, chat trực tiếp, chat nhóm, truyền file, và churn test (peer crash rồi quay lại).
- Đóng gói và triển khai: Đóng gói bằng Docker, viết script quản lý.

CHƯƠNG II. CƠ SỞ LÝ THUYẾT CỦA HỆ THỐNG CHAT P2P

2.1 Tổng quan hệ thống phân tán và mô hình P2P

2.1.1 Hệ thống phân tán

Hệ thống phân tán (Distributed System) là một tập hợp các máy tính độc lập được kết nối qua mạng, hoạt động như một hệ thống thống nhất từ góc nhìn người dùng. Các máy tính trong hệ thống phân tán giao tiếp và phối hợp với nhau thông qua việc truyền thông điệp.

Đặc điểm của hệ thống phân tán:

- Tính trong suốt (Transparency): Người dùng không nhận biết được rằng hệ thống gồm nhiều máy tính kết nối với nhau.
- Khả năng mở rộng (Scalability): Có thể thêm hoặc bớt nút mạng mà không ảnh hưởng đến hoạt động của toàn hệ thống.
- Tính chịu lỗi (Fault Tolerance): Hệ thống tiếp tục hoạt động khi một hoặc một số nút gặp sự cố.
- Tính đồng thời (Concurrency): Nhiều người dùng có thể truy cập và sử dụng hệ thống cùng một lúc.
- Các thách thức trong hệ thống phân tán:
- Clock Skew và Clock Drift: Các máy tính có đồng hồ vật lý không đồng bộ, dẫn đến khó xác định thứ tự sự kiện.
- CAP Theorem: Không thể đồng thời đảm bảo cả tính nhất quán (Consistency), tính sẵn sàng (Availability), và tính chịu phân tách (Partition tolerance). Hệ thống P2PChat ưu tiên Availability và Partition tolerance.
- Xử lý trùng lặp và lỗi mạng: Cần cơ chế retry, ACK, và idempotency.

2.1.2 Mô hình Peer-to-Peer

P2P là một mô hình kiến trúc phân tán trong đó mỗi nút mạng (peer) vừa đóng vai trò là người cung cấp dịch vụ (server) vừa là người sử dụng dịch vụ (client). Các peer trao đổi trực tiếp với nhau mà không cần thông qua một trung gian tập trung.

Bảng 2.1 Các mô hình P2P thông dụng

Loại	Mô tả	Ví dụ
Pure P2P	Không có server trung tâm. Peer tìm nhau qua flooding hoặc DHT.	Gnutella, Kademia
Hybrid P2P	Có server hỗ trợ (bootstrap/index) nhưng tin nhắn đi trực tiếp P2P.	Skype (cũ), P2PChat
Structured P2P	Sử dụng DHT để tổ chức và tìm kiếm peer có hệ thống.	Chord, Pastry, CAN

P2PChat thuộc loại Hybrid P2P: Bootstrap Server đóng vai trò bootstrap/discovery, nhưng sau khi peers đã biết nhau, chúng giao tiếp trực tiếp mà không qua Bootstrap.

Ưu điểm của mô hình P2P:

- Không có điểm chết đơn (no single point of failure) ngay cả khi một peer gặp sự cố, các peer khác vẫn giao tiếp được.
- Tận dụng tài nguyên phân tán: băng thông, CPU, bộ nhớ của tất cả các peer.
- Khả năng mở rộng tự nhiên: khi thêm peer, hệ thống có thêm tài nguyên.
- Quyền riêng tư tốt hơn: tin nhắn đi trực tiếp, không qua server trung tâm.

Nhược điểm của mô hình P2P:

- Khó đảm bảo tính nhất quán cao.
- NAT và firewall có thể cản trở kết nối trực tiếp.
- Khó quản lý và giám sát tập trung.
- Mỗi peer có năng lực xử lý và băng thông hạn chế.

2.1.3 So sánh P2P và Client-Server

Bảng 2.2 So sánh giữa mô hình P2P và mô hình Client-Server

Tiêu chí	Peer-to-Peer	Client-Server
Mô hình kiến trúc	Phân tán	Tập trung
Server trung tâm	Không bắt buộc (Hybrid P2P có bootstrap server)	Bắt buộc
Tin nhắn đi qua server	Chỉ trong giai đoạn discovery	Luôn luôn
Chi phí vận hành	Thấp (tài nguyên phân tán)	Cao (cần server mạnh)
Quyền riêng tư	Cao hơn	Thấp hơn
Khả năng mở rộng	Mở rộng tự nhiên	Phục thuộc vào server
Tính nhất quán	Khó đảm bảo hơn	Dễ đảm bảo
Triển khai	Phức tạp hơn	Đơn giản
Điểm chết đơn	Không	Có (server là điểm chết đơn)

Lựa chọn của P2PChat: Hệ thống sử dụng mô hình Hybrid P2P, Bootstrap Server chỉ đóng vai trò hỗ trợ peer discovery, không nằm trên đường truyền tin nhắn chính. Điều này kết hợp ưu điểm của cả hai mô hình: dễ triển khai (nhờ Bootstrap Server) nhưng vẫn đảm bảo tính phân tán cho việc truyền tin nhắn.

2.2 TCP Socket và đa luồng

2.2.1 TCP Socket

TCP là giao thức truyền tải hướng kết nối, đáng tin cậy, đảm bảo thứ tự và toàn vẹn dữ liệu. TCP được sử dụng làm giao thức truyền thông chính trong hệ thống P2PChat vì các đặc tính: Reliability - TCP đảm bảo dữ liệu đến đích, không mất mát, không trùng lặp, đúng thứ tự nhờ cơ chế ACK và sequence number; Flow Control - cơ chế sliding window kiểm soát tốc độ gửi để tránh tràn bộ đệm; Congestion Control - TCP giảm tốc

độ khi phát hiện nghẽn mạng; Connection-oriented - trước khi truyền dữ liệu, hai bên phải thiết lập kết nối qua cơ chế three-way handshake.

Trong P2PChat, mỗi peer mở một ServerSocket (TCP server) để lắng nghe kết nối đến, đồng thời sử dụng Socket (TCP client) để kết nối đến các peer khác. Mỗi peer vừa là client vừa là server.

2.2.2 Giao tiếp giữa các tiến trình

Trong hệ thống phân tán, các tiến trình giao tiếp với nhau qua hai cơ chế chính. Message Passing (Truyền thông điệp): các tiến trình trao đổi dữ liệu qua các thông điệp được gửi qua mạng, thường dùng cho giao tiếp giữa các máy tính khác nhau; trong P2PChat: mỗi thông điệp chat, group message, heartbeat đều là message passing qua TCP. IPC qua Shared Memory: các tiến trình trên cùng một máy chia sẻ vùng nhớ chung, trong P2PChat: các thread trong cùng một peer giao tiếp qua đối tượng Java shared (ví dụ: PeerManager, GroupCache được truyền giữa các thread).

2.2.3 Xử lý đa kết nối và đa luồng

Trong một peer, có nhiều kết nối TCP đồng thời và nhiều hoạt động đồng thời (nhận tin nhắn, gửi tin nhắn, heartbeat, giao diện web). P2PChat sử dụng mô hình thread-per-connection kết hợp Executor Service.

Thread Safety được đảm bảo qua: ConcurrentHashMap cho danh sách peer online; AtomicLong cho Lamport Clock; CopyOnWriteArrayList cho các danh sách cần duyệt an toàn; SQLite với chế độ WAL đảm bảo thread safety.

2.3 Các vấn đề kỹ thuật nền tảng

2.3.1 Bootstrap Server

Bootstrap Server là một máy chủ có địa chỉ cố định, đóng vai trò điểm vào duy nhất giúp peer mới tìm được các peer khác trong mạng. Sau khi peer đã biết nhau, Bootstrap không tham gia vào quá trình truyền tin. Nó có một số vai trò sau:

- Lưu danh sách các peer đang online
- Cập nhật danh sách đó cho peer mới tham gia
- Theo dõi trạng thái peer qua heartbeat

Bootstrap Server không phải server trung tâm theo nghĩa Client-Server truyền thống. Nó không xử lý, không relay tin nhắn, không lưu trữ nội dung. Chỉ đơn thuần là "danh bạ" của mạng.

Bảng 2.3 Ưu điểm và nhược điểm của Bootstrap Server

Ưu điểm	Nhược điểm
Đơn giản, dễ triển khai Peer tìm kiếm nhau nhanh chóng Không tạo nút thắt cổ chai cho tin nhắn	Là điểm tập chung khi khởi động Nếu sập thì các peer mới không thể tham gia vào được Cần được duy trì hoạt động liên tục để cập nhập trạng thái các peer

2.3.2 Election Algorithm

Election Algorithm (thuật toán bầu chọn) là cơ chế giúp các node trong hệ thống phân tán tự động bầu ra một node đóng vai trò điều phối mà không cần sự can thiệp từ bên ngoài.

Trong hệ thống phân tán, nhiều tác vụ cần một node đứng ra điều phối (COORD): tin nhắn nhóm, quản lý tài nguyên chung, lưu trữ trạng thái. Khi những COORD hiện tại sập, hệ thống cần tự động bầu người thay thế để quá trình cập nhập trạng thái nhóm không bị gián đoạn.

Có 2 loại thuật toán bầu chọn phổ biến là: Bully Algorithm và Ring Algorithm

Bảng 2.4 So sánh thuật toán Bully Algorithm và Ring Algorithm

Tiêu chí	Bully Algorithm	Ring Algorithm
Nguyên tắc	Node ID cao nhất tự xưng COORD khi không có phản hồi từ node ID cao hơn	Thông điệp đi vòng quanh ring, node ID cao nhất trong danh sách được bầu
Tốc độ	Nhanh hơn — node thắng tự xác định ngay khi timeout	Chậm hơn — phải chờ thông điệp đi hết một vòng ring
Số thông điệp	Nhiều	Ít hơn
Độ phức tạp	Đơn giản — chỉ cần so sánh ID và gửi broadcast	Phức tạp hơn — phải duy trì cấu trúc vòng tròn logic
Phù hợp	Mạng nhỏ, ít peer, cần bầu chọn nhanh	Mạng lớn hơn, cần tiết kiệm băng thông

Do nhóm sử dụng cơ chế này chủ yếu trong chat nhóm, nơi có số lượng peer thường không quá lớn nên chọn dạng Bully Algorithm là phù hợp và dễ triển khai nhất.

2.3.3 Store-and-Forward

Store-and-Forward là một cơ chế truyền dữ liệu trong mạng máy tính, trong đó mỗi nút trung gian sẽ nhận toàn bộ gói tin hoặc thông điệp, lưu tạm thời vào bộ nhớ đệm, sau đó mới chuyển tiếp đến nút kế tiếp trên đường đi đến đích. Cơ chế này thường được sử dụng trong các hệ thống truyền thông không đồng bộ, mạng chuyển mạch gói, email, hàng đợi tin nhắn và một số mô hình mạng ngang hàng.

Trong mô hình Store-and-Forward, dữ liệu không nhất thiết phải được truyền trực tiếp từ nguồn đến đích trong một lần. Thay vào đó, thông điệp có thể đi qua nhiều nút trung gian. Tại mỗi nút, hệ thống kiểm tra dữ liệu nhận được, xác định tuyến đường tiếp theo, sau đó lưu trữ và chuyển tiếp khi điều kiện truyền phù hợp. Nếu đường truyền tạm thời bị gián đoạn hoặc nút đích chưa sẵn sàng nhận dữ liệu, thông điệp vẫn có thể được lưu lại để gửi sau.

Ưu điểm chính của cơ chế Store-and-Forward là tăng độ tin cậy trong truyền thông. Vì dữ liệu được lưu tạm tại các nút trung gian, hệ thống có thể xử lý các tình huống như mất kết nối, nghẽn mạng hoặc nút nhận chưa hoạt động. Cơ chế này cũng giúp giảm yêu cầu phải có kết nối liên tục giữa bên gửi và bên nhận.

Tuy nhiên, Store-and-Forward cũng có một số hạn chế. Do mỗi nút trung gian phải nhận đủ dữ liệu rồi mới chuyển tiếp, độ trễ truyền tin có thể tăng lên, đặc biệt khi thông điệp có kích thước lớn hoặc phải đi qua nhiều nút. Ngoài ra, các nút trung gian cần có bộ nhớ để lưu trữ tạm thời dữ liệu, đồng thời phải có cơ chế quản lý hàng đợi và xử lý lỗi phù hợp.

Trong hệ thống P2P Chat, Store-and-Forward có thể được áp dụng để hỗ trợ việc gửi tin nhắn khi người nhận đang ngoại tuyến hoặc khi kết nối trực tiếp giữa hai peer không khả dụng. Tin nhắn sẽ được lưu tạm tại một peer trung gian hoặc một thành phần lưu trữ, sau đó được chuyển tiếp đến người nhận khi người đó kết nối lại. Nhờ đó, hệ thống có thể cải thiện khả năng truyền tin và đảm bảo tin nhắn không bị mất trong trường hợp kết nối mạng không ổn định.

Trong project P2PChat, Store-and-Forward là cơ chế giúp tin nhắn vẫn được đảm bảo giao đến người nhận ngay cả khi peer nhận đang offline hoặc kết nối trực tiếp thất bại. Cơ chế này được triển khai thông qua mailbox-server, đóng vai trò như một hộp thư trung gian để lưu trữ tạm thời các tin nhắn chưa thể giao ngay.

Project còn sử dụng outbox cục bộ tại peer gửi để lưu trạng thái các tin nhắn đang chờ xử lý. Nhờ đó, nếu việc gửi trực tiếp hoặc gửi vào mailbox bị lỗi tạm thời, hệ thống có thể retry nên thay vì làm mất tin nhắn. Với direct message, nội dung offline được mã hóa trước khi lưu vào mailbox. Với group message, mailbox lưu danh sách các thành viên chưa nhận và theo dõi ACK riêng cho từng thành viên.

Như vậy, Store-and-Forward trong P2PChat không thay thế hoàn toàn truyền P2P trực tiếp, mà đóng vai trò cơ chế dự phòng. Tin nhắn vẫn được gửi trực tiếp khi có thể; chỉ khi người nhận không khả dụng, hệ thống mới lưu tạm vào mailbox và chuyển tiếp lại khi người nhận online. Cách làm này giúp hệ thống ổn định hơn trong môi trường P2P, nơi các peer có thể online/offline bất kỳ lúc nào.

2.3.4 Mã hóa đầu cuối

Mã hóa đầu cuối là cơ chế mã hóa đầu cuối, trong đó tin nhắn được mã hóa tại phía người gửi và chỉ được giải mã tại phía người nhận hợp lệ. Các thành phần trung gian như bootstrap server, mailbox server hoặc hạ tầng mạng chỉ tham gia truyền/lưu dữ liệu đã mã hóa, không có khóa cần thiết để đọc nội dung thật của tin nhắn.

Trong project P2PChat, E2EE dựa trên mô hình mã hóa lai, kết hợp giữa RSA và AES. RSA là thuật toán mã hóa bất đối xứng, sử dụng một cặp khóa gồm public key và private key. Public key có thể chia sẻ cho các peer khác, còn private key được giữ bí mật tại peer sở hữu. AES là thuật toán mã hóa đối xứng, dùng cùng một khóa để mã hóa và giải mã dữ liệu.

Lý do phải kết hợp RSA và AES là vì mỗi thuật toán có vai trò khác nhau. AES có tốc độ nhanh, phù hợp để mã hóa nội dung tin nhắn hoặc payload có kích thước lớn.

Tuy nhiên, AES cần một khóa bí mật chung giữa người gửi và người nhận; nếu khóa này bị gửi trực tiếp qua mạng thì có thể bị lộ. RSA giải quyết vấn đề phân phối

khóa: người gửi có thể dùng public key của người nhận để mã hóa khóa AES, và chỉ người nhận có private key tương ứng mới mở được khóa AES đó.

Vì vậy, hệ thống không mã hóa hai lần cùng một nội dung. Nội dung tin nhắn được mã hóa bằng AES, còn khóa AES được mã hóa bằng RSA. Cách làm này tận dụng ưu điểm của cả hai loại mã hóa: AES đảm bảo hiệu năng khi xử lý dữ liệu, còn RSA đảm bảo việc trao đổi khóa diễn ra an toàn.

AES là thuật toán mã hóa đối xứng. Điều này có nghĩa là cùng một khóa bí mật được dùng cho cả quá trình mã hóa và giải mã dữ liệu. Nếu người gửi dùng một khóa AES để mã hóa tin nhắn, người nhận phải có đúng khóa đó mới có thể giải mã lại nội dung ban đầu.

AES hoạt động trên các khối dữ liệu có kích thước cố định. Dữ liệu đầu vào được chia thành các khối, sau đó thuật toán thực hiện nhiều vòng biến đổi trên từng khối để tạo ra dữ liệu mã hóa. Các vòng biến đổi này gồm những bước như thay thế byte, hoán vị vị trí dữ liệu, trộn cột và kết hợp với khóa vòng. Nhờ các bước này, dữ liệu gốc bị biến đổi thành dạng khó suy luận nếu không có khóa.

Trong các hệ thống hiện đại, AES thường được dùng cùng các chế độ hoạt động như GCM. AES-GCM không chỉ mã hóa dữ liệu mà còn hỗ trợ kiểm tra tính toàn vẹn và xác thực dữ liệu. Điều này giúp phát hiện trường hợp dữ liệu bị sửa đổi trong quá trình truyền. AES có ưu điểm là tốc độ nhanh, hiệu quả với dữ liệu lớn và phù hợp để mã hóa trực tiếp nội dung tin nhắn.

RSA là thuật toán mã hóa bất đối xứng, sử dụng một cặp khóa gồm public key và private key. Public key có thể công khai cho người khác sử dụng, còn private key phải được giữ bí mật bởi chủ sở hữu. Dữ liệu được mã hóa bằng public key chỉ có thể được giải mã bằng private key tương ứng.

Cơ chế của RSA dựa trên bài toán toán học về phân tích một số lớn thành tích của hai số nguyên tố. Khi tạo khóa, hệ thống sinh ra hai số nguyên tố lớn, từ đó tính ra public key và private key. Việc mã hóa và giải mã dựa trên phép lũy thừa modulo. Về mặt thực tế, nếu không có private key, việc suy ra nội dung gốc từ dữ liệu đã mã hóa là rất khó.

RSA có ưu điểm lớn là giải quyết được bài toán phân phối khóa. Người gửi không cần chia sẻ trước một khóa bí mật với người nhận, mà chỉ cần biết public key của người nhận. Tuy nhiên, RSA có tốc độ chậm hơn AES và không phù hợp để mã hóa trực tiếp dữ liệu lớn. Vì vậy, RSA thường được dùng để mã hóa khóa đối xứng, chữ ký số hoặc xác thực danh tính, thay vì mã hóa toàn bộ nội dung tin nhắn.

Bảng 2.5 So sánh thuật toán mã hóa AES và RSA

Tiêu chí	AES	RSA
Loại mã hóa	Đối xứng	Bất đối xứng
Khóa sử dụng	Một khóa chung để mã hóa và giải mã	Một cặp khóa public và private
Tốc độ	Nhanh	Chậm hơn AES
Phù hợp	Mã hóa nội dung tin nhắn/pay load	Mã hóa khóa AES, trao đổi khóa an toàn
Vai trò trong E2EE	Mã hóa nội dung tin nhắn	Bảo vệ khóa AES bằng public key của người nhận

CHƯƠNG III. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG CHAT P2P

3.1 Kiến trúc tổng thể

3.1.1 Mô hình Hybrid P2P với Bootstrap Server

P2PChat được xây dựng theo mô hình Hybrid P2P — kết hợp giữa một máy chủ tập trung nhỏ (Bootstrap Server) đóng vai trò điều phối ban đầu và một mạng lưới ngang hàng hoàn toàn phân tán cho luồng dữ liệu chính.

Bảng 3.1 Các yêu cầu và giải pháp cho mô hình Hybrid P2P

Yêu cầu	Giải pháp
Peer mới cần biết ai đang online	Bootstrap Server giữ registry tập trung
Dữ liệu chat không đi qua trung gian	Kết nối TCP trực tiếp Peer → Peer
Hệ thống tiếp tục hoạt động khi Bootstrap sập	Cache cục bộ SQLite tại mỗi Peer
Tin nhắn không mất khi người nhận offline	Mailbox Server lưu trữ ngoài băng

Trong kiến trúc này:

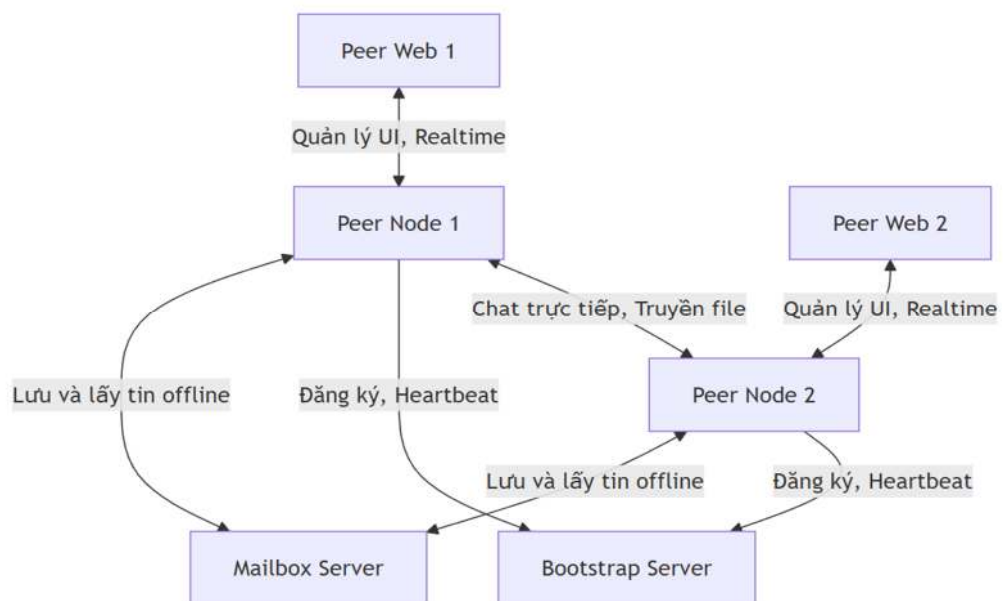
- Bootstrap Server chỉ xử lý Control Plane: đăng ký, khám phá peer, heartbeat. Nó không truyền tải bất kỳ tin nhắn nào.
- Mailbox Server đóng vai trò Store-and-Forward tách biệt: giữ tin nhắn cho peer offline và xóa sau khi giao thành công, TTL tối đa 7 ngày.
- Peer Node vừa là client (gửi tin), vừa là TCP server (nhận tin), vừa tự quản lý dữ liệu cục bộ qua SQLite.

3.1.2 Các thành phần và vai trò

Hệ thống P2PChat gồm bốn thành phần chính, được containerize độc lập và giao tiếp qua TCP/HTTP:

Bảng 3.2 Các thành phần và vai trò của chúng trong P2PChat

Thành phần	Vai trò chính
Bootstrap Server	Đăng ký peer, phân phối danh sách peer, theo dõi heartbeat, trả endpoint Mailbox
Mailbox Server	Lưu tin nhắn offline (STORE_MESSAGE), phục vụ pull khi peer online, quản lý TTL và delivery ACK
Peer Node	TCP server/client P2P, REST API, WebSocket, quản lý nhóm DHT-lite, outbox retry, mã hóa E2EE
Peer Web	Giao diện người dùng: chat, quản lý nhóm, trạng thái file transfer, realtime qua WebSocket



Hình 3.1 Sơ đồ kiến trúc tổng thể

3.2 Giao thức trao đổi thông điệp

3.2.1 Cấu trúc thông điệp

Hệ thống P2PChat sử dụng định dạng dữ liệu JSON để truyền tải qua mạng lưới. Ở tầng mã nguồn, hệ thống thiết kế theo pattern "Fat DTO" – sử dụng duy nhất một đối tượng Message để chứa tất cả các trường dữ liệu có thể có của hệ thống.

Tuy nhiên, để tối ưu hóa băng thông mạng, không phải gói tin nào cũng gửi toàn bộ các trường này. Quá trình chuyển đổi sang JSON (Serialization) sẽ tự động cắt bỏ các trường mang giá trị null. Do đó, mỗi thông điệp gửi đi cấu tạo gồm 2 phần: Cấu trúc lõi (bắt buộc) và Các trường mở rộng (tùy thuộc vào loại lệnh). Đây là cấu trúc chung của Message đó:

```
{
  "type": "DIRECT_MESSAGE",
  "messageId": "550e8400-e29b-41d4-a716-446655440000",
  "sender": "alice",
  "receiver": "bob",
  "groupName": "Nhóm Đồ Án",
  "content": "Hello Bob, file tài liệu gửi bạn nhé!",
  "timestamp": 1716715200000,
  "encryptionAlgorithm": "AES-256",
  "senderKeyId": "key-alice-01",
  "receiverKeyId": "key-bob-01",
  "groupId": "group-xyz-123",
  "lamportClock": 15,
  "cacheVersion": 2,
  "requestId": "req-999-idempotency",
  "members": ["alice", "bob", "charlie"],
  "COORDs": ["alice"],
  "version": 3,
  "changeType": "ADD",
  "affected": "bob",
  "requester": "alice",
  "newMember": "dave",
  "target": "charlie",
  "newOwner": "bob",
  "groupMode": "OPEN",
  "reason": "Từ chối vì không đủ quyền",
  "filename": "Bao_Cao.pdf",
  "fileSize": 15728640,
  "sha256": "8d969eef6ecad3c29a3a6292...",
  "filePort": 9001,
  "transferId": "transfer-abc-456",
  "totalChunks": 15,
  "resumeChunkIndex": 5,
  "chunkIndexes": [0, 1, 2, 3]
}
```

Cấu trúc lõi (Core Payload): Bất kỳ gói tin nào chạy trong mạng cũng đều phải chứa tối thiểu các thông tin định tuyến cơ bản này. Chúng gồm có:

- *type*: Phân loại thông điệp - để biết thông điệp đó dùng để làm gì
- *sender*: Định danh người gửi
- *timestamp*: Thời điểm khởi tạo (UNIX Epoch)

```
{
  "type": "...",
  "sender": "...",
  "timestamp": 1716715200000
}
```

Các biến thể gói tin thực tế (Payload Variants): Tùy vào giá trị của *type*, gói tin sẽ được đính kèm thêm các nhóm thuộc tính tương ứng:

Đầu tiên là biến thể **Nhắn tin** (Data Message): Dành cho việc chat cá nhân. Bổ sung thêm trường *người nhận (receiver)* và *nội dung (content)*.

```
{
  "type": "DIRECT_MESSAGE",
  "messageId": "123e4567-e89b-12d3...",
  "sender": "alice",
  "receiver": "bob",
  "content": "Chào Bob, tí họp nhé!"
}
```

Tiếp theo là biến thể **Điều khiển Nhóm** (Group Control): Dành cho các lệnh như đuổi người, mời vào nhóm. Chúng có thêm *mã nhóm (groupId)* để xác định nhóm.

```
{
  "type": "GROUP_KICK",
  "groupId": "group-abc-123",
  "requester": "alice", // Người ra lệnh đuổi
  "target": "charlie"  // Người bị đuổi
}
```

Cuối cùng là biến thể **Luồng truyền File** (File Transfer): Dành cho việc đàm phán gửi file dung lượng lớn bằng luồng riêng. Nó sẽ cung cấp *tên file (filename)*, *kích thước file (fileSize)* để người nhận xác định, đồng thời kèm theo *cổng kết nối (filePort)* để kéo file về máy.

```
{
  "type": "FILE_OFFER",
  "sender": "alice",
  "receiver": "bob",
  "filename": "BaoCao.pdf",
  "fileSize": 15728640,
  "filePort": 9001      // Cổng mà Alice mở sẵn để đợi Bob kết nối vào tải file
}
```

3.2.2 Các loại thông điệp và ý nghĩa

Dựa trên trường type, toàn bộ các thông điệp của hệ thống được chia thành 4 nhóm nghiệp vụ chính là Quản trị hạ tầng & Khám phá mạng, Truyền tải dữ liệu, Quản lý Nhóm Serverless và Truyền tải File luồng riêng

Bảng 3.3 Các thông điệp trong nhóm Quản trị hạ tầng & Khám phá mạng

STT	Mã thông điệp	Ý nghĩa
1	REGISTER	Lệnh máy trạm xin đăng ký và tham gia mạng lưới với máy chủ Bootstrap.
2	REGISTER_ACK	Bootstrap xác nhận cho phép tham gia và trả về dữ liệu.
3	REGISTER_NACK	Bootstrap từ chối yêu cầu đăng ký (do lỗi tên hoặc trùng lặp).
4	PEER_LIST	Gói dữ liệu chứa danh sách IP/Port của các máy đang hoạt động.
5	PEER_JOIN	Thông báo Broadcast tới toàn mạng khi có

		một máy mới vừa bật.
6	PEER_LEAVE	Thông báo Broadcast tới toàn mạng khi một máy vừa tắt/ngắt kết nối.
7	HEARTBEAT	Tín hiệu sinh tồn do máy trạm định kỳ báo lên Bootstrap.
8	HEARTBEAT_ACK	Bootstrap phản hồi xác nhận đã nhận được tín hiệu sinh tồn.
9	DISCOVER	Lệnh máy trạm chủ động xin lại danh sách mạng lưới (PEER_LIST) thủ công.
10	RESOLVE_MAILBOX	Yêu cầu Bootstrap cung cấp địa chỉ IP của Mailbox Server.
11	RESOLVE_MAILBOX_ACK	Trả lời cung cấp IP/Port của Mailbox.
12	STORE_MESSAGE	Đẩy gói tin nhắn lên Mailbox nhờ giữ hộ (do người nhận đang offline).
13	STORE_ACK	Mailbox báo đã lưu tin nhắn vào hàng đợi an toàn.
14	PULL_MESSAGES	Lệnh gửi từ máy trạm yêu cầu Mailbox xả các tin nhắn tồn đọng về.
15	PULL_RESPONSE	Gói dữ liệu chứa các chuỗi tin nhắn Offline trả về từ Mailbox.
16	DELIVERY_ACK	Máy trạm báo đã kéo tin về xong để Mailbox tiến hành xóa rác.
17	CHECK_DELIVERY	(Truy vấn) Kiểm tra xem người nhận đã kéo tin nhắn đó về chưa.
18	CHECK_DELIVERY_	Trả về trạng thái tin nhắn (đã nhận / chưa

	RESPONSE	nhận).
19	MAILBOX_QUOTA_EXCEEDED	Cảnh báo dung lượng cấp cho người dùng tại Mailbox bị đầy.
20	MAILBOX_MESSAGE_EXPIRED	Cảnh báo tin nhắn bị quá hạn thời gian lưu trữ chờ (TTL).
21	PEER_LOOKUP_REQ	Lệnh hỏi P2P để nhờ bạn bè tìm địa chỉ IP của một người (Dùng khi Bootstrap sập).
22	PEER_LOOKUP_RESP	Kết quả trả về IP được lôi ra từ bộ đệm của một máy trạm khác.

Bảng 3.4 Các thông điệp trong nhóm Truyền tải dữ liệu

STT	Mã thông điệp	Ý nghĩa
1	DIRECT_MESSAGE	Gửi tin nhắn cá nhân 1-1 (Văn bản hoặc Ảnh Base64).
2	GROUP_MESSAGE	Gửi tin nhắn nội dung vào không gian nhóm chat.
3	BROADCAST	Gửi tin nhắn phát thanh (mọi máy trạm trong mạng đều hiển thị).
4	ACK	Tín hiệu xác nhận một gói dữ liệu P2P đã đến đích an toàn.
5	SYSTEM	Tin nhắn thông báo tự động sinh ra từ hệ thống (Ví dụ: "Hoàn vừa vào mạng").
6	ERROR	Gói tin báo lỗi chung trong quá trình xử lý.

Bảng 3.5 Các thông điệp trong nhóm Quản lý Nhóm Serverless

STT	Mã thông điệp	Ý nghĩa
1	COORD_INIT	Thiết lập một máy trạm làm máy chủ ảo quản lý nhóm (COORD).
2	COORD_GOSSIP	Lan truyền (gossip) thông tin ai đang làm máy chủ ảo trên mạng.
3	COORD_RESIGN	Trưởng nhóm cũ chủ động từ chức và bàn giao quyền lực.
4	GROUP_ADD	Lệnh yêu cầu thêm người vào nhóm / Xin gia nhập nhóm.
5	GROUP_JOINED	Xác nhận đã tham gia nhóm thành công.
6	GROUP_ADD_REJECTED	Thông báo yêu cầu gia nhập bị từ chối (Không đủ quyền).
7	GROUP_LEAVE	Lệnh thông báo một thành viên chủ động tự rời khỏi nhóm.
8	GROUP_KICK	Lệnh yêu cầu trưởng nhóm đuổi một thành viên.
9	GROUP_KICKED	Thông báo tới toàn mạng rằng 1 thành viên đã chính thức bị đuổi.
10	GROUP_DISBAND	Lệnh ra quyết định giải tán hoàn toàn nhóm chat.
11	GROUP_DISBANDED	Thông báo nhóm đã giải tán, ép các máy xóa bộ nhớ Cache nhóm này.

12	GROUP_UPDATED	Lệnh yêu cầu cập nhật danh sách thành viên/quyền lực lên giao diện.
13	GROUP_RESYNC_REQ	Yêu cầu máy chủ ảo cung cấp dữ liệu gốc để sửa lỗi lệch bộ đệm (Cache).
14	GROUP_RESYNC_RESP	Trả về dữ liệu gốc để các máy thành viên sửa lỗi bộ đệm.
15	CACHE_STALE	Cảnh báo dữ liệu nhóm tại máy nội bộ đã bị cũ/lệch pha.
16	GROUP_HISTORY_DIGEST	Mã băm (Hash) tóm tắt lịch sử chat P2P để đối chiếu xem có bị mất tin nào không.
17	GROUP_HISTORY_REQUEST	Yêu cầu đồng bộ lại (xin lại) các đoạn chat nhóm bị thiếu lúc tắt máy.
18	GROUP_HISTORY_RESPONSE	Trả về các tin nhắn nhóm bị thiếu từ máy của bạn bè (P2P Relay).

Bảng 3.6 Các thông điệp trong Nhóm Truyền tải File luồng riêng

STT	Mã thông điệp	Ý nghĩa
1	FILE_OFFER	Đề nghị đàm phán gửi file cho một người (Gửi thông số file/cổng Port).
2	FILE_ACCEPT	Chấp thuận đề nghị, sẵn sàng mở luồng tải file.
3	FILE_REJECT	Từ chối không nhận file.
4	FILE_DONE	Tín hiệu báo luồng byte data thô đã truyền tải hoàn tất.

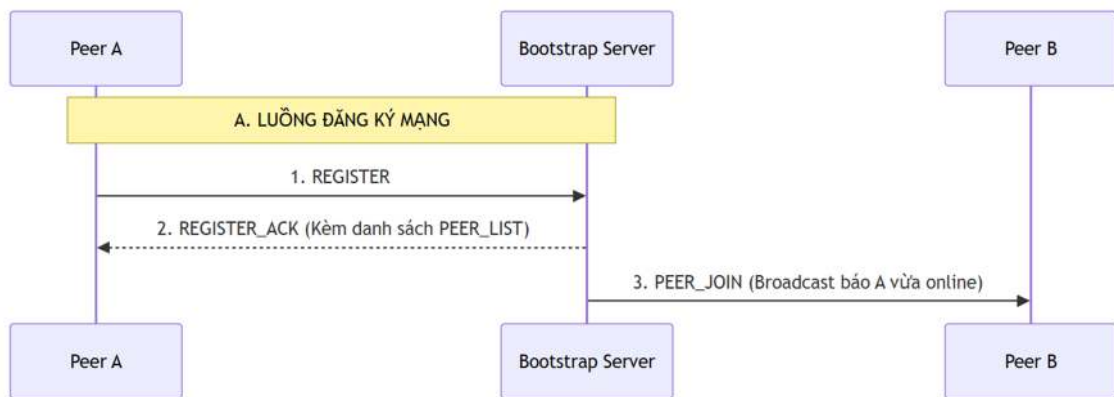
5	FILE_ACK	Tín hiệu xác nhận file đã được ráp và lưu an toàn vào ổ cứng.
6	FILE_GROUP_OFFER	Đề nghị đàm phán gửi một file cho tập thể nhóm chat.
7	FILE_HAVE_REQ	Xin bản đồ phân mảnh (Chunk list) file mà bạn bè đang có.
8	FILE_HAVE_RESP	Phản hồi danh sách các mảnh file đã tải xong, sẵn sàng để chia sẻ.
9	FILE_RESUME	Lệnh yêu cầu tải nối tiếp (Resume) một file đang bị đứt quãng.

3.2.3 Quy trình trao đổi

Quy trình giao tiếp mạng là sự kết hợp chặt chẽ giữa mô hình Server-Client (khi giao tiếp với Bootstrap để lấy thông tin) và mô hình mạng P2P thuần túy (khi hai máy trạm truyền dữ liệu cho nhau). Dưới đây là 3 luồng trao đổi cốt lõi:

A. Luồng Đăng ký & Khám phá mạng (Peer Discovery):

1. Peer mở kết nối TCP đến máy chủ Bootstrap và gửi lệnh REGISTER.
2. Bootstrap xác thực kết nối, lưu Peer vào bộ nhớ và trả về gói REGISTER_ACK (bên trong chứa thông tin IP/Port của toàn bộ mạng lưới hiện tại).
3. Cùng lúc đó, Bootstrap lập tức broadcast thông điệp PEER_JOIN tới tất cả các máy trạm khác để thông báo: "Có một thành viên mới vừa online".



Hình 3.2 Luồng đăng ký mạng

B. Luồng Nhắn tin & Lưu trữ (Messaging & Store-and-Forward):

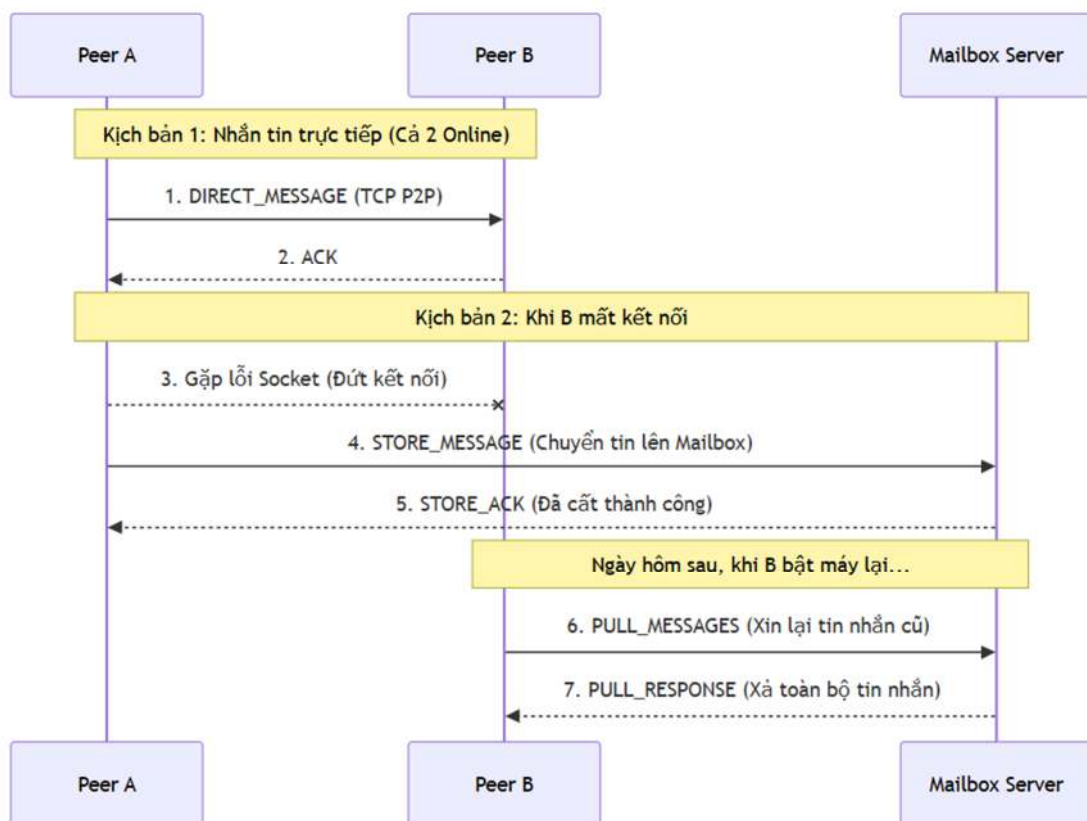
Luồng này có 2 kịch bản:

Kịch bản 1: Nhắn tin trực tiếp (Cả 2 đang online):

1. Peer A kết nối thẳng TCP đến IP của Peer B và đẩy lệnh `DIRECT_MESSAGE`.
2. Peer B nhận tin, giải mã JSON và gửi trả ngay một gói `ACK`.

Kịch bản 2: Nhắn tin khi bên nhận offline (Store-and-Forward):

1. Peer A nhắn tin nhưng Peer B đã tắt máy, Socket lập tức báo lỗi đứt kết nối.
2. Nhờ cơ chế Fallback, Peer A sẽ tự động bọc gói tin đó thành loại `STORE_MESSAGE` và vớt lên Mailbox Server nhờ giữ hộ (Mailbox sẽ phản hồi `STORE_ACK`).
3. Khi Peer B mở máy lại vào hôm sau, B chủ động gửi `PULL_MESSAGES` lên Mailbox. Các tin nhắn kẹt lại sẽ được trút xuống máy B an toàn. B xác nhận bằng `DELIVERY_ACK` để hệ thống dọn rác.

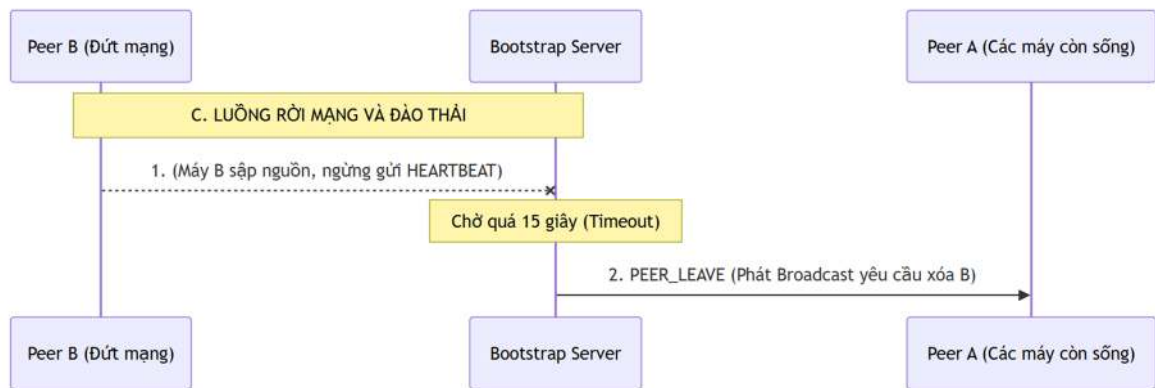


Hình 3.3 Luồng nhắn tin và lưu trữ tin nhắn

C. Luồng Rời mạng & Đào thải

Mạng lưới có cơ chế tự động dọn dẹp các kết nối "chết" để tránh lỗi giệt lag.

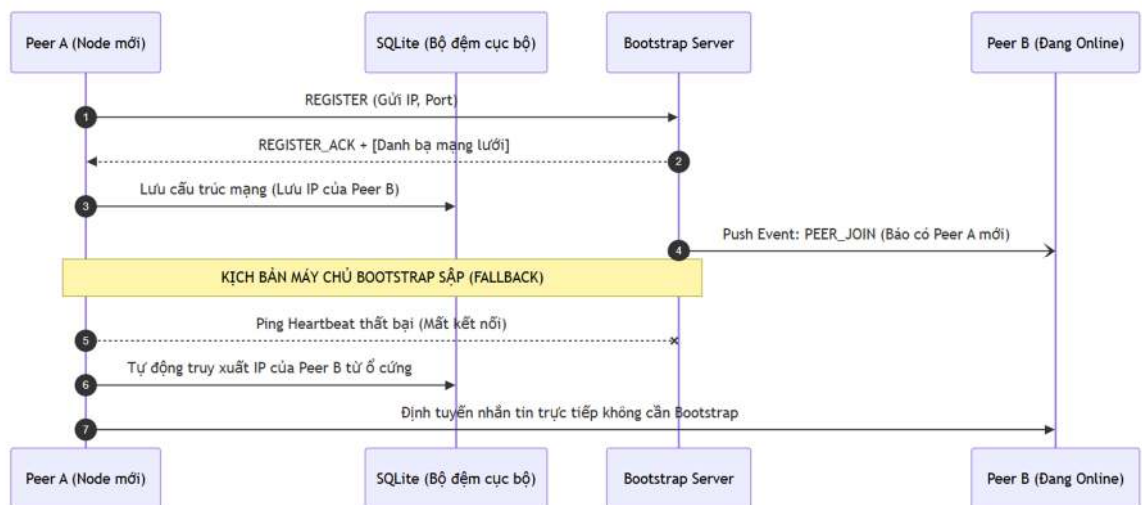
1. Mỗi 5 giây, các máy trạm phải tự động gửi lệnh HEARTBEAT lên Bootstrap để báo hiệu mình còn sống.
2. Nếu người dùng rút điện hoặc tắt app đột ngột, nhịp Heartbeat dừng lại. Chờ quá 15 giây, Bootstrap chính thức liệt máy đó vào trạng thái "Dead".
3. Bootstrap gửi Broadcast lệnh PEER_LEAVE đi toàn hệ thống. Mọi máy trạm nhận được lệnh này sẽ lập tức xóa máy trạm đã chết ra khỏi bộ nhớ nội bộ, chấm dứt toàn bộ giao dịch.



Hình 3.4 Luồng rời mạng và đào thải

3.3 Cơ chế Peer Discovery

Cơ chế Khám phá mạng lưới (Peer Discovery) sử dụng mô hình Hybrid P2P, trong đó một Bootstrap Server đóng vai trò như một thư mục trung tâm để quản lý danh bạ mạng lưới, nhưng hoàn toàn không can thiệp vào luồng dữ liệu giữa các node.



Hình 3.5 Cơ chế Peer Discovery và FallBack

3.3.1 Mô hình Khám phá tập trung

Để giải quyết bài toán khởi tạo mạng, hệ thống thiết kế một Directory Node (Máy chủ Bootstrap) đóng vai trò như một danh bạ hội tụ. Thay vì một máy trạm mới phải gửi gói tin dò tìm mù quáng (Ping/Pong) đi khắp nơi, nó chỉ cần một điểm neo (Anchor) duy nhất là Bootstrap. Tại đây, cấu trúc dữ liệu PeerRegistry trên RAM của Bootstrap

sẽ lập tức lập bản đồ định danh của người dùng với IP/Port thực tế. Mô hình này giúp thời gian khám phá toàn bộ mạng lưới được tối ưu hóa xuống mức $O(1)$.

3.3.2 Lan truyền cấu trúc mạng kiểu đẩy

Khám phá mạng không chỉ là việc một máy mới tìm thấy mạng, mà còn là mạng lưới phải phát hiện ra máy mới. Hệ thống thiết kế cơ chế cập nhật bằng mô hình Hướng sự kiện (Event-Driven). Thay vì bắt các máy trạm (Client) phải liên tục hỏi Bootstrap xem có ai mới vào không (Pull-based polling - rất tốn tài nguyên), hệ thống áp dụng cơ chế Đẩy (Push-based). Mỗi khi có biến động Topology (một Node bật lên hoặc tắt đi), Bootstrap đóng vai trò là quạt phát sóng (Fan-out), đẩy ngay các sự kiện PEER_JOIN hoặc PEER_LEAVE xuống toàn bộ đồ thị mạng. Các máy khách nhờ đó tự động duy trì được Bảng định tuyến (Routing Table) cục bộ theo thời gian thực.

3.3.3 Bộ đệm định tuyến và Khả năng tự phục hồi

Bài toán đặt ra: Nếu máy chủ Bootstrap sập, làm sao các Peer khám phá ra nhau? Kiến trúc hệ thống không phụ thuộc 100% vào Bootstrap. Mỗi máy trạm được tích hợp một bộ đệm cục bộ (RecentPeersCache) gắn với cơ sở dữ liệu SQLite (known_peers).

- Mỗi khi khám phá ra một Peer mới và có tương tác, máy trạm sẽ lưu vĩnh viễn IP/Port của Peer đó vào ổ cứng.
- Nếu hạ tầng Bootstrap sập, mạng lưới sẽ tự động kích hoạt chế độ Fallback: Các máy trạm tự động nạp danh sách IP từ SQLite lên RAM. Dù không thể khám phá thêm người mới, chúng vẫn có thể định tuyến và nhắn tin trực tiếp cho các nút cũ. Khi Bootstrap sống lại, hệ thống sẽ tự động đối chiếu và đồng bộ lại danh bạ mạng mới nhất.

3.4 Cơ chế chat nhóm

3.4.1 Bầu cử phi tập trung bằng HRW

Khác với các hệ thống dùng thuật toán bầu cử tôn kém (như Raft hay Bully), hệ thống P2PChat này thiết kế cơ chế tự suy luận (Deterministic Coordinator Selection). Bằng cách sử dụng thuật toán Băm trọng số ngẫu nhiên (HRW) lên ID của nhóm và ID của các thành viên, tất cả các máy trạm đều tự tính toán ra chính xác một người làm

Điều phối nhóm (COORD) mà không cần tốn một gói tin bầu cử nào. Nếu *Điều phối nhóm* đột ngột sập, hàm Hash ở các máy còn lại sẽ tự động trở về thành viên có điểm số cao thứ hai, tạo ra sự chuyển giao quyền lực với độ trễ bằng 0.

Công thức tính điểm để được chọn làm điều phối là:

$$Score(i, G) = SHA256(G || "|" || i) \quad (1)$$

Trong đó:

- 1) i : Là username của thành viên i .
- 2) G : Là groupId của nhóm
- 3) $||$: Là phép nối chuỗi
- 4) $SHA256()$: Là hàm băm SHA-256

3.4.2 Giao thức Gossip và Giải quyết xung đột

Để các COORD trong hệ thống luôn đồng bộ về danh sách thành viên nhóm, chúng sử dụng giao thức Gossip. Mỗi nhóm được gán một nhãn thời gian logic (Version). Các COORD liên tục rỉ tai nhau về phiên bản nhóm của mình. Khi xảy ra xung đột, kiến trúc áp dụng quy tắc ưu tiên Version lớn hơn; nếu Version bằng nhau, hệ thống dùng mã Hash của mảng dữ liệu để làm Tie-breaker, đảm bảo trạng thái nhóm hội tụ.



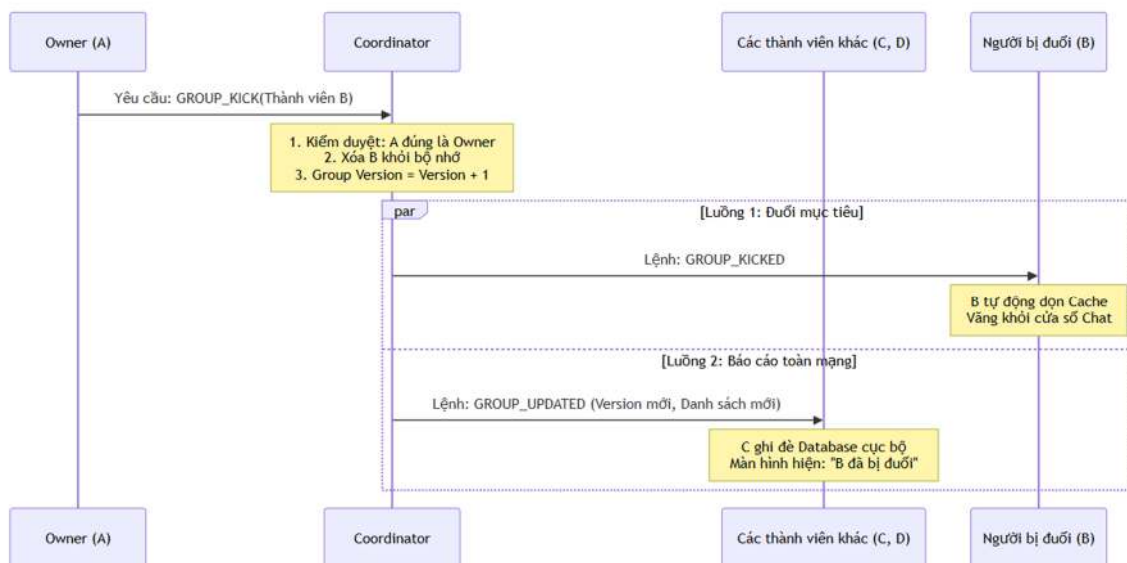
Hình 3.6 Giao thức Gossip và giải quyết xung đột

3.4.3 Cơ chế Cập nhật Thành viên và Vai trò của COORD

Trong mạng Serverless P2P, các thành viên bình thường không có quyền tự ý thay đổi danh sách nhóm trong bộ đệm của họ. Tất cả mọi hành động làm thay đổi trạng thái nhóm đều phải tuân theo luồng cập nhật tập trung thông qua COORD.

Vai trò của COORD trong quy trình thao tác: Khi có một sự kiện thay đổi (Thêm người, Đuổi người, Tự dời đi, Giải tán), quy trình tiêu chuẩn diễn ra qua 3 bước nghiêm ngặt:

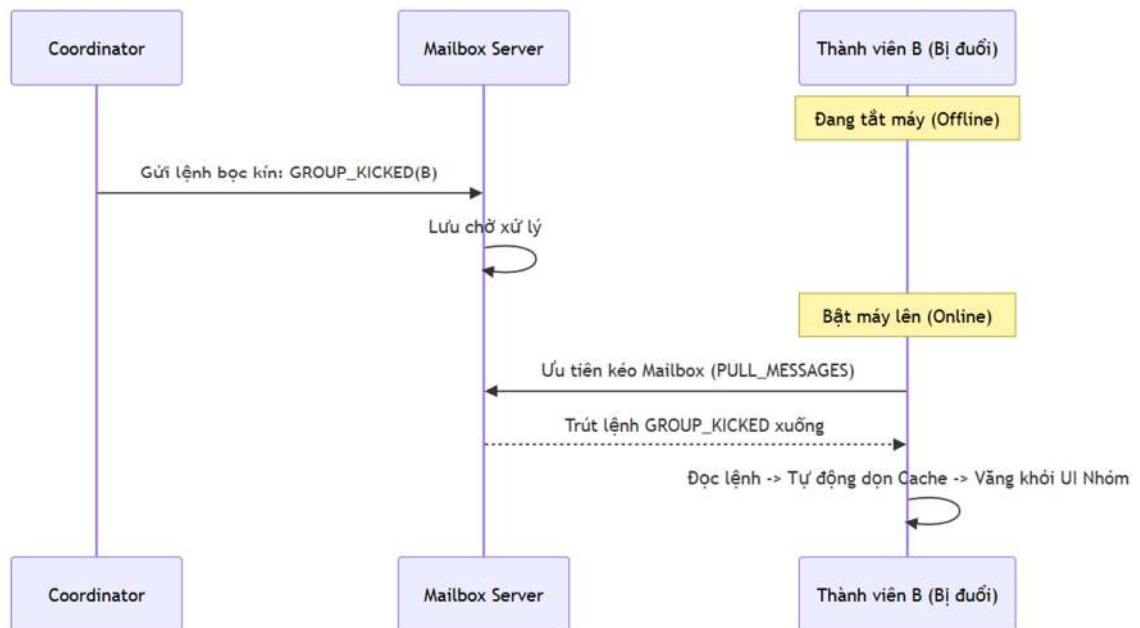
1. **Tiếp nhận và Kiểm duyệt:** Khi A muốn đuổi thành viên B, A không tự xóa B khỏi máy mình, mà phải đóng gói một lệnh GROUP_KICK (chứa ID của B) gửi thẳng tới COORD. COORD đang đảm nhiệm sẽ kiểm tra quyền hạn. Nếu A không phải Owner (Chủ nhóm), lệnh bị từ chối.
2. **Thay đổi trạng thái và Nâng phiên bản:** Nếu lệnh hợp lệ, COORD thực hiện xóa B khỏi danh sách nội bộ của nó. Điểm mấu chốt ở đây là COORD phải tăng nhãn thời gian logic của nhóm (Group Version) lên +1. Nhờ danh sách thành viên mới, COORD cũng tự động chạy lại thuật toán HRW Hash để xem Ban Điều Phối có ai bị thay đổi không.
3. **Phát sóng và Buộc đồng bộ:** Để toàn mạng biết B đã bị đuổi, COORD chịu trách nhiệm bắn đi 3 luồng thông điệp đồng thời:
 - Gửi lệnh GROUP_KICKED trực tiếp cho B để máy của B tự hủy Cache và vắng khỏi giao diện.
 - Gửi tín hiệu GOSSIP cho các COORD dự phòng khác để chúng đồng bộ lên Version mới nhất.
 - Gửi thông điệp GROUP_UPDATED (bên trong chứa toàn bộ danh sách thành viên mới nhất và Version mới) tới tất cả các thành viên còn lại. Khi các máy trạm nhận được GROUP_UPDATED, chúng sẽ ghi đè danh sách cũ trên SQLite, bôi đen giao diện và cập nhật lại số lượng thành viên thực tế.



Hình 3.7 Cơ chế cập nhập thành viên trong nhóm

3.4.5 Nhất quán cuối trong Control Plane

Vấn đề khó nhất của chat nhóm là “Khủng hoảng trạng thái” (Split-brain) khi một người bị đuổi khỏi nhóm lúc đang người đó offline. Vì dữ liệu lưu ở Local Cache, khi người bị đuổi bật máy lên, máy của họ vẫn tưởng họ là thành viên (Thành viên ma) do dữ liệu ở local của họ không được cập nhập khi offline. Họ có thể bơm dữ liệu cũ vào mạng và phá hỏng trạng thái nhóm. Hệ thống giải quyết triệt để bằng nguyên lý Nhất quán cuối thông qua Mailbox. Lệnh đuổi sẽ được ném lên Mailbox. Ngay khoảnh khắc người bị đuổi nối mạng, việc đầu tiên máy tính ép làm là kéo hộp thư. Máy sẽ tải lệnh đuổi về, tự động xóa sạch bộ đệm SQLite và RAM của nhóm đó, đá văng người dùng ra khỏi giao diện, chặn đứng lây nhiễm trạng thái sai lệch.



Hình 3.8 Luồng hoạt động của đuổi thành viên khỏi nhóm khi offline

3.5 Cơ chế truyền file

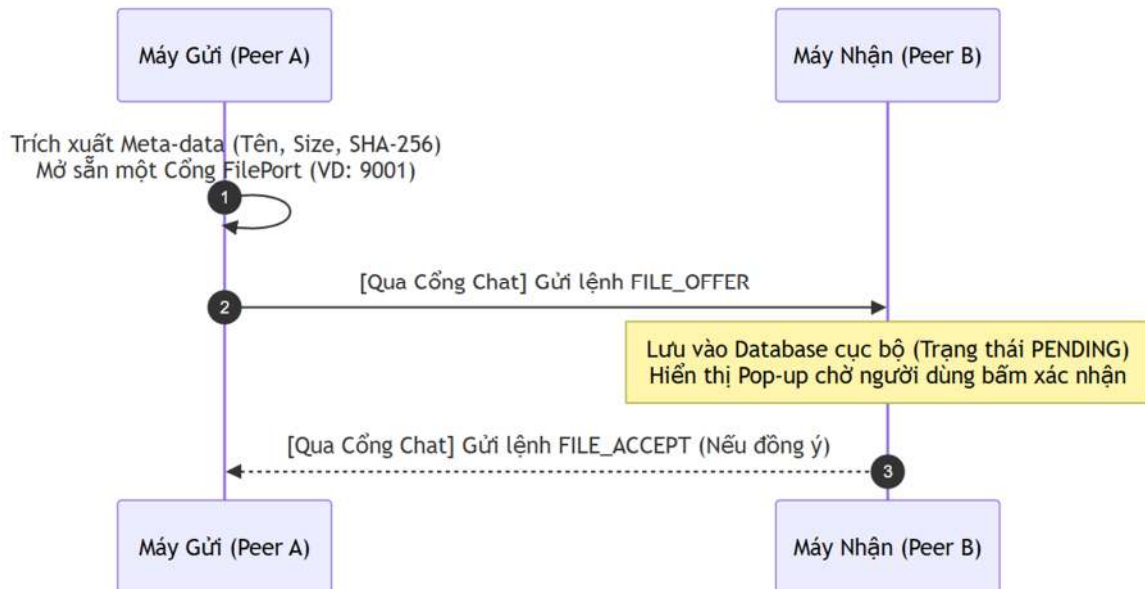
Khác với tin nhắn văn bản, việc gửi các tệp tin dung lượng lớn (hàng trăm MB) không thể sử dụng chung đường truyền JSON của hệ thống Chat. Nguyên nhân là do bộ phân tích cú pháp JSON sẽ làm phình to bộ nhớ (Memory Bloat) và việc truyền file lớn sẽ gây ra hiện tượng nghẽn cổ chai, làm các tin nhắn chat phía sau bị kẹt lại không gửi được. Để giải quyết, hệ thống áp dụng kiến trúc Luồng kép. Vòng đời truyền file được chia làm 4 chặng cốt lõi:

3.5.1 Giao thức Thỏa thuận kênh truyền

Trước khi truyền bất kỳ Byte dữ liệu nào, hai máy trạm phải "nói chuyện" với nhau qua Cổng Chat mặc định để xin phép.

- Người gửi tự động quét file, trích xuất siêu dữ liệu (Meta-data) bao gồm: Tên tệp, Kích thước, Mã băm SHA-256 để chống giả mạo, Tổng số phân mảnh (Total Chunks), và Cổng dữ liệu (FilePort) đang chờ sẵn.

- Máy gửi đóng gói dữ liệu này vào lệnh FILE_OFFER gửi cho máy nhận. Máy nhận lưu tạm vào SQLite ở trạng thái Pending và đẩy thông báo lên màn hình chờ người dùng xác nhận.

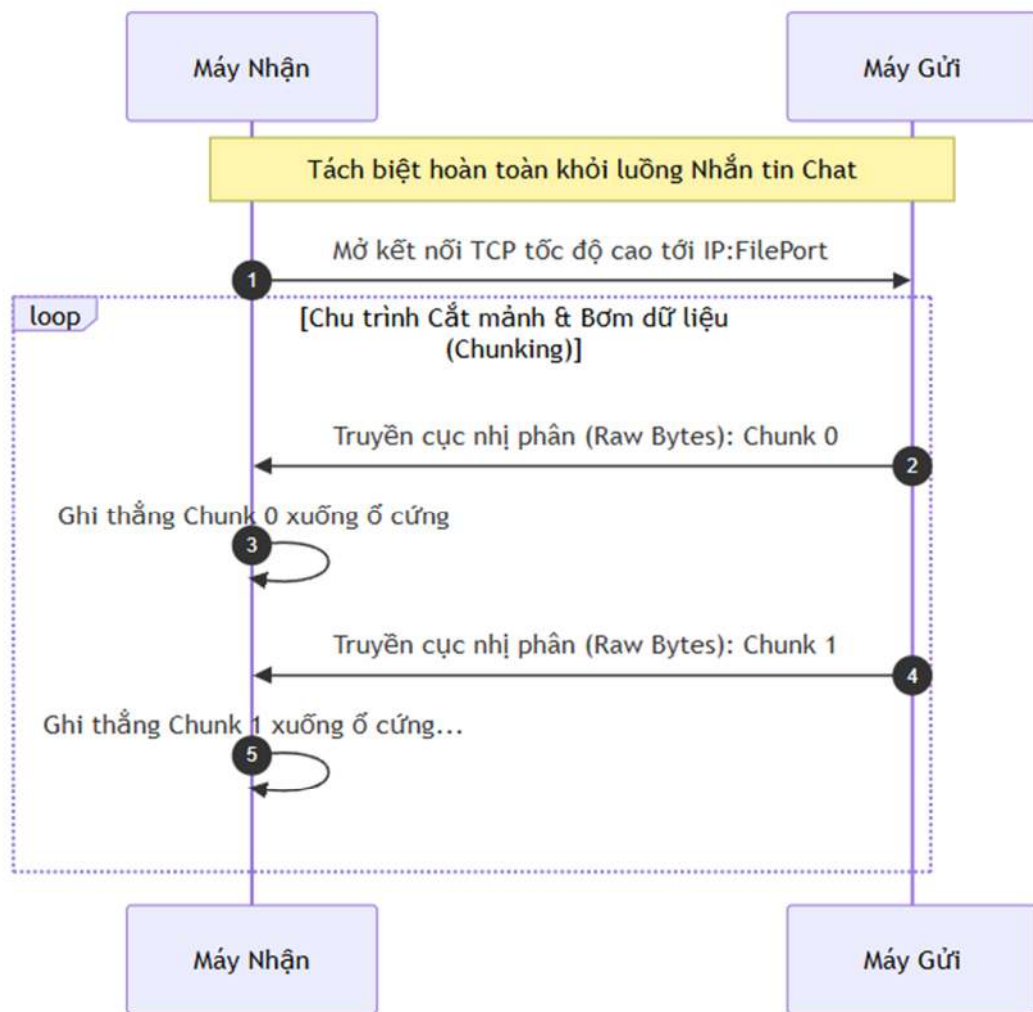


Hình 3.9 Luồng Thỏa thuận trước khi truyền file

3.5.2 Khởi tạo Luồng nhị phân và Cắt mảnh

Ngay khi người dùng B bấm đồng ý (gửi FILE_ACCEPT tới A), luồng Control Plane đã làm xong nhiệm vụ. Trọng tâm chuyển sang luồng Data Plane.

- Máy trạm B chủ động khởi tạo một Socket TCP thô kết nối thẳng vào IP và cổng dữ liệu của máy A.
- Để tránh tràn RAM, tệp tin gốc trên máy A không được đọc toàn bộ vào bộ nhớ. Nó được cắt thành các phân mảnh nhỏ gọn (Chunking) với kích thước cố định (khoảng 64KB). Socket sẽ liên tục bơm các mảnh nhị phân này qua lại với tốc độ tối đa của đường truyền vật lý.



Hình 3.10 Khởi tạo luồng nhị phân và cắt mảnh

3.5.3 Cơ chế Phục hồi đứt gãy

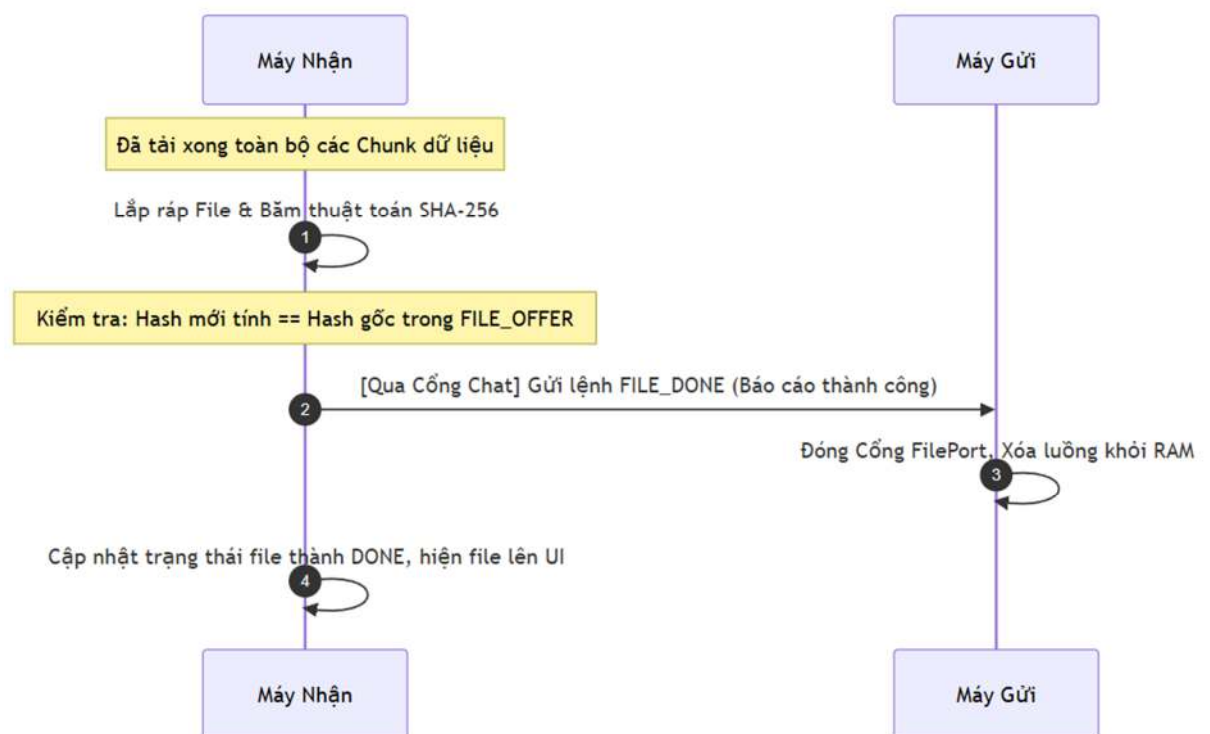
Đặc tính của mạng P2P là thiếu ổn định. Nếu truyền một file 2GB mà bị đứt mạng ở 1.9GB, việc tải lại từ đầu là thảm họa kiến trúc. Hệ thống tích hợp một cơ sở dữ liệu SQLite (FileTransferRepository) để đánh dấu trạng thái.

- Khi bị đứt cáp, máy nhận lưu lại số thứ tự của Chunk cuối cùng tải thành công.
- Khi có mạng lại, máy nhận đóng gói con trỏ resumeChunkIndex vào gói FILE_ACCEPT gửi đi. Máy gửi nhận được lệnh, thay vì đọc file từ byte số 0, nó thực hiện dịch con trỏ file (File Seek) thẳng đến vị trí bị đứt và chỉ bơm các Chunk còn thiếu.

3.5.4 Xác thực Toàn vẹn và Đóng kết nối

Một rủi ro trong truyền tải chia mảnh là gói tin có thể bị can thiệp, thất thoát byte hoặc dính mã độc.

- Khi tải xong Chunk cuối cùng, máy nhận sẽ tiến hành ráp toàn bộ các mảnh thành file hoàn chỉnh.
- Máy nhận tự động chạy thuật toán băm SHA-256 trên toàn bộ file vừa ráp và đối chiếu với mã Hash nằm trong FILE_OFFER ban đầu. Nếu trùng khớp hoàn toàn, máy nhận gửi FILE_DONE qua luồng Chat để máy gửi có thể đóng Socket truyền file, giải phóng RAM. Nếu dữ liệu bị sai lệch, file bị đánh dấu là Corrupted và tự hủy.



Hình 3.11 Cơ chế xác thực toàn vẹn khi gửi file

3.6 Cơ chế store-and-forward& Mail box

3.6.1 Mục đích

Trong hệ thống P2PChat, các peer không phải lúc nào cũng online đồng thời hoặc có thể kết nối TCP trực tiếp với nhau. Nếu chỉ dựa vào truyền trực tiếp P2P, tin nhắn sẽ bị mất khi peer nhận offline, mất kết nối hoặc không phản hồi.

Vì vậy, project triển khai cơ chế Store-and-Forward thông qua mailbox-server. Cơ chế này cho phép tin nhắn được lưu tạm tại mailbox khi chưa thể giao ngay, sau đó được peer nhận lấy về khi online trở lại.

3.6.2 Thành phần tham gia

Cơ chế Store-and-Forward trong project gồm các thành phần chính:

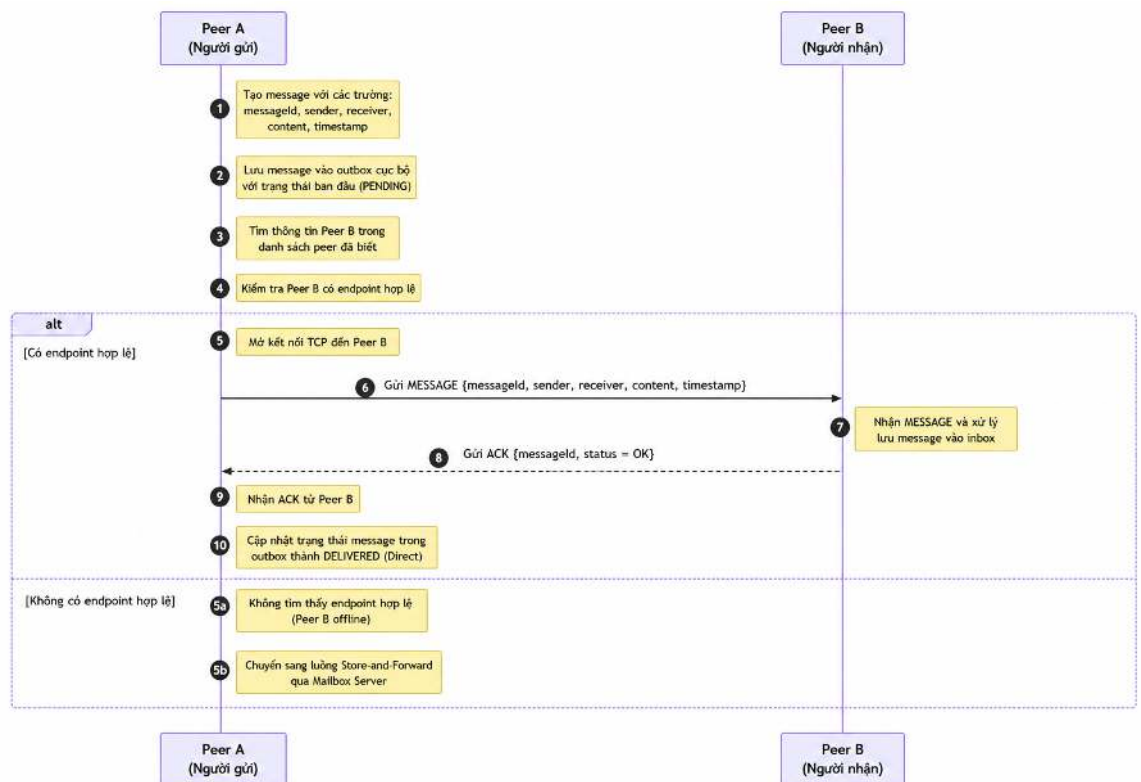
Bảng 3.7 Thành phần tham gia quá trình store-and-forward

Thành phần	Vai trò
peer-node	Gửi tin nhắn trực tiếp, lưu outbox cục bộ, retry và pull tin từ mailbox
bootstrap-server	Quản lý peer online, heartbeat, discovery và cung cấp địa chỉ mailbox
mailbox-server	Lưu trữ tin nhắn offline và giao lại khi peer nhận yêu cầu
SQLite	Lưu dữ liệu offline message và trạng thái giao nhận

Trong đó, mailbox-server không thay thế hoàn toàn truyền P2P trực tiếp. Hệ thống vẫn ưu tiên gửi trực tiếp giữa các peer. Mailbox chỉ được dùng khi gửi trực tiếp thất bại hoặc peer nhận không khả dụng.

3.6.3 Luồng gửi tin nhắn trực tiếp

Khi Peer A gửi tin nhắn cho Peer B, hệ thống thực hiện các bước sau:



Hình 3.12 Luồng gửi tin nhắn trực tiếp

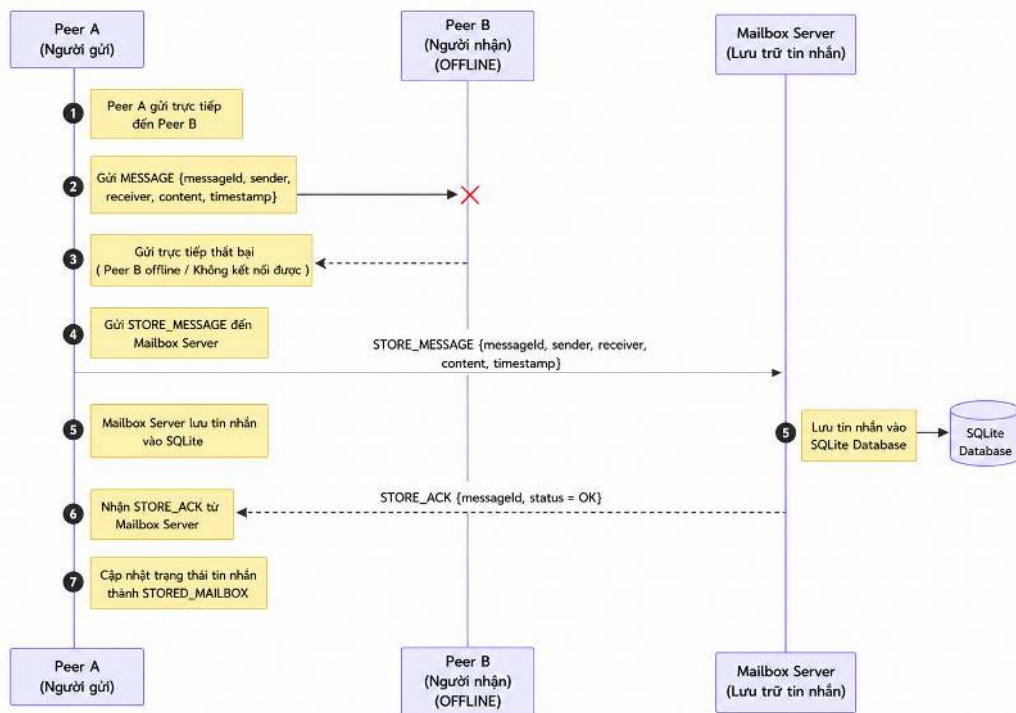
Nhờ vậy, tin nhắn không bị mất dù người nhận chưa online tại thời điểm gửi.

3.6.4 Chuyển sang Store-and-Forward khi gửi trực tiếp thất bại

Nếu Peer A không gửi trực tiếp được cho Peer B, ví dụ Peer B offline, mất kết nối, không mở port hoặc không phản hồi sau các lần retry, hệ thống sẽ chuyển sang cơ chế Store-and-Forward.

Khi đó, Peer A đóng gói tin nhắn thành một MailboxEnvelope và gửi đến mailbox-server bằng message type STORE_MESSAGE. Mailbox server kiểm tra dữ liệu, lưu tin nhắn vào SQLite và trả về STORE_ACK.

Luồng xử lý:



Hình 3.13 Luồng đóng gói và gửi lên mailbox khi gửi tin trực tiếp thất bại

3.6.5 Lưu trữ tin nhắn trong Mailbox

Mailbox server lưu tin nhắn offline trong bảng `offline_messages`. Mỗi bản ghi đại diện cho một tin nhắn đang chờ giao hoặc đã được giao.

Một số thông tin chính được lưu gồm:

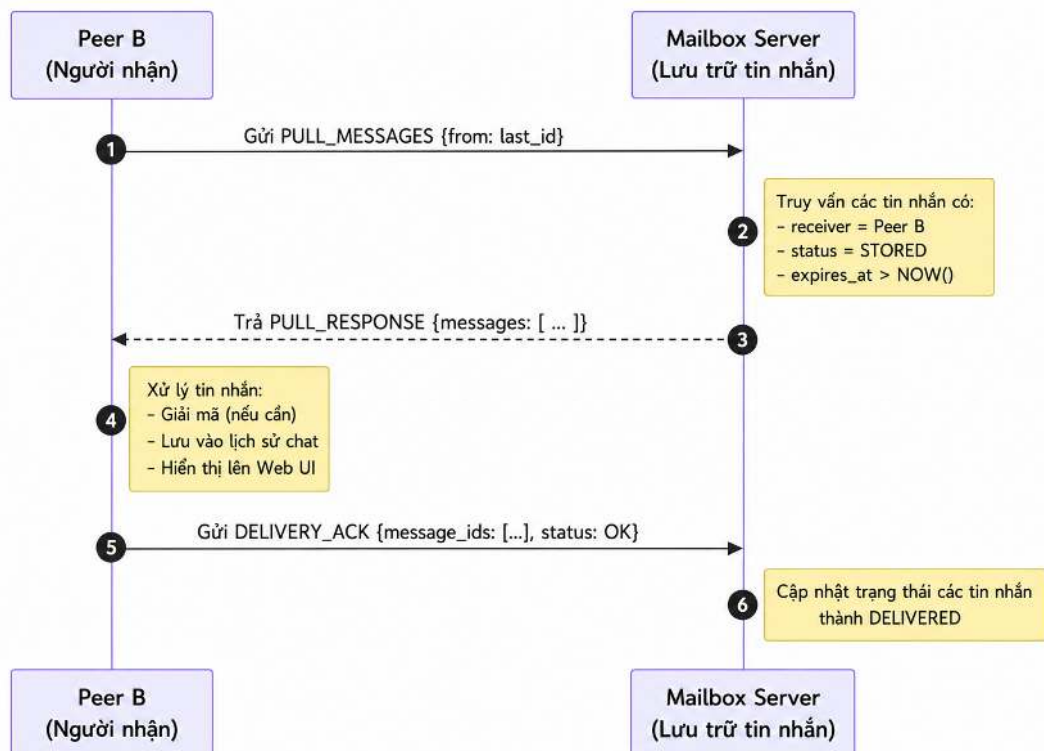
Trường	Ý nghĩa
<code>message_id</code>	Định danh duy nhất của tin nhắn
<code>conversation_id</code>	Định danh cuộc hội thoại
<code>sender</code>	Người gửi
<code>receiver</code>	Người nhận
<code>message_type</code>	Loại tin nhắn
<code>payload_ciphertext</code>	Nội dung/payload đã được đóng gói, với direct message là dữ liệu mã hóa
<code>payload_hash</code>	Hash dùng để kiểm tra trùng lặp hoặc xung đột
<code>mailbox_stored_at</code>	Thời điểm mailbox lưu tin
<code>expires_at</code>	Thời điểm tin nhắn hết hạn
<code>status</code>	Trạng thái tin nhắn: STORED, DELIVERED, EXPIRED
<code>group_id, group_members</code>	Thông tin dùng cho group message

Mailbox có cơ chế kiểm tra trùng message_id. Nếu cùng message_id và cùng payload_hash, tin nhắn được xem là bản gửi lặp. Nếu cùng message_id nhưng khác payload_hash, mailbox coi đó là xung đột.

3.6.6 Peer nhận lấy tin nhắn từ Mailbox

Khi Peer B online lại, peer sẽ đăng ký hoặc heartbeat với bootstrap server. Đồng thời, peer chạy tiến trình nền để kéo tin nhắn từ mailbox.

Quá trình nhận tin offline gồm các bước:



Hình 3.14 Luồng peer lấy tin nhắn bị lỡ từ mailbox

3.6.7 Outbox cục bộ tại Peer gửi

Ngoài mailbox server, project còn sử dụng outbox cục bộ tại peer gửi để tăng độ tin cậy. Trước khi gửi, tin nhắn được lưu vào bảng outbound_messages. Bảng này giúp peer theo dõi trạng thái gửi và thực hiện retry khi gặp lỗi. Nhờ outbox, nếu gửi trực tiếp lỗi hoặc gửi lên mailbox thất bại, tin nhắn vẫn còn trong cơ sở dữ liệu cục bộ và có thể được retry bởi tiến trình nền.

3.6.8 Retry và cập nhật trạng thái giao nhận

Peer gửi có tiến trình nền chạy định kỳ để xử lý các tin nhắn chưa hoàn tất. Tiến trình này thực hiện các nhiệm vụ:

- retry gửi trực tiếp nếu có thông tin peer nhận;
- retry lưu tin nhắn vào mailbox nếu lần trước thất bại;
- kiểm tra với mailbox xem tin nhắn đã được người nhận pull hay chưa;
- cập nhật trạng thái outbox để Web UI hiển thị đúng trạng thái gửi.

Khi người nhận đã pull tin nhắn và gửi DELIVERY_ACK, mailbox đánh dấu tin là DELIVERED. Peer gửi có thể kiểm tra trạng thái này thông qua mailbox và cập nhật outbox từ STORED_MAILBOX sang DELIVERED.

3.6.9 Hỗ trợ tin nhắn nhóm

Với group message, hệ thống vẫn ưu tiên gửi trực tiếp đến từng thành viên trong nhóm. Nếu một số thành viên không nhận được tin, danh sách các thành viên bị miss sẽ được lưu vào mailbox.

Mailbox lưu group message kèm group_id và group_members. Khi từng thành viên online lại và pull tin nhắn, mailbox ghi nhận ACK riêng cho từng người trong bảng group_delivery_acks. Khi tất cả thành viên trong danh sách đã ACK, tin nhắn nhóm được xem là đã giao đủ.

3.6.10 Cơ Chế Store-and-Forward Qua Peer Trung Gian

Bên cạnh mailbox server, hệ thống bổ sung cơ chế Store-and-Forward qua peer trung gian để xử lý trường hợp mailbox server không khả dụng. Khi peer gửi không thể gửi trực tiếp cho peer nhận và cũng không thể lưu tin vào mailbox, tin nhắn sẽ được lưu tạm tại một số peer online khác trong mạng. Các peer này đóng vai trò relay, lưu hộ bản tin đã mã hóa và chuyển tiếp lại khi peer nhận online.

Luồng ưu tiên gửi tin nhắn trong hệ thống là:

Direct P2P -> Mailbox Server -> Relay Peer -> Local Outbox Retry

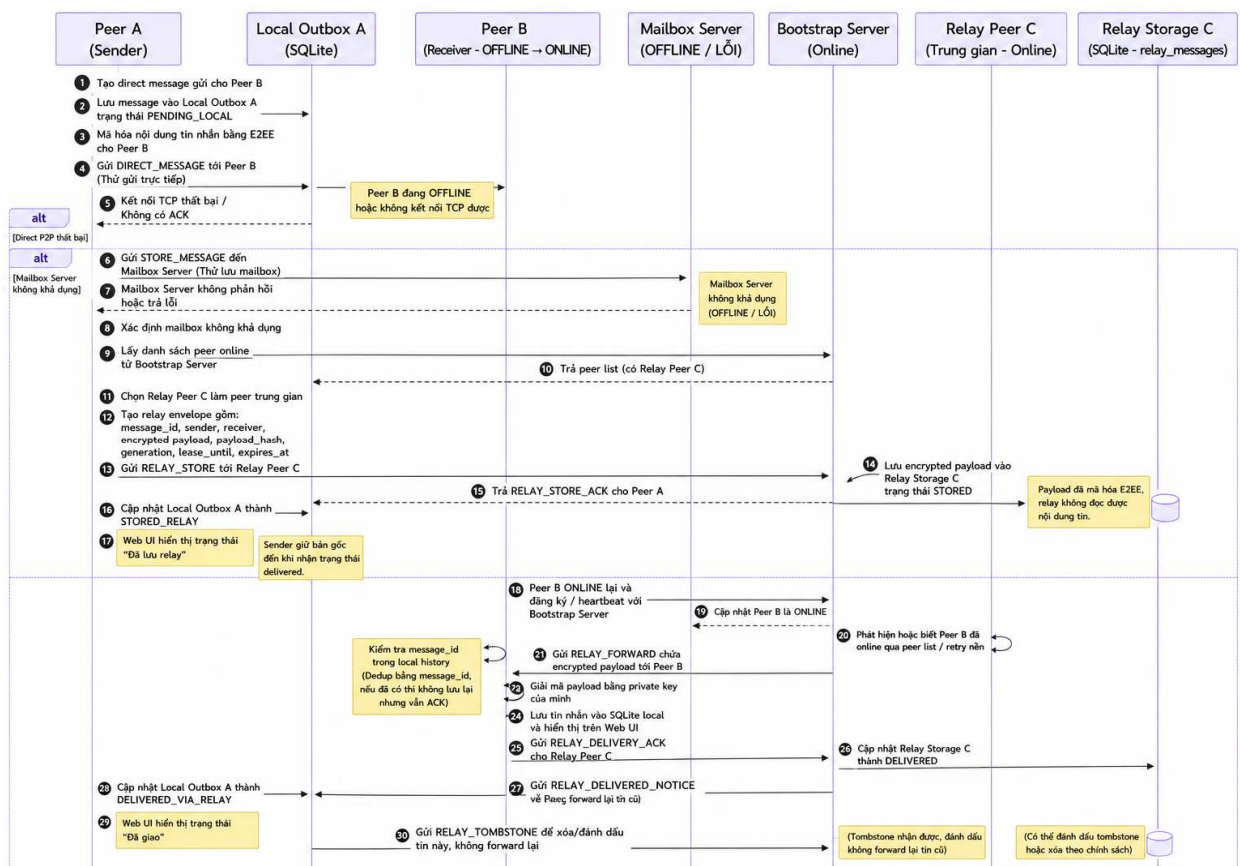
Khi Peer A gửi tin cho Peer B nhưng Peer B offline, hệ thống trước tiên thử lưu tin vào mailbox server. Nếu mailbox cũng lỗi hoặc circuit breaker của mailbox đang mở,

Peer A chọn một số peer online làm relay. Tin nhắn trước khi gửi đến relay đã được mã hóa đầu cuối cho Peer B, vì vậy relay chỉ lưu dữ liệu mã hóa và không đọc được nội dung tin nhắn.

Để tăng khả năng giao tin trong môi trường P2P có peer online/offline liên tục, hệ thống không chỉ chọn một relay duy nhất. Mỗi tin nhắn có thể được lưu tại tối đa 4 relay peer, gồm 1 primary relay và 3 backup relay. Primary relay là peer chủ động forward tích cực hơn, còn backup relay chủ yếu giữ bản sao dự phòng hoặc forward khi lease của primary hết hạn.

Mỗi relay assignment có thời hạn lease. Nếu hết lease mà tin chưa được xác nhận là đã giao, sender sẽ xoay vòng sang relay khác. Relay cũ không bị xóa ngay mà được chuyển sang trạng thái superseded để tránh mất bản sao trong lúc relay mới chưa lưu thành công. Relay đã fail sẽ được đưa vào cooldown trong một khoảng thời gian để tránh bị chọn lại ngay.

Cụ thể sơ đồ tuần tự:



Hình 3.15 Sơ đồ tuần tự trường hợp dùng relay.

3.6.11 Kết luận

Cơ chế Store-and-Forward trong project P2PChat giúp hệ thống hoạt động ổn định hơn trong môi trường P2P, nơi các peer có thể online hoặc offline bất kỳ lúc nào. Hệ thống vẫn giữ nguyên ưu tiên truyền trực tiếp giữa các peer, nhưng bổ sung mailbox server làm cơ chế dự phòng khi truyền trực tiếp thất bại.

Nhờ kết hợp mailbox server, outbox cục bộ, retry nền, ACK giao nhận và TTL cho tin nhắn, project hạn chế mất tin nhắn và hỗ trợ tốt hơn cho các tình huống mạng không ổn định hoặc peer nhận chưa sẵn sàng kết nối.

3.7 Cơ chế mã hóa đầu cuối

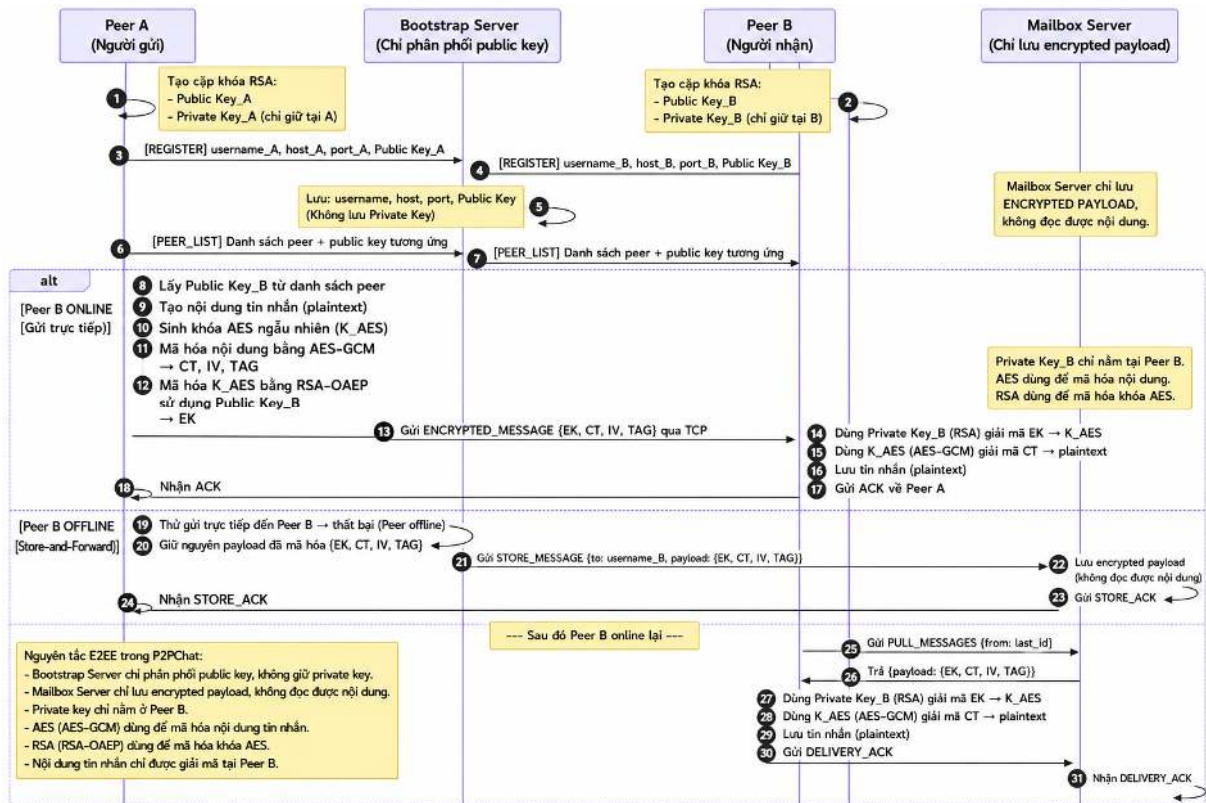
Trong project P2PChat, cơ chế mã hóa đầu cuối được triển khai cho tin nhắn trực tiếp giữa hai peer. Mục tiêu là đảm bảo nội dung tin nhắn chỉ được mã hóa tại peer gửi và chỉ được giải mã tại peer nhận. Các thành phần trung gian như bootstrap server, mailbox server hoặc đường truyền mạng không có private key nên không thể đọc được nội dung thật.

Mỗi peer khi khởi tạo sẽ có một cặp khóa RSA gồm public key và private key. Public key được chia sẻ cho các peer khác thông qua bootstrap server trong quá trình đăng ký peer. Ngược lại, private key chỉ được lưu cục bộ tại peer sở hữu và không được gửi ra ngoài.

Khi Peer A gửi tin nhắn cho Peer B, Peer A lấy public key của Peer B từ danh sách peer đã biết. Sau đó hệ thống sinh một khóa AES ngẫu nhiên để mã hóa nội dung tin nhắn. Nội dung sau khi mã hóa trở thành ciphertext. Tiếp theo, khóa AES này được mã hóa bằng RSA-OAEP với public key của Peer B. Gói tin gửi đi gồm nội dung đã mã hóa, khóa AES đã được bọc, thuật toán mã hóa và thông tin định danh khóa.

Khi Peer B nhận được tin nhắn, Peer B dùng private key RSA của mình để giải mã khóa AES. Sau khi lấy lại được khóa AES, Peer B dùng khóa này để giải mã ciphertext và khôi phục nội dung tin nhắn gốc. Nếu khóa người nhận không khớp hoặc dữ liệu bị thay đổi, quá trình giải mã sẽ thất bại.

Cơ chế này cũng được áp dụng khi tin nhắn direct phải lưu tạm qua mailbox server. Trong trường hợp peer nhận offline hoặc gửi trực tiếp thất bại, peer gửi sẽ mã hóa payload trước, sau đó gửi bản mã đến mailbox server để lưu trữ. Mailbox server chỉ lưu dữ liệu đã mã hóa, không có khả năng giải mã nội dung. Khi peer nhận online lại, peer kéo tin nhắn từ mailbox về và tự giải mã bằng private key của mình.



Hình 3.16 Cơ chế mã hóa đầu cuối

3.8 Mô phỏng churn

3.8.1 Giới thiệu

Churn (hay còn gọi là tỷ lệ rời mạng) là hiện tượng các peer trong mạng P2P liên tục tham gia và rời khỏi mạng một cách không đồng bộ. Trong thực tế, người dùng mạng ngang hàng thường không thông báo trước khi offline. Họ có thể tắt ứng dụng, mất kết nối internet, hoặc máy tính bị tắt đột ngột.

Mô phỏng churn giúp đánh giá khả năng chịu đựng của mạng P2P trước sự biến động liên tục này, thông qua việc:

- Mô phỏng các peer join/crash theo các phân phối xác suất thực tế

- Theo dõi ảnh hưởng của từng sự kiện đến trạng thái mạng
- Đo lường 3 chỉ số cốt lõi: MDR, CT, CO

3.8.2 Mô hình mô phỏng

Thay vì mô phỏng theo bước thời gian cố định (tick-based), mô phỏng này sử dụng hàng đợi ưu tiên (priority queue — min-heap) sắp xếp theo thời gian mô phỏng. Mỗi sự kiện được xử lý ngay tại thời điểm xảy ra mà không cần chờ real-time.

Ưu điểm so với mô hình tick-based:

- Chính xác tuyệt đối về thời gian sự kiện.
- Hiệu suất cao vì chỉ xử lý sự kiện thực sự xảy ra.
- Dễ dàng mở rộng thêm loại sự kiện mới.

3.8.3 Các thành phần chính

Bảng 3.8 Các thành phần

Lớp / Module	Chức năng
BootstrapNode	Node trung tâm: quản lý danh sách peer online
RoutingTable	Bảng định tuyến: láng giềng của mỗi peer
Peer	Đối tượng peer: trạng thái, mailbox, inbox
Network	Mạng P2P toàn cục: quản lý tất cả peer
Metrics	Tracker: thu thập và tính MDR, CT, CO
Simulation	Engine: event loop, scheduling, handlers

3.8.4 Mạng P2P và Topology

Mỗi peer duy trì bảng định tuyến chứa danh sách láng giềng. Khi peer join/crash, tất cả bảng liên quan được cập nhật. Hệ thống hỗ trợ 3 loại topology:

Bảng 3.9 Các loại Topology

Topology	Mô tả	Đặc điểm
random	Mỗi peer chọn ngẫu nhiên max_neighbors peer khác	Mô phỏng mạng P2P tự nhiên
ring	Peer kết nối với 2 láng giềng liền kề	Đơn giản, ít liên kết
mesh	Peer kết nối với tất cả peer còn lại	Độ liên kết cao, chịu lỗi tốt

3.8.5 Các tham số mô phỏng

Bảng 3.10 Các tham số mô phỏng churn

Tham số	Flag	Mặc định	Ý nghĩa
n-peers	--n-peers	10	Tổng số peer trong mạng
duration	--duration	300s	Thời gian mô phỏng (giây)
mean-session	--mean-session	60s	Trung bình thời gian sống trước khi crash
mean-downtime	--mean-downtime	30s	Trung bình thời gian offline trước khi rejoin
msg-interval	--msg-interval	5s	Khoảng cách trung bình giữa 2 tin nhắn
heartbeat-interval	--heartbeat-interval	5s	Chu kỳ heartbeat (Ping/Pong)

lambda-join	--lambda-join	0.05 peer/s	Tốc độ peer mới tham gia (Poisson)
topology	--topology	random	Cấu trúc liên kết mạng
max-neighbors	--max-neighbors	4	Số láng giềng tối đa mỗi peer
seed	--seed	42	Hạt giống random (để tái lập kết quả)

3.8.6 Các loại sự kiện

Mô phỏng xử lý 8 loại sự kiện chính trong vòng event loop:

Bảng 3.11 Các sự kiện trong mô phỏng churn

Sự kiện	Mô tả
REGISTER	Peer mới đăng ký với Bootstrap Node
ACK	Bootstrap xác nhận đăng ký thành công
CRASH	Peer rời mạng đột ngột (không báo trước)
REJOIN	Peer quay lại mạng sau downtime
SEND_MSG	Peer gửi tin nhắn chat đến peer khác
PING	Peer gửi heartbeat đến láng giềng
CONVERGE	Mạng hội tụ sau 1 crash (routing tables ổn định)
END	Mô phỏng kết thúc tại sim_time = duration

3.8.7 Các độ đo đầu ra

3.8.7.1 MDR — Message Delivery Ratio

Ý nghĩa: Tỷ lệ phần trăm tin nhắn chat thực sự đến được người nhận so với tổng số tin nhắn đã gửi.

$$\text{MDR} = \frac{\text{Số tin nhắn đã delivered}}{\text{Tổng số tin nhắn gửi}} \times 100\%$$

Phân loại:

- MDR Direct: tin gửi khi receiver đang online → giao ngay lập tức.
- MDR Mailbox: tin gửi khi receiver offline → lưu mailbox, delivered khi receiver quay lại.

Bảng 3.12 Đánh giá ngưỡng của chỉ số MDR

Ngưỡng MDR	Đánh giá
$\geq 80\%$	[OK] — Mạng hoạt động tốt
$< 80\%$	[WARN] — Tỷ lệ mất tin cao

3.8.7.2 CT — Convergence Time

Ý nghĩa: Thời gian trung bình để tất cả bảng định tuyến trong mạng ổn định lại sau khi một peer bị crash.

$$\text{CO} = \frac{\text{Control_bytes}}{\text{Data_bytes} + \text{Control_bytes}} \times 100\%$$

Cơ chế đo lường: Peer crash → thêm vào `_pending_convergence`. Mỗi heartbeat cycle kiểm tra xem crashed peer còn trong bảng định tuyến của ai không. Khi không còn → ghi nhận convergence.

Công thức lý thuyết: $CT \approx 2 \times \text{heartbeat_interval}$ (trung bình phát hiện trong 2 chu kỳ heartbeat).

Bảng 3.13 Đánh giá ngưỡng của chỉ số CT

Ngưỡng CT	Đánh giá
$\leq HB \times 4$	[OK] — Mạng hội tụ nhanh
$> HB \times 4$	[WARN] — Hội tụ chậm, cần tối ưu

3.8.7.3 CO — Control Overhead

Ý nghĩa: Tỷ lệ băng thông tiêu tốn cho gói tin điều khiển (Ping, REGISTER, ACK, UPDATE) trên tổng lưu lượng mạng.

$$CO = \frac{\text{Control_bytes}}{\text{Data_bytes} + \text{Control_bytes}} \times 100\%$$

Bảng 3.14 Kích thước các loại gói tin

Loại gói tin	Kích thước (bytes)
REGISTER	128
ACK	128
PING	64
PONG	64
UPDATE (route)	80
CHAT_MSG (data)	256

Bảng 3.15 Đánh giá ngưỡng của chỉ số CT

Ngưỡng CO	Đánh giá
$\leq 80\%$	[OK] — Tỷ lệ data/control hợp lý

> 80%

[WARN] — Quá nhiều control packet

CHƯƠNG IV. TRIỂN KHAI HỆ THỐNG

4.1 Môi trường và công nghệ sử dụng

Bảng 4.1 Môi trường và công nghệ sử dụng

Công nghệ	Phiên bản	Vai trò
Java	17+	Backend: Bootstrap, Mailbox, Peer
Maven (Shade Plugin)	3.9+	Build fat JAR cho mỗi module
Javalin	6.1.3	HTTP server + WebSocket trong Peer Node
React	18.2	Frontend Web UI
SQLite JDBC	3.45.1.0	Database cục bộ tại mỗi peer và Mailbox
Gson	2.10.1	JSON serialization/deserialization
Docker	20+	Container hóa toàn bộ hệ thống
Tailscale	Mới nhất	VPN cho kết nối P2P qua Internet
Python	3.x	Peer launcher HTTP server (port 9200)

Yêu cầu hệ thống:

- Java JDK 17+
- Maven 3.9+
- Node.js 18+, npm 9+
- Docker 20+
- Windows 10/11 hoặc Linux (WSL2)

4.2 Cấu trúc mã nguồn và phân chia module

```
P2PChat/
├── run.sh                # Build/start/stop Docker stack, tạo peers-index.html
├── build.sh              # Build React + 3 Maven modules cho local run
├── Dockerfile            # Multi-stage image cho peer-node kèm React build
├── peers-index.html      # Trang link nhanh tới các peer đang chạy
├── bootstrap-server/
│   ├── Dockerfile
│   ├── pom.xml
│   └── src/main/java/com/mycompany/p2pchat/
│       ├── bootstrap/    # BootstrapApp, BootstrapServer, dashboard, registry
│       ├── model/
│       ├── protocol/
│       └── utils/
├── mailbox-server/
│   ├── Dockerfile
│   ├── pom.xml
│   └── src/main/
│       ├── java/com/mycompany/p2pchat/mailbox/
│       └── resources/schema.sql
├── peer-node/
│   ├── Dockerfile
│   ├── pom.xml
│   └── src/main/
│       ├── java/com/mycompany/p2pchat/
│       │   ├── peer/     # PeerNode, PeerClient, PeerServer, mailbox, E2EE,
│       │   └── COORD
│       └── database/     # SQLite repositories: messages, outbox, file
```

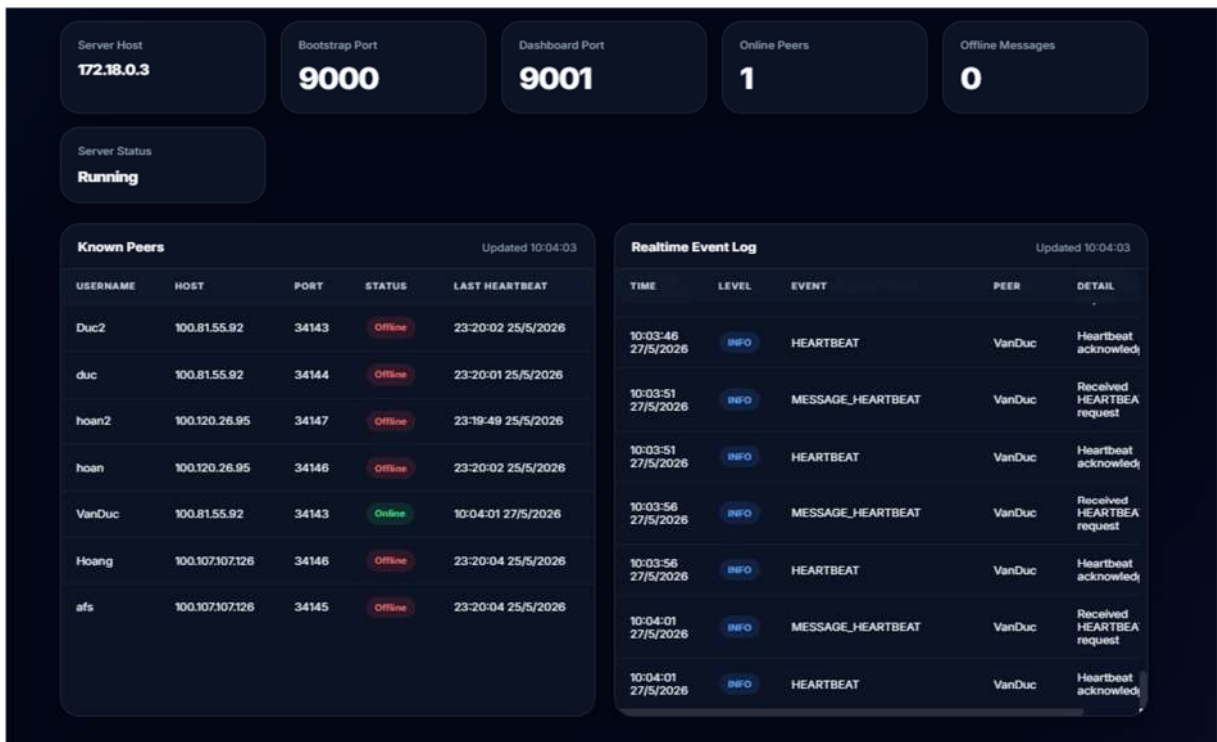
```

transfers
|   |   |── filetransfer/    # Offer/accept/chunk transfer/resume
|   |   |── model/
|   |   |── protocol/
|   |   |── utils/
|   |   └── resources/
|       |── schema.sql
|       |── static/          # React build được copy vào đây
└── peer-web/
    ├── package.json
    └── src/
        ├── api.js
        ├── App.jsx
        ├── App.css
        └── components/

```

4.3 Bootstrap Server

Bootstrap Server được khởi tạo và lắng nghe tại cổng 9001, đóng vai trò là thành phần trung tâm chịu trách nhiệm theo dõi trạng thái các peer trong mạng. Server tiếp nhận và xử lý các sự kiện đăng ký tham gia, rời khỏi mạng, cũng như các tín hiệu HEARTBEAT định kỳ từ từng peer để duy trì danh sách peer đang hoạt động.



Hình 4.1 Giao diện dashboard của Bootstrap Server

Bootstrap Server quản lý các trạng thái (Online/Offline) của các peer qua các thông điệp, khi HEARTBEAT rời mạng nó sẽ đưa ra tín hiệu TIMEOUT.

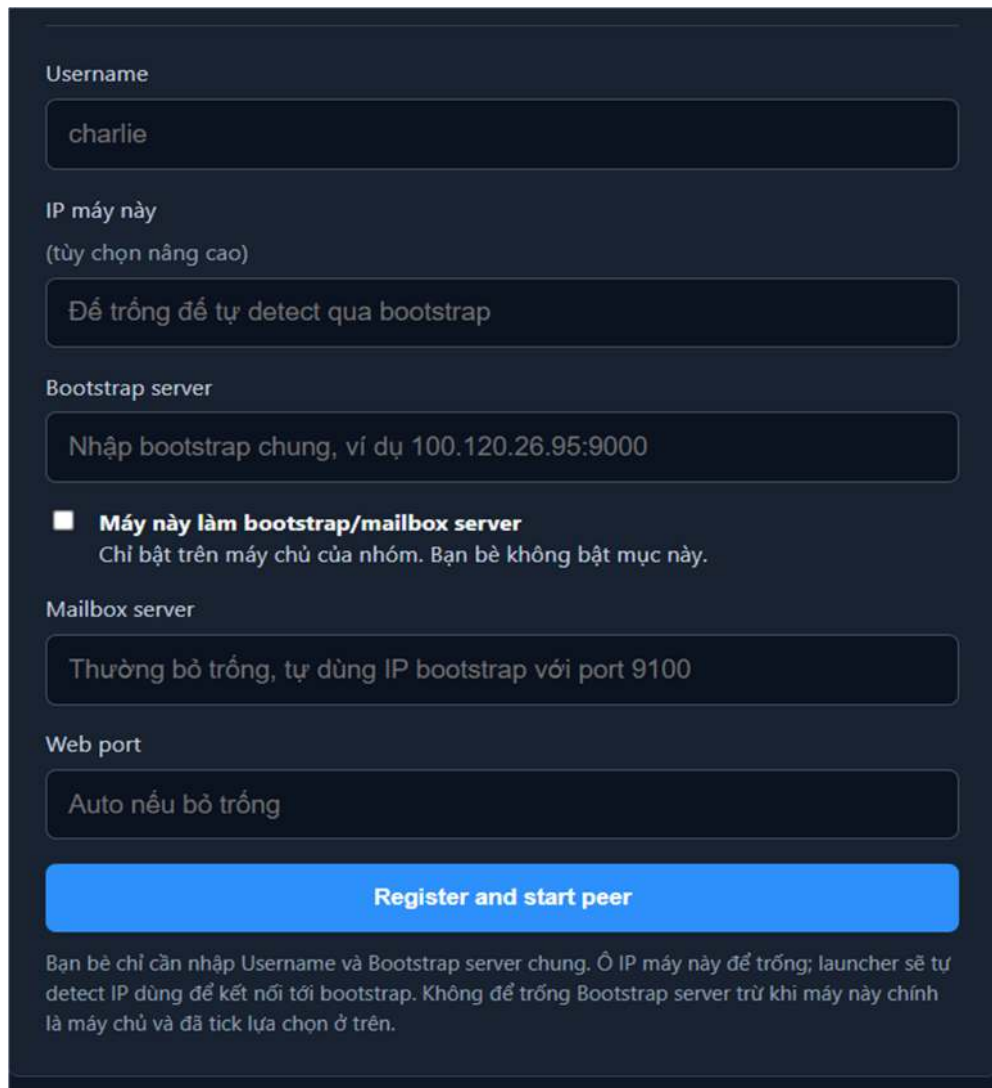
10:25:01 27/5/2026	INFO	HEARTBEAT	Hoang	Heartbeat acknowledged
10:25:02 27/5/2026	INFO	MESSAGE_HEARTBEAT	hoan2	Received HEARTBEAT request
10:25:02 27/5/2026	INFO	HEARTBEAT	hoan2	Heartbeat acknowledged
10:25:04 27/5/2026	INFO	MESSAGE_RESOLVE_MAILBOX	Hoang	Received RESOLVE_M request
10:25:05 27/5/2026	WARN	HEARTBEAT_TIMEOUT	VanDuc	No heartbeat 45000 ms

Hình 4.2 Trạng thái của các Peer có thể theo dõi trên Dashboard của Bootstrap Server

4.4 Peer Node

Trong thực tế triển khai hệ thống P2PChat, người dùng sẽ tương tác trực tiếp với Peer Node thông qua trình duyệt web. Quá trình hoạt động và các chức năng trên giao diện được chia làm hai giai đoạn chính: Khởi tạo Peer và Giao diện Chat chính.

4.4.1 Khởi tạo Peer



The image shows a web form for registering a peer node. It has a dark theme with light-colored text. The form includes several input fields and a checkbox. At the bottom, there is a large blue button and a paragraph of explanatory text.

Username
charlie

IP máy này
(tùy chọn nâng cao)
Để trống để tự detect qua bootstrap

Bootstrap server
Nhập bootstrap chung, ví dụ 100.120.26.95:9000

☐ **Máy này làm bootstrap/mailbox server**
Chỉ bật trên máy chủ của nhóm. Bạn bè không bật mục này.

Mailbox server
Thường bỏ trống, tự dùng IP bootstrap với port 9100

Web port
Auto nếu bỏ trống

Register and start peer

Bạn bè chỉ cần nhập Username và Bootstrap server chung. Ở IP máy này để trống; launcher sẽ tự detect IP dùng để kết nối tới bootstrap. Không để trống Bootstrap server trừ khi máy này chính là máy chủ và đã tick lựa chọn ở trên.

Hình 4.3 Giao diện trang đăng ký peer

Đăng ký Peer mới:

- Username: Người dùng nhập tên định danh của mình (ví dụ: alice, bob).
- Host IP: Cần nhập địa chỉ IP của máy để truy cập (IP mạng nội bộ hoặc VPN Tailscale để các Peer khác có thể liên lạc trực tiếp).

- Bootstrap Server: Nhập IP và cổng 9000 của máy chạy bootstrap server hoặc tích chọn để máy hiện tại tự chạy bootstrap server bằng IP của nó. Bootstrap server hoạt động độc lập với peer khởi động cùng máy.
- Mailbox và Web port thường để tự động (Không nhập gì cả)

Tiếp tục với Peer cũ:

- Danh sách các Peer đã được tạo và lưu cấu hình trước đó sẽ hiển thị tại đây. Chỉ cần bấm nút "Open" để quay lại phòng chat mà không cần cấu hình lại từ đầu.

P2PChat

Mở lại peer đang có hoặc đăng ký một peer mới trên máy này.

Continue

@VanDuc

http://localhost:33143

Open

Username

charlie

IP máy này
(tùy chọn nâng cao)

Để trống để tự detect qua bootstrap

Bootstrap server

Nhập bootstrap chung, ví dụ 100.120.26.95:9000

☒ **Máy này làm bootstrap/mailbox server**
Chỉ bật trên máy chủ của nhóm. Bạn bè không bật mục này.

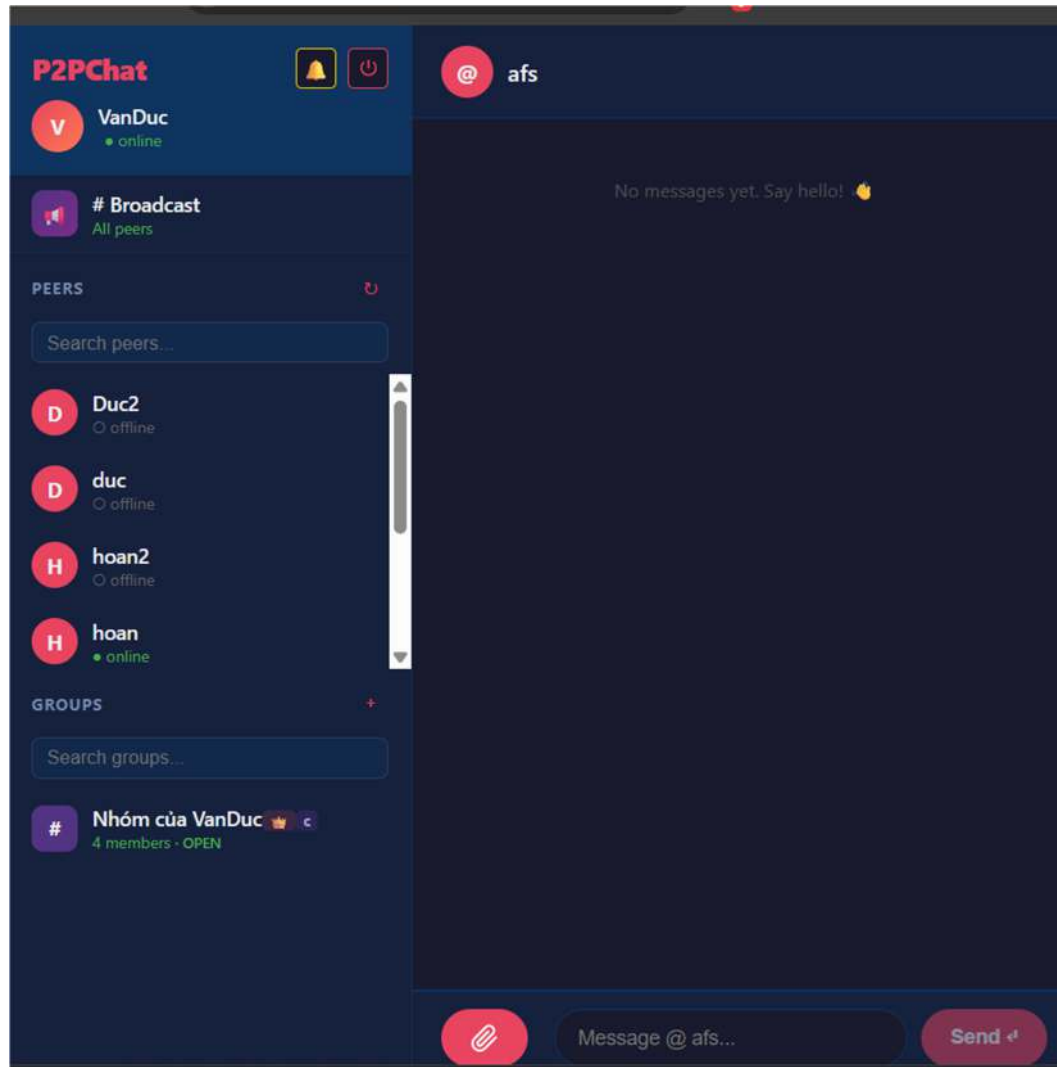
Mailbox server

Thường bỏ trống, tự dùng IP bootstrap với port 9100

Hình 4.4 Khi đã đăng ký có thể vào lại nhanh bằng nút Open

4.4.2. Giao diện Chat chính

Giao diện chat chính là khu vực trung tâm của hệ thống, được thiết kế gồm ba thành phần chính: khu vực thông tin người dùng, danh sách liên hệ và vùng hội thoại.



Hình 4.5 Giao diện chat chính

Thanh bên trái (Sidebar): Hiển thị thông tin người dùng hiện tại gồm ảnh đại diện, tên tài khoản và trạng thái hoạt động. Phần này cũng chứa các nút chức năng như bật tắt âm thanh thông báo và đăng xuất khỏi hệ thống.

Khu vực Broadcast: Cho phép người dùng gửi hoặc nhận thông báo chung đến toàn bộ các peer đang online trong mạng.

Danh sách Peer: Hiển thị toàn bộ người dùng đã kết nối trong hệ thống cùng trạng thái trực tuyến. Người dùng có thể tìm kiếm nhanh bằng thanh tìm kiếm hoặc cập nhật lại danh sách thông qua nút làm mới.

Danh sách nhóm: Hiển thị các nhóm trò chuyện mà người dùng tham gia, bao gồm tên nhóm, số lượng thành viên và trạng thái của nhóm.

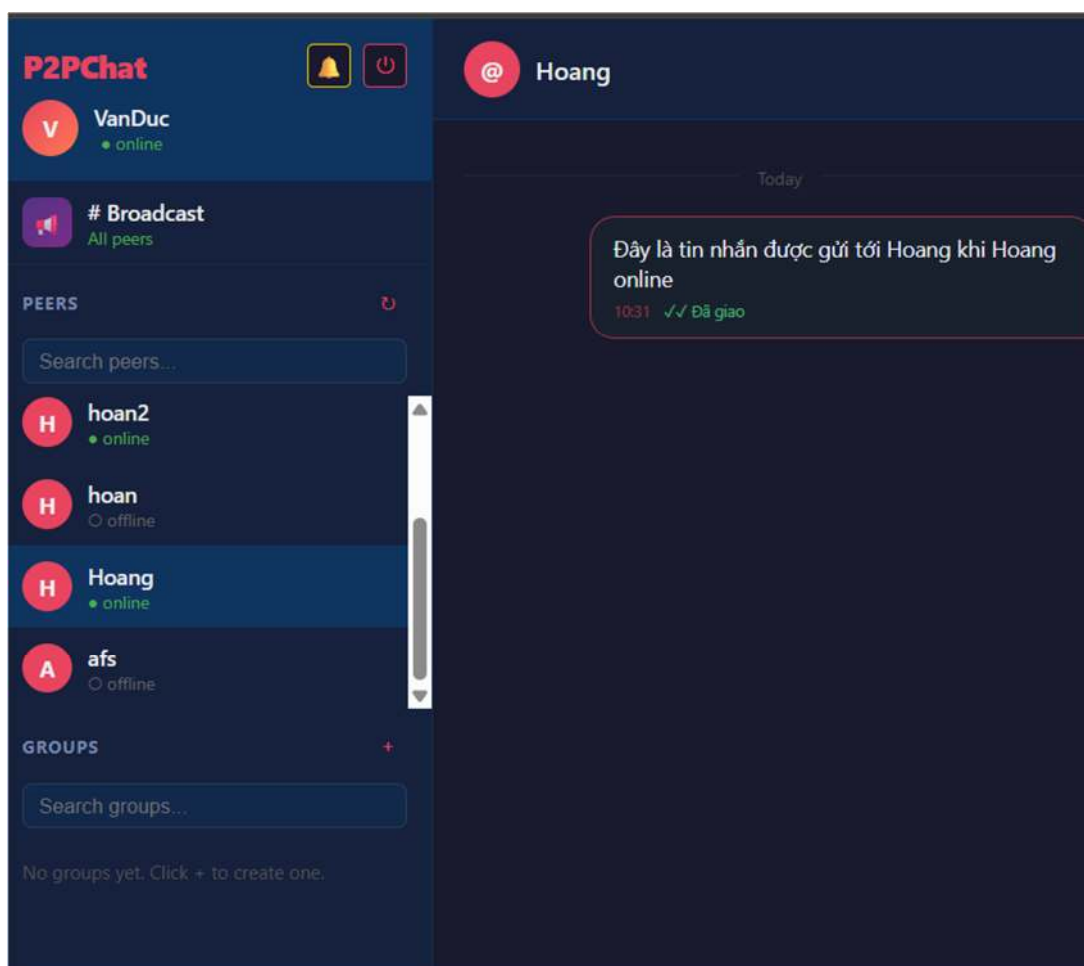
Vùng hội thoại: Hiển thị nội dung tin nhắn của cuộc trò chuyện đang được chọn, bao gồm trò chuyện cá nhân hoặc nhóm.

Thanh nhập tin nhắn: Đặt ở phía dưới giao diện, hỗ trợ nhập nội dung tin nhắn, đính kèm tệp và gửi dữ liệu đến đối tượng đang trò chuyện.

4.5 Chat trực tiếp

4.5.1 Chat khi hai peer đều online.

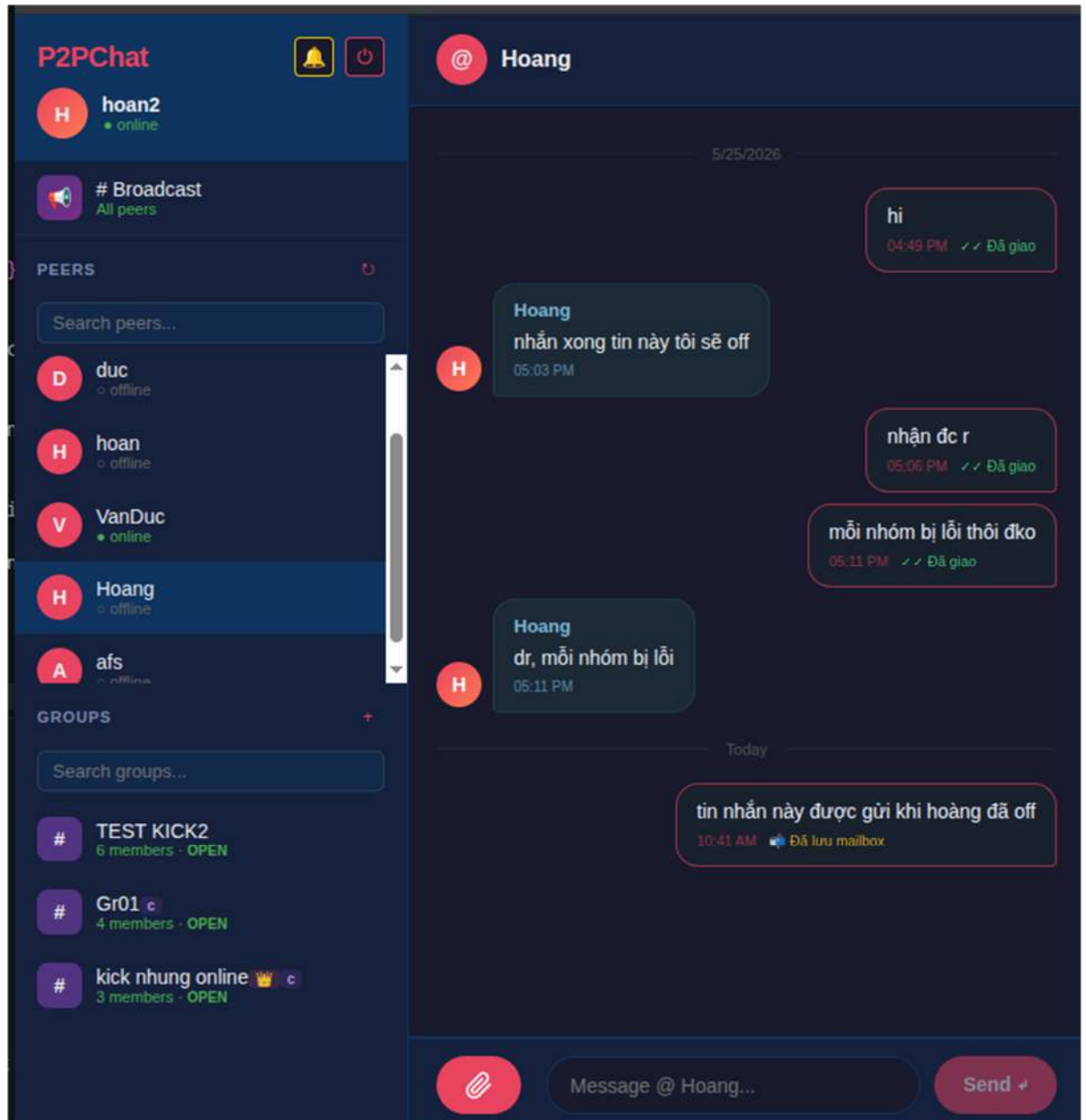
Khi cả người gửi và người nhận đều đang online, tin nhắn sẽ ngay lập tức được gửi đi và hiển thị trạng thái tin nhắn là “*Đã giao*”.



Hình 4.6 Gửi tin nhắn khi 2 peer đều online

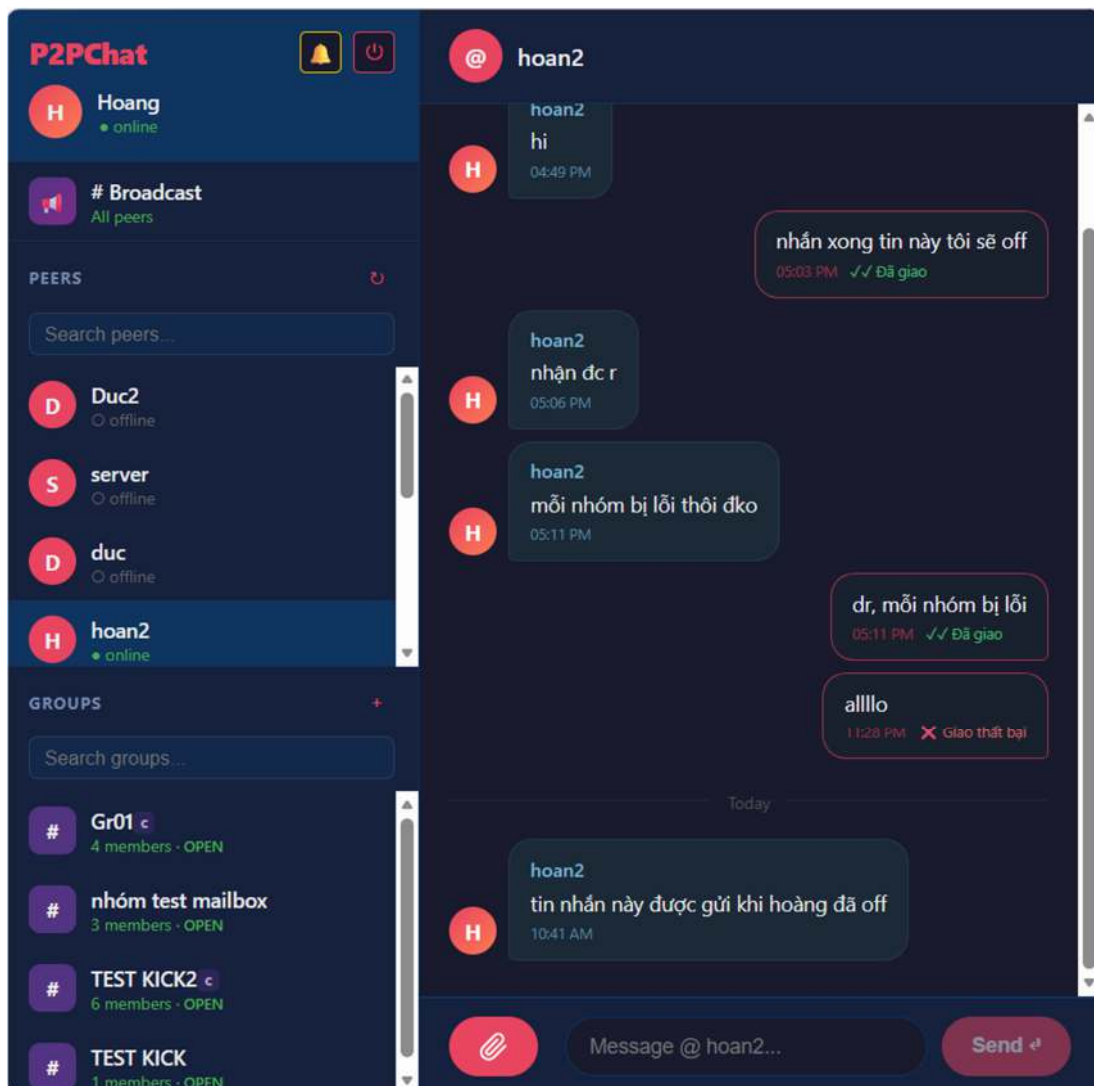
4.5.2 Chat khi một peer offline

Trong trường hợp người nhận (Peer nhận) đang ở trạng thái offline, tin nhắn sẽ được gửi lên mailbox



Hình 4.7 Chat cho một peer offline

Tin nhắn khi này sẽ ở trên mailbox server, tới khi Peer kia quay trở lại (online) thì sẽ ngay lập tức nhận được tin nhắn dù trước đó đã offline.



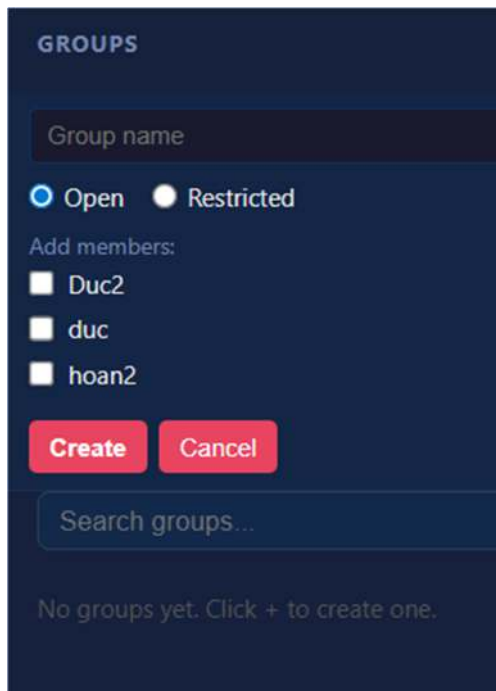
Hình 4.8 Nhận được tin nhắn sau khi peer online lại

4.6 Chat nhóm

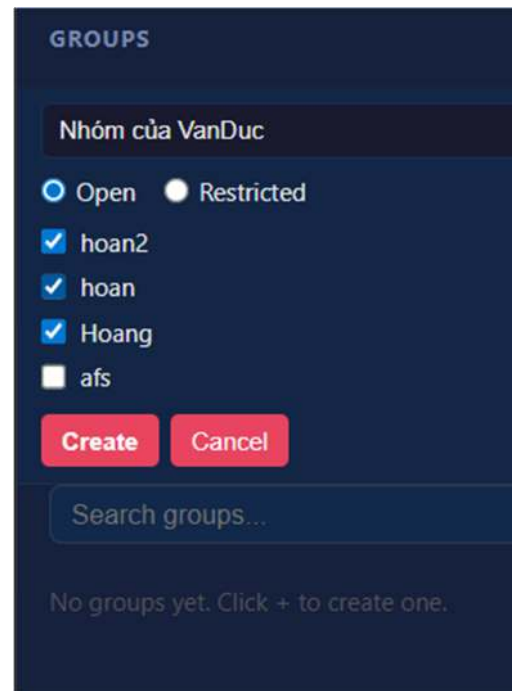
Để chat nhóm cần phải tạo ra một nhóm cho các thành viên. Có 2 trạng thái nhóm là Open và Restricted.

- *Open*: Trạng thái phòng mở, các thành viên tham gia có quyền thêm thành viên khác vào nhóm một cách tự do.
- *Restricted*: Trạng thái phòng giới hạn, chỉ chủ nhóm và các thành viên được bầu làm điều phối viên (COORD) mới có quyền thêm thành viên vào nhóm.

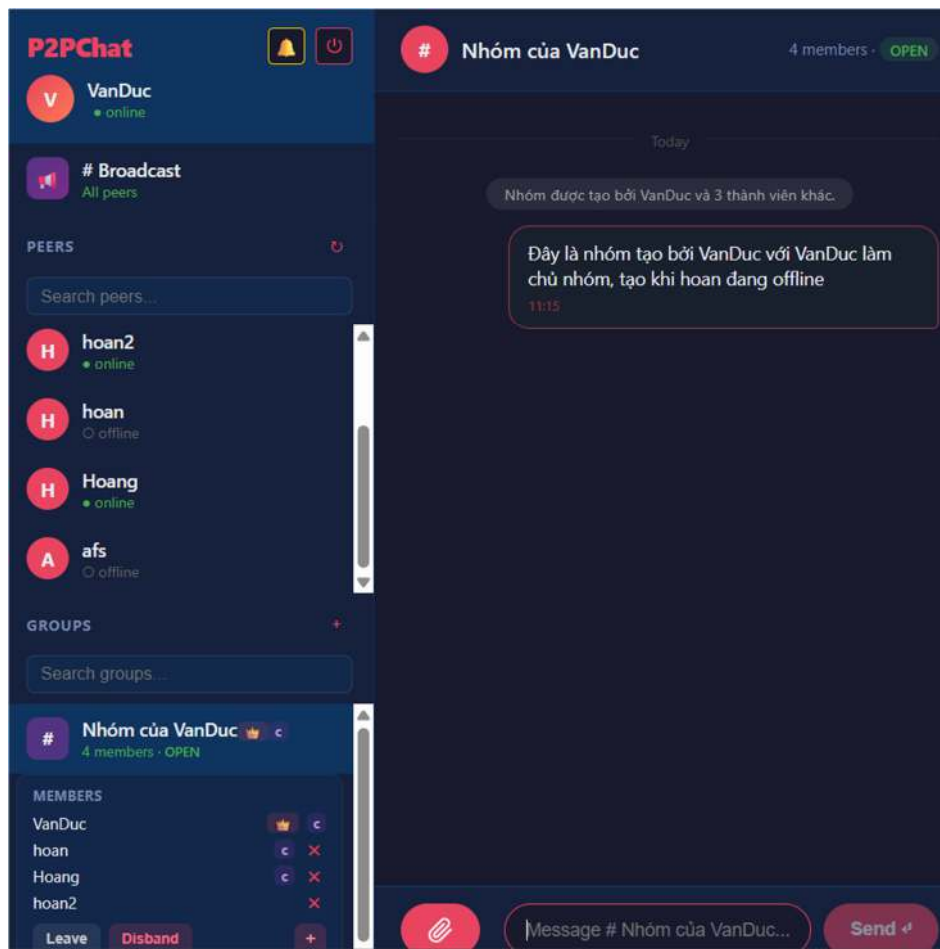
Nhóm có thể thêm thành viên dù họ đang online hay offline trên hệ thống và cần nhập tên nhóm (tên nhóm không được trống).



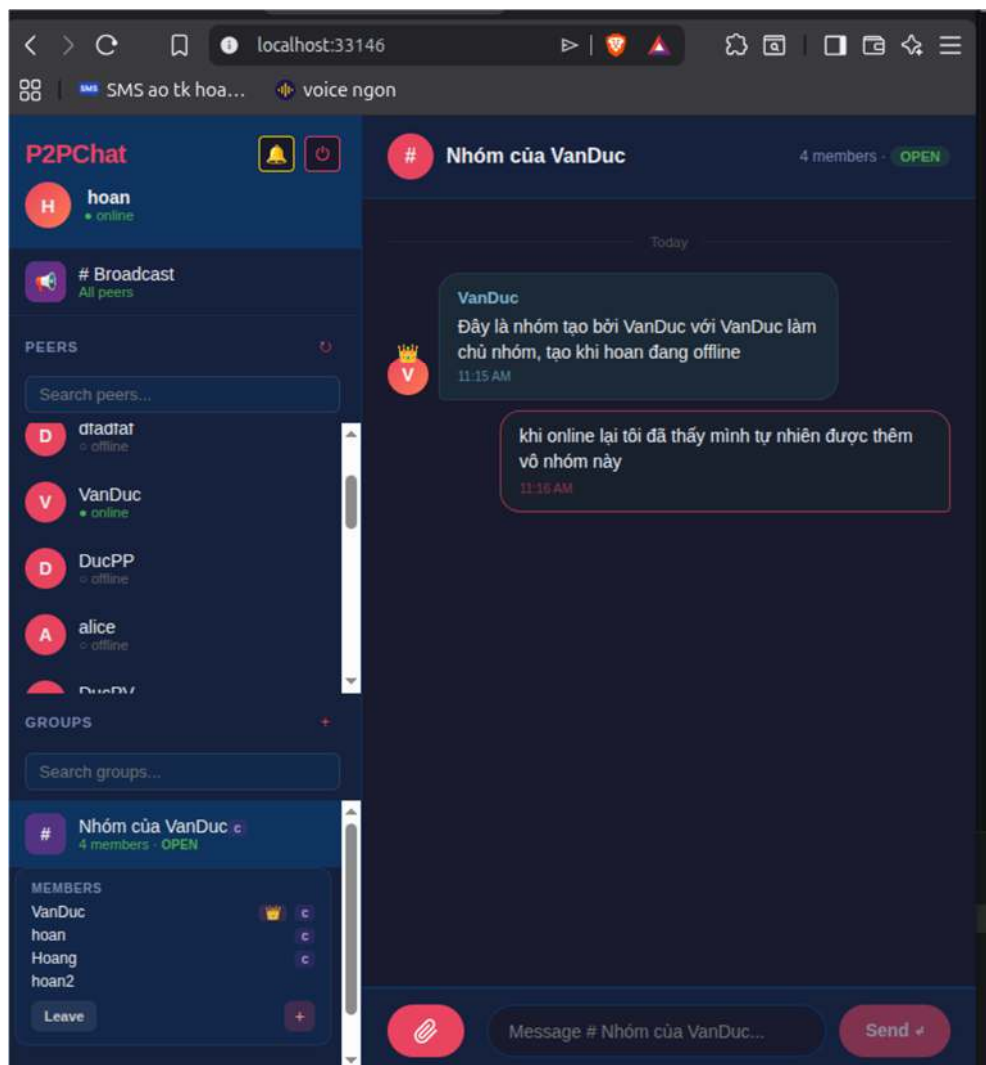
Hình 4.8a Tạo nhóm



Hình 4.8b Thêm thành viên vào nhóm



Hình 4.9 Tạo nhóm khi thành viên hoan đang offline



Hình 4.10 Khi hoan online thì đã vào được nhóm

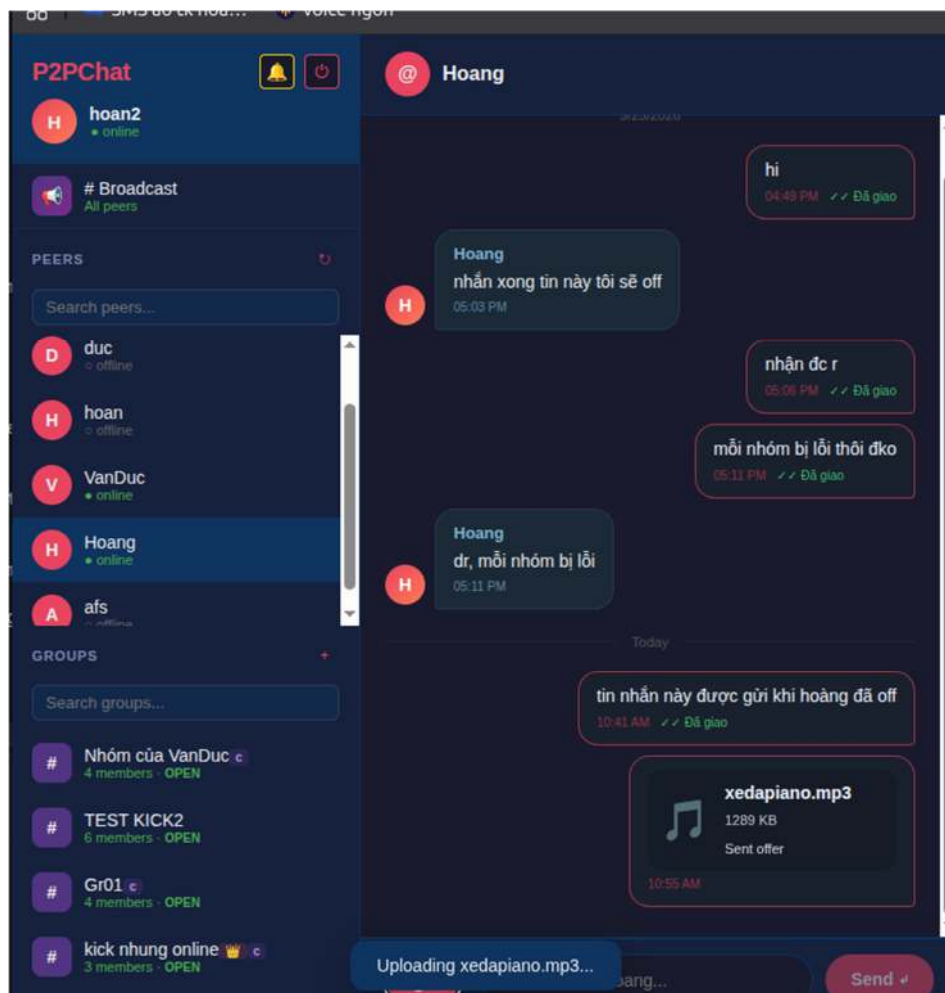
Khi VanDuc tạo nhóm trong lúc hoan đang offline, sau khi hoan đăng nhập trở lại, hệ thống tự động thêm người dùng này vào nhóm và hiển thị các tin nhắn đã được gửi trước đó. Điều này cho thấy dữ liệu nhóm và cơ chế đồng bộ tin nhắn hoạt động chính xác, đảm bảo thành viên có thể nhận được lịch sử tin nhắn kể từ thời điểm được thêm vào nhóm.

4.7 Truyền file

Tính năng truyền tệp tin trong P2PChat được triển khai trực quan trên giao diện người dùng, cho phép chia sẻ các tệp tin trực tiếp giữa các Peer (hoặc gửi thông qua Mailbox nếu đối phương ngoại tuyến) mà không cần qua máy chủ trung gian.

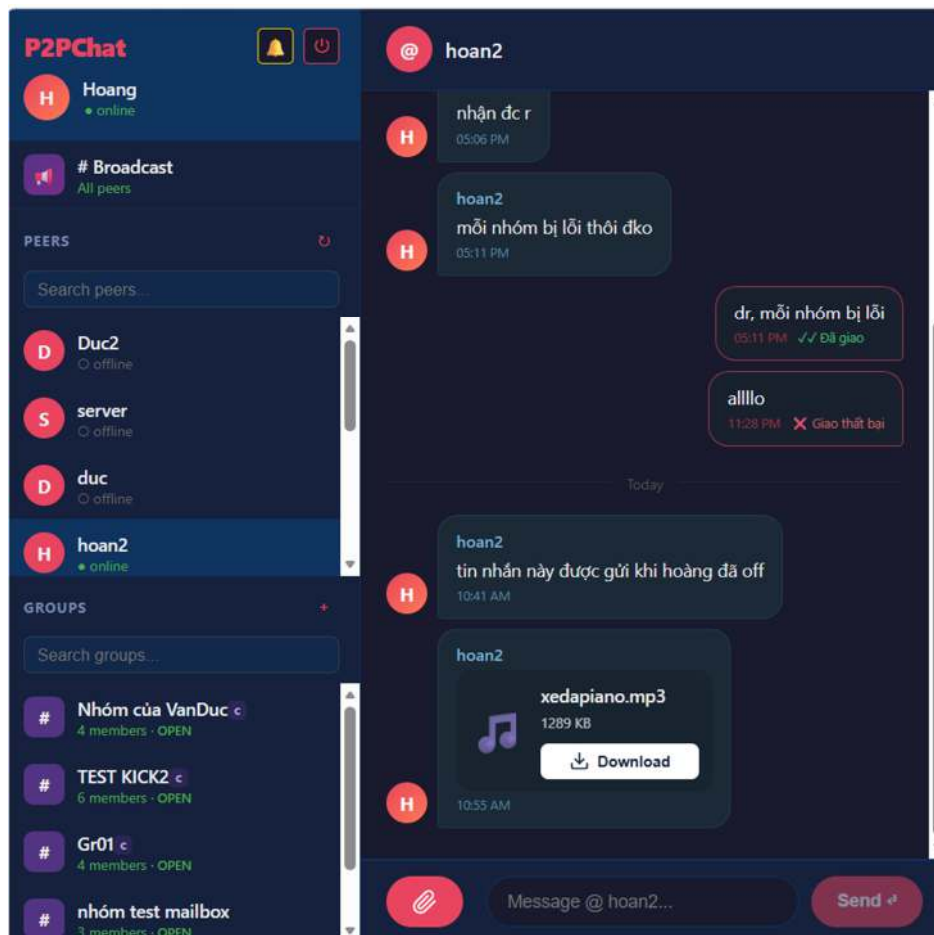
File được gửi đi dưới dạng 1 tin nhắn và tùy vào vai trò là người nhận hay gửi mà nó có giao diện khác nhau:

- **Với người gửi:** Nó là một tin nhắn có icon biểu tượng của dạng file gửi đi, tên file, kích thước file và trạng thái là *Sent offer* tức là đã thành công gửi và mở port để người khác có thể truy cập và download nó xuống.

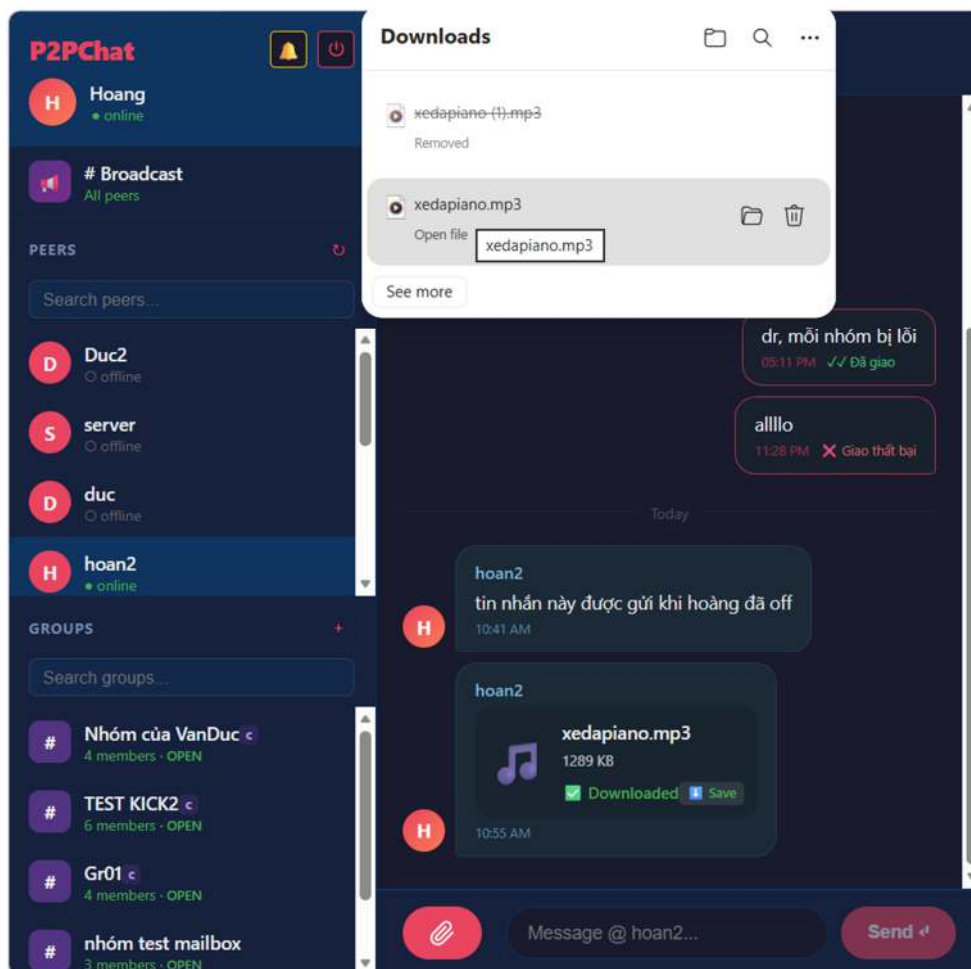


Hình 4.11 Gửi file qua tin nhắn riêng

- **Với người nhận:** Nó là một tin nhắn có icon, cũng có thông tin tên file, dung lượng file, khác biệt ở chỗ lúc này ở bên người nhận sẽ có một nút Download. Người nhận khi ấn nút này sẽ tự động kết nối tới cổng filePort bên gửi và tiến hành tải từng chunk file về.



Hình 4.12 Bên nhận đã nhận được tin nhắn file gửi

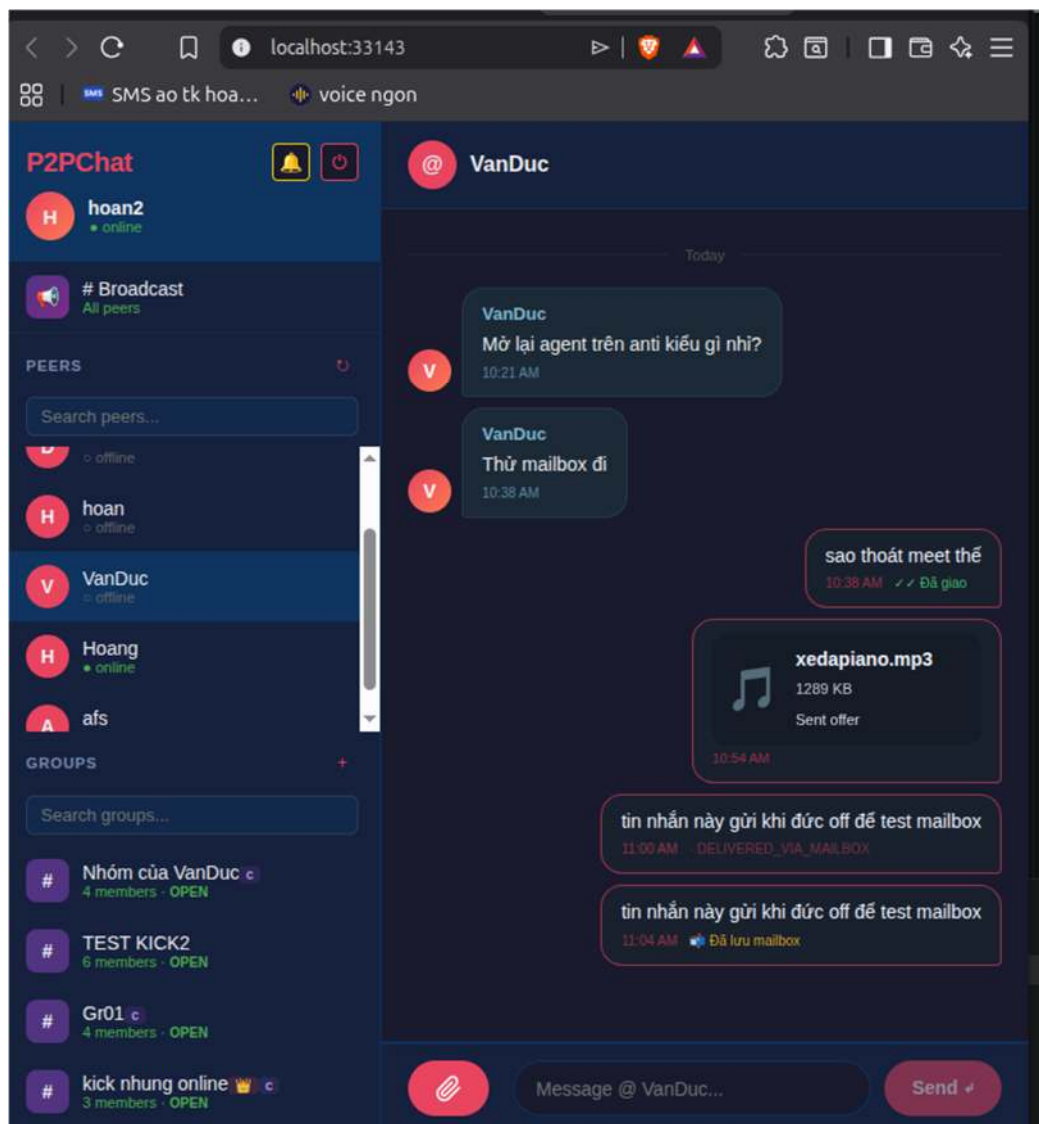


Hình 4.13 Khi ấn nút Download thì sẽ tiến hành tải file về máy

Sau khi ấn Download thì file sẽ được tiến hành tải về máy, tin nhắn đó sẽ cập nhật trạng thái file là đã được tải thành công chưa hay tải lỗi. Ngoài ra có thêm nút Save để khi người dùng có nhu cầu download lại file khi cần.

4.8 Mailbox & Outbox

Khi một peer tiến hành gửi một tin nhắn tới peer khác. Hệ thống kiểm tra trạng thái và phát hiện hiện peer đang Offline. Tin nhắn lập tức được chuyển hướng sang Mailbox Server. Tại đây, hệ thống thực hiện mã hóa nội dung tin nhắn và lưu trữ vào Database của Mailbox.



Hình 4.14 Trạng thái tin nhắn khi gửi cho người offline

Đồng thời hệ thống in ra Log báo tin nhắn đã lưu

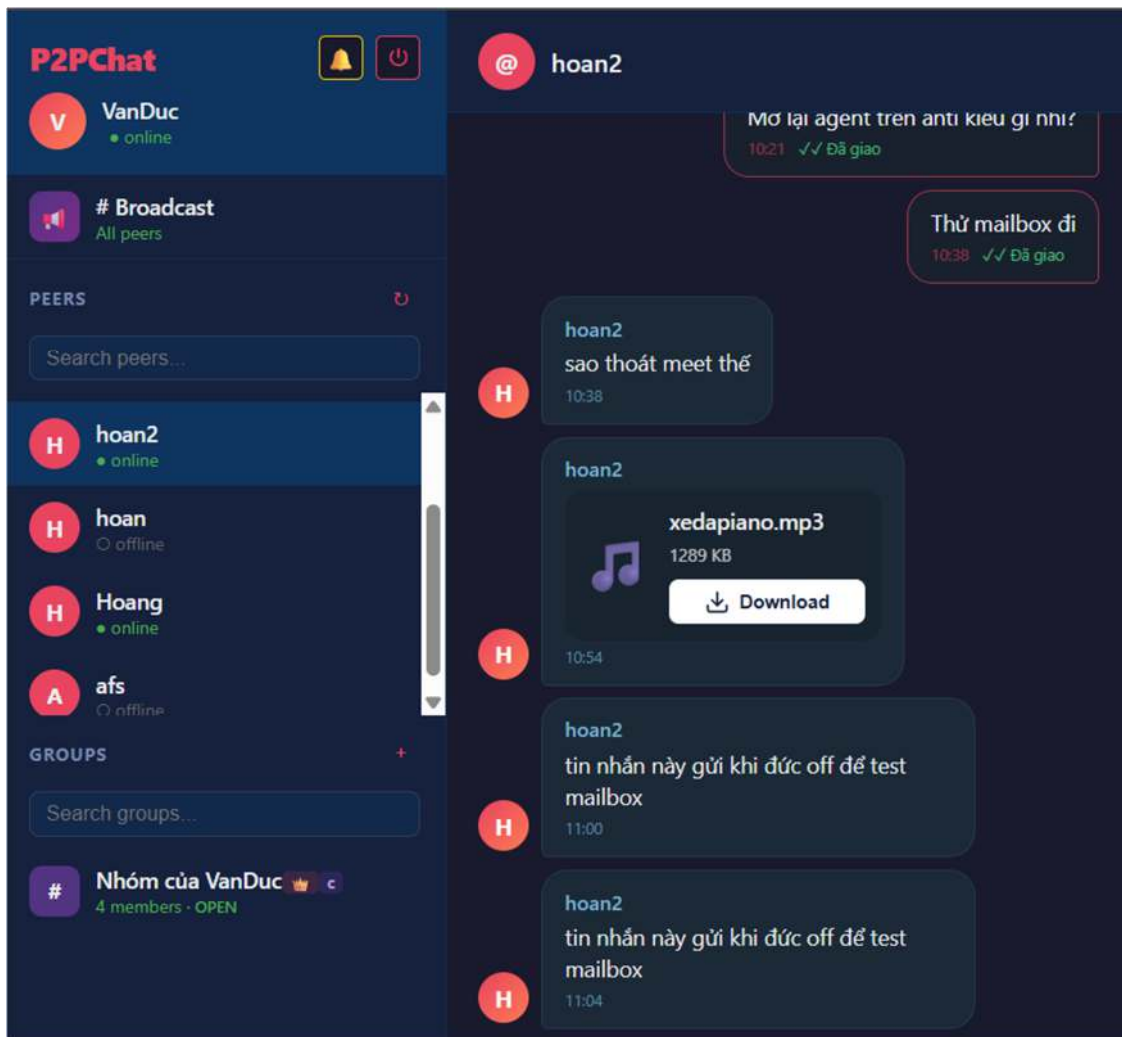
```

=== Mailbox Server running on port 9100 ===
[STORE] hoan2 đang lưu tin nhắn (ID: e15e24a1-2e51-4eee-bd9b-d2be52f31942) gửi tới VanDuc
[STORE] hoan2 đang lưu tin nhắn (ID: e15e24a1-2e51-4eee-bd9b-d2be52f31942) gửi tới VanDuc

```

Hình 4.15 Hệ thống ghi nhận Log đã lưu tin nhắn tại Mailbox Server

Sau khi peer online trở lại sẽ gửi tín hiệu đồng bộ hoặc Mailbox Server chủ động quét Database và phát hiện có tin nhắn đang chờ xử lý peer. Mailbox Server tiến hành lấy dữ liệu tin nhắn đã mã hóa từ Database và chuyển phát về cho peer.



Hình 4.16 Peer nhận được tin nhắn sau khi online trở lại

Đồng thời hệ thống in ra log thông báo đã gửi tin nhắn:

```
=== Mailbox Server running on port 9100 ===
[STORE] hoan2 đang lưu tin nhắn (ID: e15e24a1-2e51-4eee-bd9b-d2be52f31942) gửi tới VanDuc
[STORE] hoan2 đang lưu tin nhắn (ID: e15e24a1-2e51-4eee-bd9b-d2be52f31942) gửi tới VanDuc
[PULL] Đã trả về 1 tin nhắn cho VanDuc
```

Hình 4.17: Hệ thống in ra Log đã gửi tin nhắn.

```

-----
Sender: hoan | Receiver: Hoang | Algo: RSA-OAEP-256+A256GCM
Payload: {"version":1,"algorithm":"RSA-OAEP-256+A256GCM","senderKeyId":"ffd089a874ddc8ec282b95
d42498a985","receiverKeyId":"9d56548712682f432ecdc7082f5fc450","iv":"3VqPcJkUlc1d6Lxd","wrappe
dKey":"IA4833ewNBmXhP1KAw1KFEx05XnAwimwqGgFyo6pHdk6nDCTjuNyhL/QkEziZgDiN5aV5cbiZFh63UgYxTGAaLL
WmF5XP9aROHKc+BI4yGVA6nWw62bndiLPwCogXJEjx51CvLZ8E3frNtKq16UuJNH2y3rV6TBw+aqgocAN7VX0ON5bT8EeQ
/5Sqq9Im0iSd5wU30Cn3cglARLr7JRFuqtahQH/GT1wAaihG4A0s2b4/U0tLgkyqsZKlxiBEsp7LUpzstRCNi//XMI8j/o
mVpVdoJqez3Zipc7q8RdSwE+e6iYZ8purL05fLfxgjZhjf8Y1M2kqKMy7Co+90MpMFw\u003d\u003d","ciphertext":
"Dh8fyZQYrY3FYkxnygyqx47VwGwJLo867eSZgyQhhDT27bhZ4f8fwjvX7cX3nIkYRi9Fp05XtBLEJhdL+53XVQd0ThLT
u1Zr7jJGhEEc9yg72Cp5AfIBjxtqM4Ag2KUXTyhcv903+LAPI2tjz8QCSbQrupHapfAC+TjY4Hd4JNb1ibuJh7NkfixfAm
TUFvKz3o/wjJFK0JBCCSZUKR0lb7nguj4d7hEPxNqK266b8LW0z76NBHXRult8hElfq7ICfr62UiqKpxo56/GEGLSWDDv7
hErIauawlfjqRZtnNxx+/Q0sUZCbQJEHgJyL4zzVwRidG5DVvu2Q+QFfaXKQ1caF0qA+Gtw6DnsZ00Hsx+IDnZgZg\u003
d\u003d"}
-----

Sender: hoan | Receiver: TaoKhiBootstrapSap | Algo: RSA-OAEP-256+A256GCM
Payload: {"version":1,"algorithm":"RSA-OAEP-256+A256GCM","senderKeyId":"ffd089a874ddc8ec282b95
d42498a985","receiverKeyId":"c64cfbcc90fa36b31199ce05fe37c430","iv":"oKvdhcvk2Pa7g0dP","wrappe
dKey":"LPq40AVg3AJjARApI5SW1NbI/UKeiJULm1eL2HT0ip8nhA8noPzeAQ2N3nR9jxrIlfiTT7V3rEQzkQHcd/4o6yC
eE41U0bm0Lw6hUBeFkPjSAjWU3MIFEMrQ8Jy4sIqXRUC6spim5w6LVLYZsXP5IzguSYwhSY77tUQ3yNVsjcPKep/p1NXTE
QUgDZ31bsMDtzFVz9fxFcXJKMzJMgKpi1ywwiVwUrmnxdeOAXjHuabTC9geiJ0hxZQPtvvbJwMTZ6D9eR0DsEnbu+K/m10
5kIZVDVbft1x6x0UevfXdZ9qeOniOKaxyrF0PDw5mV9d4kx5+LVAvHSH6NI17kQYayA\u003d\u003d","ciphertext":
"asprJYjCC2b9300+htND/QwqogSavJ88eM94e0egqiCgHauhnzpt/5lwAmv3NRxNfdE6to7upUpaTvqybv1isehsIkATP
oYci3qHUKRARHM4o0VmU8FMgXepGOXDoxsGpV66qH4APJ3hR4M7B4m7ne619/PwXDCMDnc9oBTJQA7fccU07yHUmK1gtOj
phIMw7firFaQTaZ0xiIFnXLkJezfijSjWJleWrpjAablads53LlEMETj50d7/ILN4KM254oZ+tz6moaTzE4kaqNDkaxGHV
7BlquYQa8q9CypYAXGZIPxnzWZ34VmwGSwS7rREUf0SYHw0kuwviCEl8JOG5rdI6Qr116hoV/gP7TUBCYl7+69xwT6j3VX
WwotEt3QP4QVOHA\u003d\u003d"}

```

Hình 4.18 Tin nhắn đã được mã hóa nằm trong Database của Mailbox

Tin nhắn sau khi chuyển phát thành công sẽ được xóa khỏi Mailbox để giải phóng bộ nhớ.

CHƯƠNG V. THỬ NGHIỆM HỆ THỐNG & ĐÁNH GIÁ

5.1 Kịch bản thử nghiệm

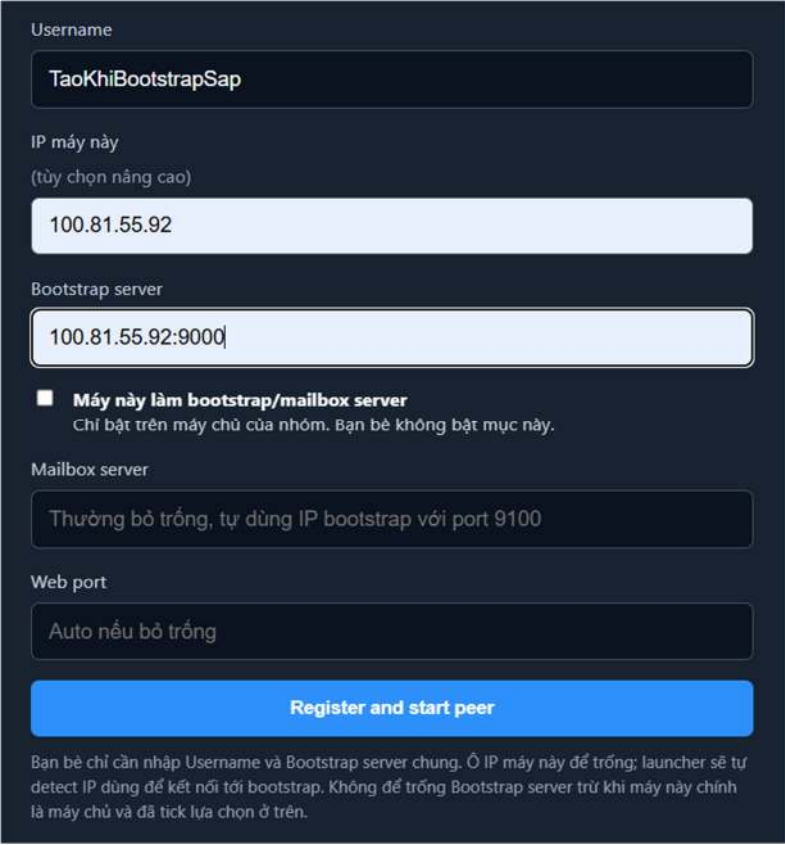
5.1.1 Kịch bản 1:

Kịch bản: Bootstrap Server gặp sự cố và bị sập

Mục đích: Kịch bản này nhằm đánh giá khả năng chịu lỗi và tính sẵn sàng của mạng P2P khi Bootstrap Server gặp sự cố đột ngột. Đồng thời, kiểm thử cơ chế tự phục hồi của hệ thống thông qua việc xác minh xem Bootstrap Server sau khi khởi động lại có thể tự động thu thập, tái cấu trúc và khôi phục chính xác trạng thái đăng ký cũng như thông tin định tuyến của các Peer đang hoạt động hay không.

Mô tả kịch bản:

1. Khi Bootstrap Server ngừng hoạt động hoàn toàn.
2. Các peer mới bật ứng dụng lên sẽ gửi yêu cầu tham gia mạng tới Bootstrap server.



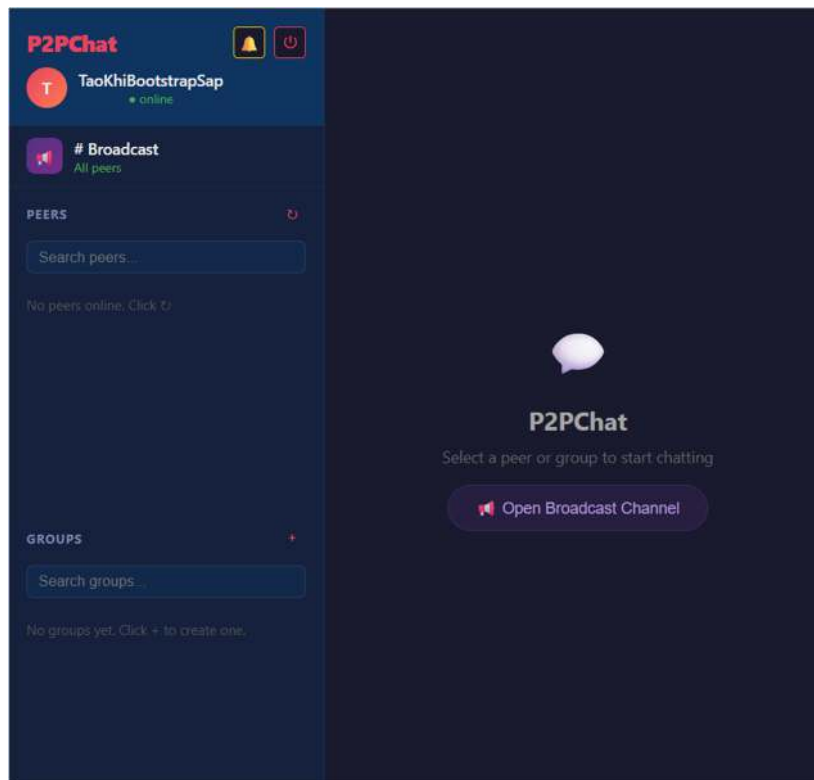
The image shows a dark-themed web interface for registering a peer. It contains several input fields and a button:

- Username:** A text input field containing "TaoKhiBootstrapSap".
- IP máy này (tùy chọn nâng cao):** A text input field containing "100.81.55.92".
- Bootstrap server:** A text input field containing "100.81.55.92:9000".
- Máy này làm bootstrap/mailbox server:** A checkbox that is currently unchecked. Below it is a note: "Chỉ bật trên máy chủ của nhóm. Bạn bè không bật mục này."
- Mailbox server:** A text input field containing "Thường bỏ trống, tự dùng IP bootstrap với port 9100".
- Web port:** A text input field containing "Auto nếu bỏ trống".
- Register and start peer:** A prominent blue button at the bottom.

Below the button, there is a small disclaimer in Vietnamese: "Bạn bè chỉ cần nhập Username và Bootstrap server chung. Ở IP máy này để trống; launcher sẽ tự detect IP dùng để kết nối tới bootstrap. Không để trống Bootstrap server trừ khi máy này chính là máy chủ và đã tick lựa chọn ở trên."

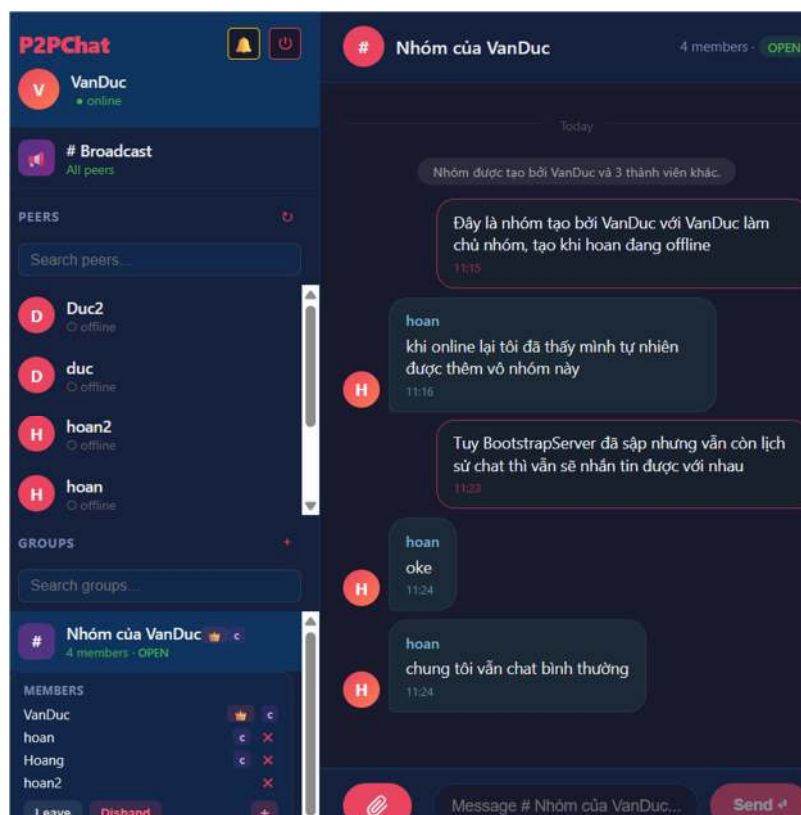
Hình 5.1 Giao diện đăng ký peer

3. Kết quả là peer mới không thể vào mạng.



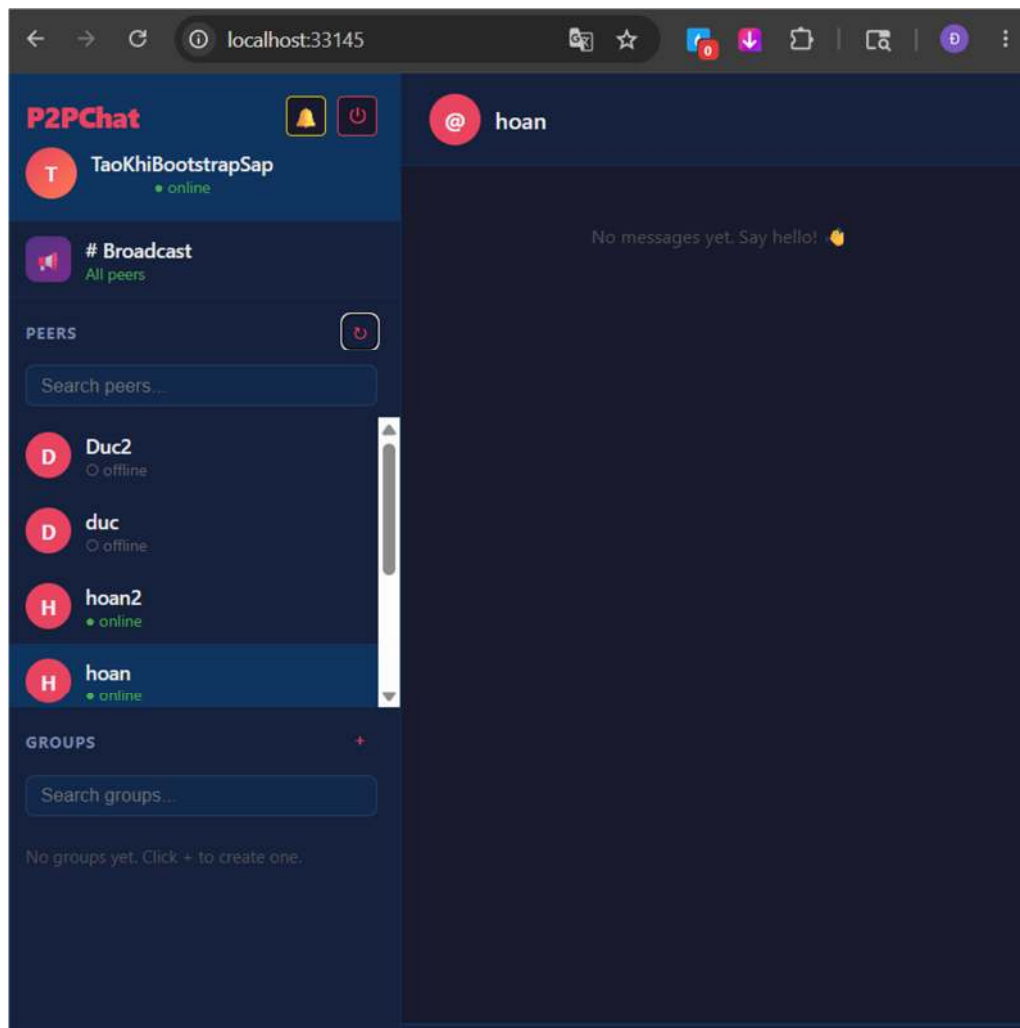
Hình 5.2 Giao diện peer khởi tạo khi Bootstrap Server bị sập

4. Các peer đã tham gia từ trước thực hiện nhắn tin cho nhau



Hình 5.3 Các peer giao tiếp khi Bootstrap Server sập

5. Các tin nhắn vẫn được truyền tải dữ liệu trực tiếp với nhau.
6. Bootstrap Server khởi động lại.
7. Người tạo mới có lại được trạng thái của những peer đăng ký với bootstrap.



Hình 5.4 Giao diện sau khi Bootstrap Server khởi động lại

Kết luận: Kết quả thực nghiệm chứng minh mạng P2P có khả năng chịu lỗi tốt khi sự cố sập Bootstrap Server không làm gián đoạn các cuộc hội thoại hiện tại giữa các Peer. Đồng thời, cơ chế tự phục hồi hoạt động chính xác, cho phép Bootstrap Server tự động tái cấu trúc và khôi phục toàn vẹn trạng thái mạng ngay sau khi khởi động lại mà không cần cấu hình thủ công.

5.1.2 Kịch bản 2

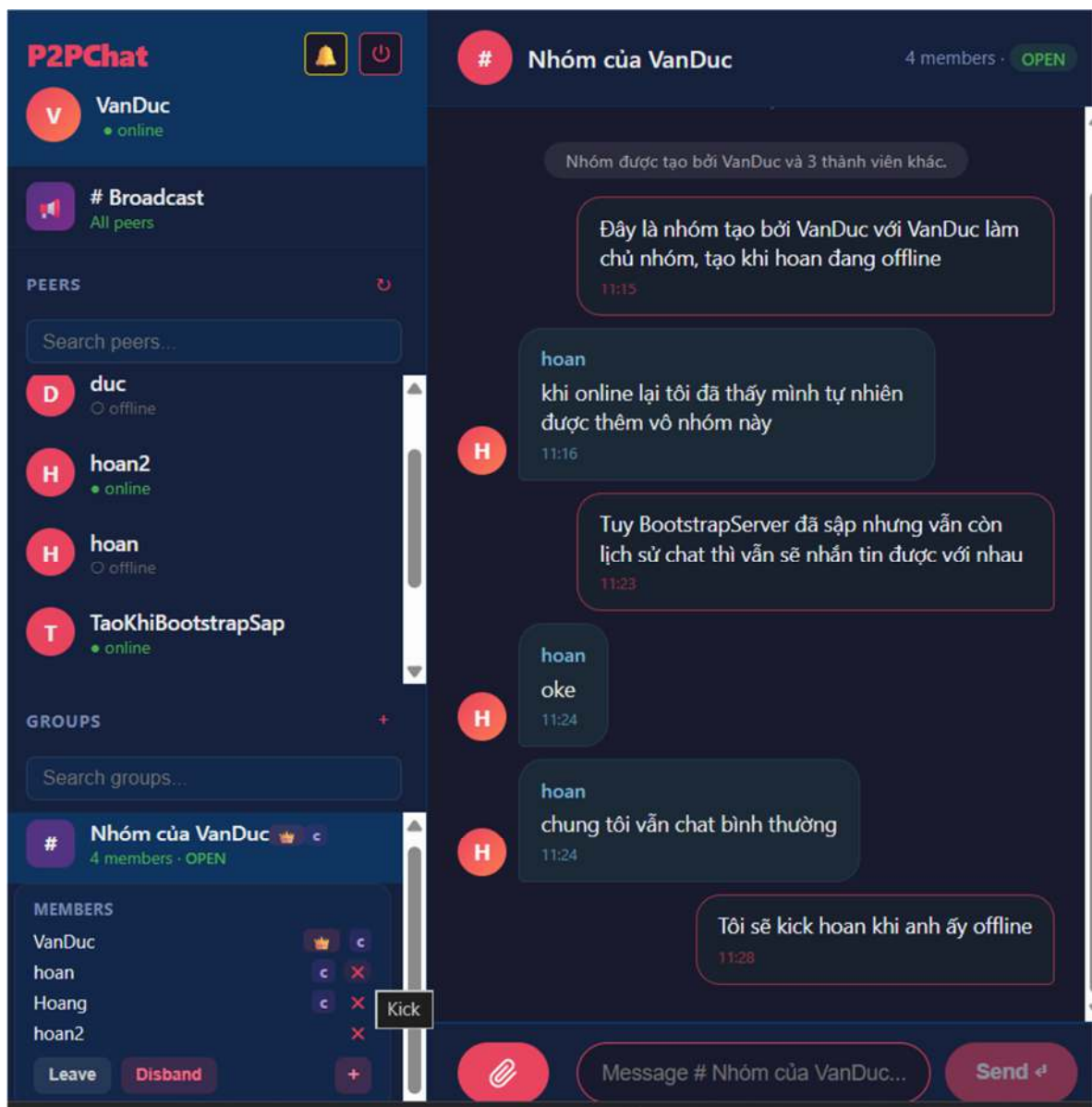
Kịch bản: Thử nghiệm đuổi thành viên khỏi nhóm khi người đó offline.

Mục đích: Kiểm tra xem cơ chế đuổi thành viên có hoạt động đúng yêu cầu khi thành viên đó offline - trường hợp dễ dẫn tới thành viên ma (Thành viên đã bị đuổi khỏi

nhóm nhưng vẫn tồn tại trong nhóm và có thể nhắn tin và xem tin nhắn do ở bộ nhớ cục bộ thành viên đó vẫn là người của nhóm). Ngoài ra, còn là kiểm tra cơ chế bầu cử lại Điều phối viên khi thành viên là Điều phối viên bị đuổi.

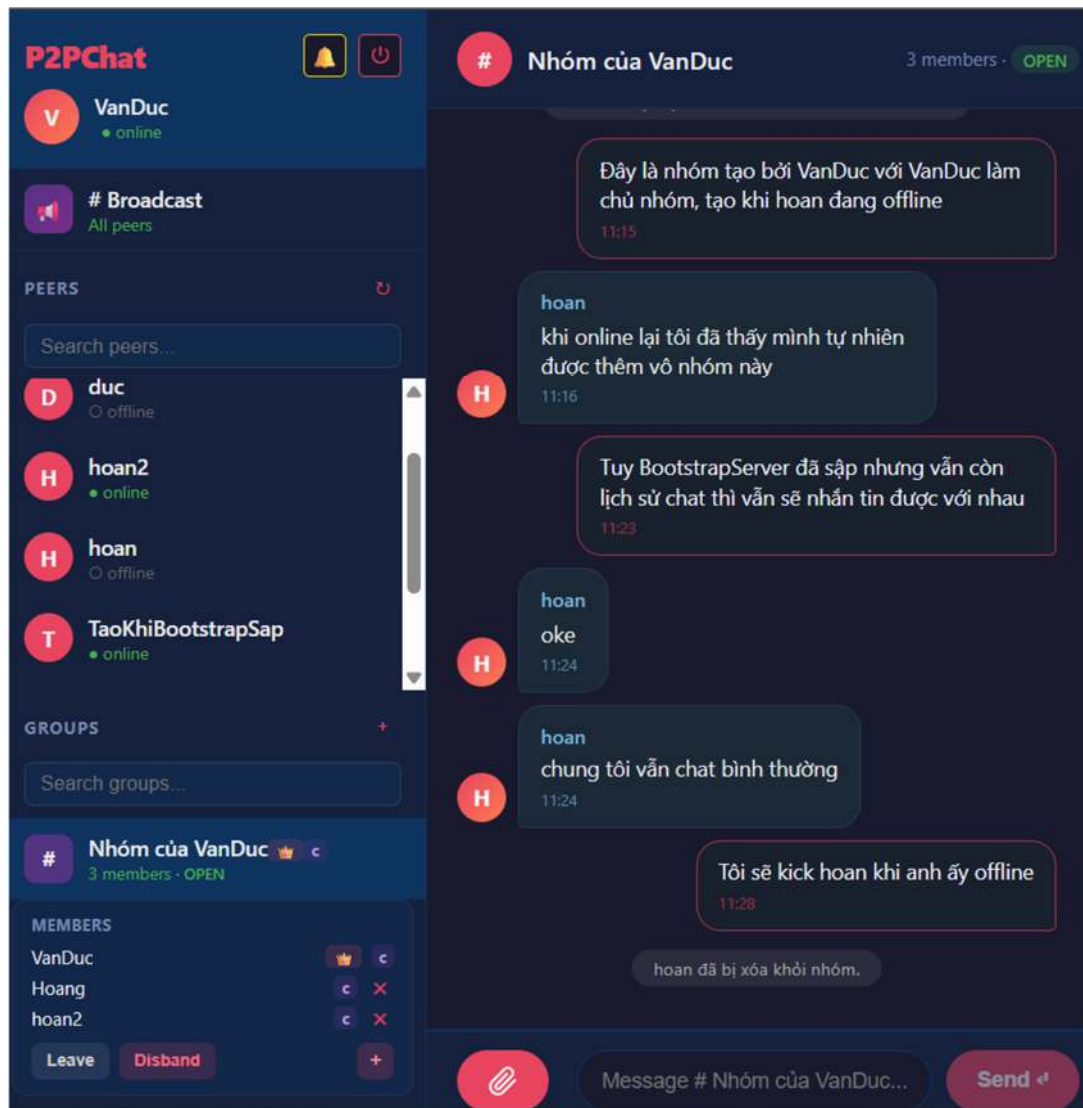
Mô tả kịch bản:

1. VanDuc là chủ nhóm của nhóm “Nhóm của VanDuc” muốn đuổi hoan - hiện đang là Điều phối viên của nhóm “Nhóm của VanDuc”.
2. Hoan khi này đang offline khỏi hệ thống.
3. VanDuc tiến hành kick (đuổi) hoan ra khỏi nhóm với quyền hạn của chủ nhóm.



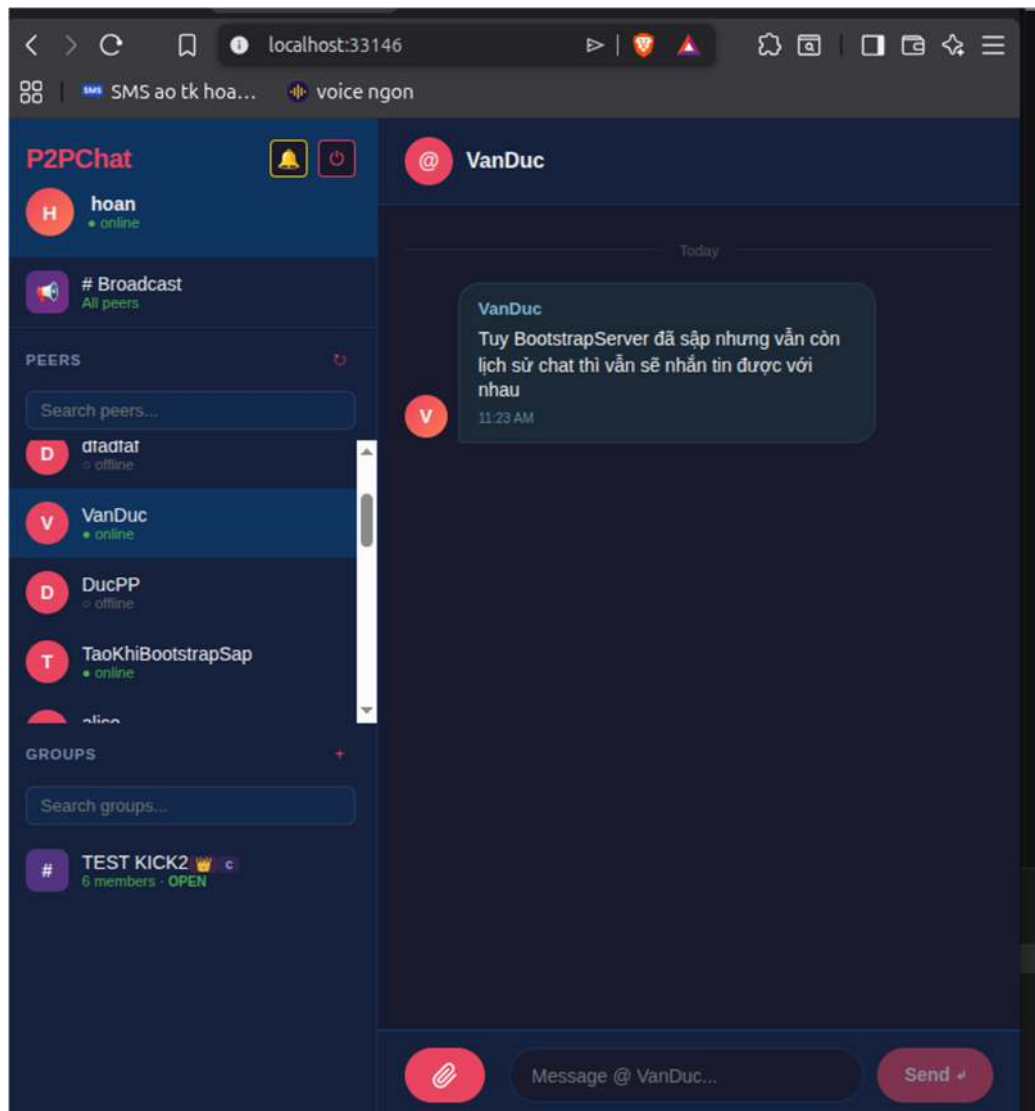
Hình 5.5 VanDuc đuổi hoan ra khỏi nhóm

4. Khi VanDuc tiến hành đuổi hoan, danh sách nhóm được cập nhập lại, không còn hoan trong danh sách thành viên nhóm.
5. Có thông báo lịch nhóm là “hoan đã bị xóa khỏi nhóm”
6. Các Peer thành viên tiến hành bầu cử lại Điều phối viên, lúc này hoan2 nghiễm nhiên được đẩy vai trò từ thành viên bình thường lên thành Điều phối viên.



Hình 5.6 Hoan bị xóa khỏi nhóm và thành viên bầu lại COORD

7. Sau khi offline xuất quá trình đuổi khỏi nhóm, hoan online lại.
8. Lúc này hoan không còn thấy mình trong nhóm nữa. Trên danh sách nhóm của hoan cũng không còn nhóm “Nhóm của VanDuc” nữa.



Hình 5.7 Hoan online và không còn thấy nhóm nữa

Kết luận: Với việc hoan thật sự bị đuổi khỏi nhóm, không thể vào nhóm hay nhận được tin nhắn hay nhắn tin vào cho thấy cơ chế đuổi thành viên khỏi nhóm của hệ thống thực sự hoạt động đúng thiết kế. Ngoài ra, khi hoan là một Điều phối viên bị đuổi khỏi nhóm, các thành viên trong nhóm đã tính toán lại và chọn ra 3 Điều phối viên mới chính là có thêm hoan2.

5.1.3 Kịch bản 3

Kịch bản: Store-and-Forward qua Peer Trung Gian khi Mailbox Server sập

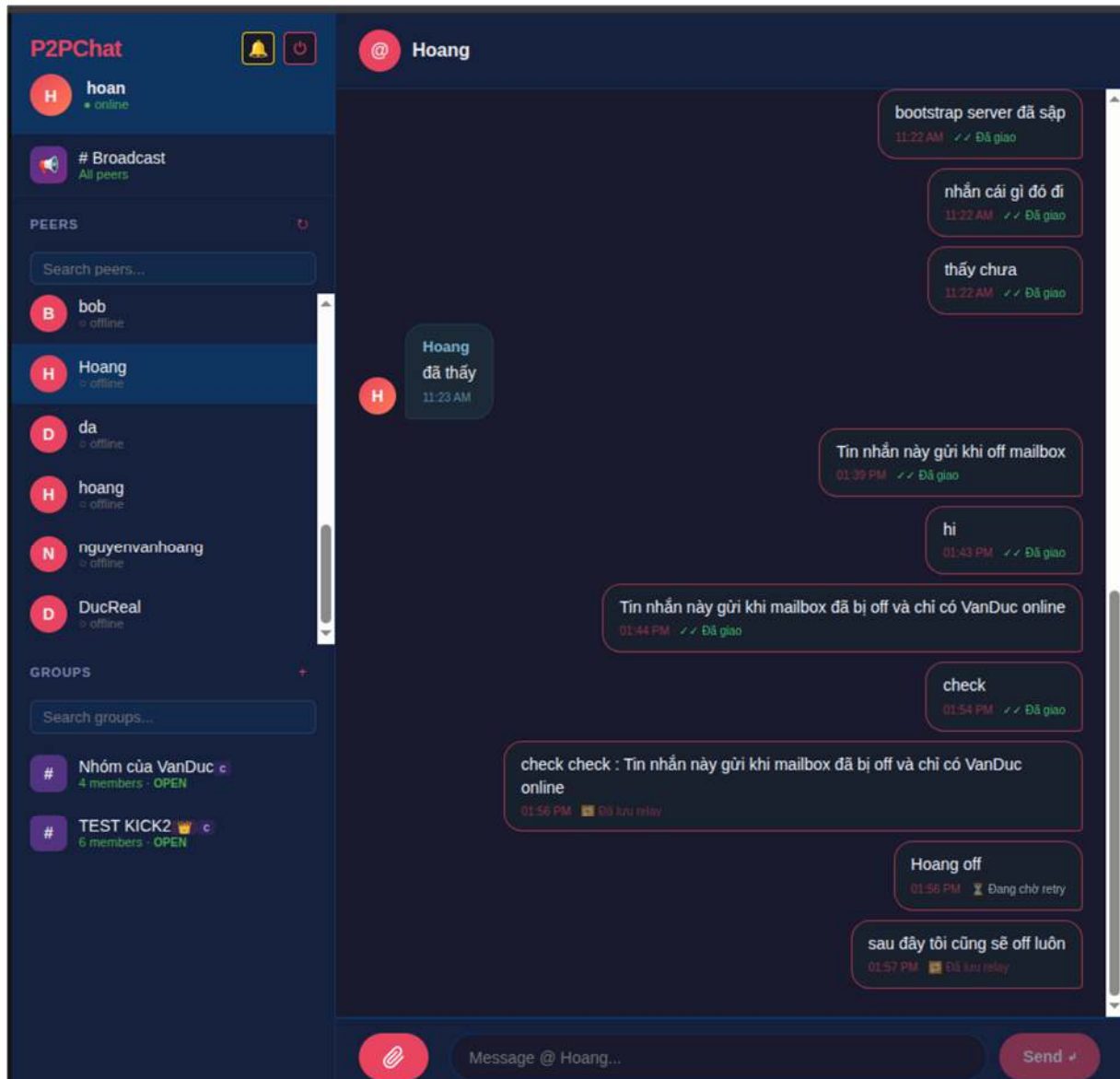
Mục tiêu: Kiểm tra khả năng chịu lỗi của hệ thống truyền tin khi cả người nhận (B) và cơ chế lưu trữ dự phòng ban đầu (Mailbox Server) đều không khả dụng. Thử nghiệm cơ chế dự phòng thứ cấp: Store-and-Forward qua Peer Trung Gian.

Mô tả kịch bản:

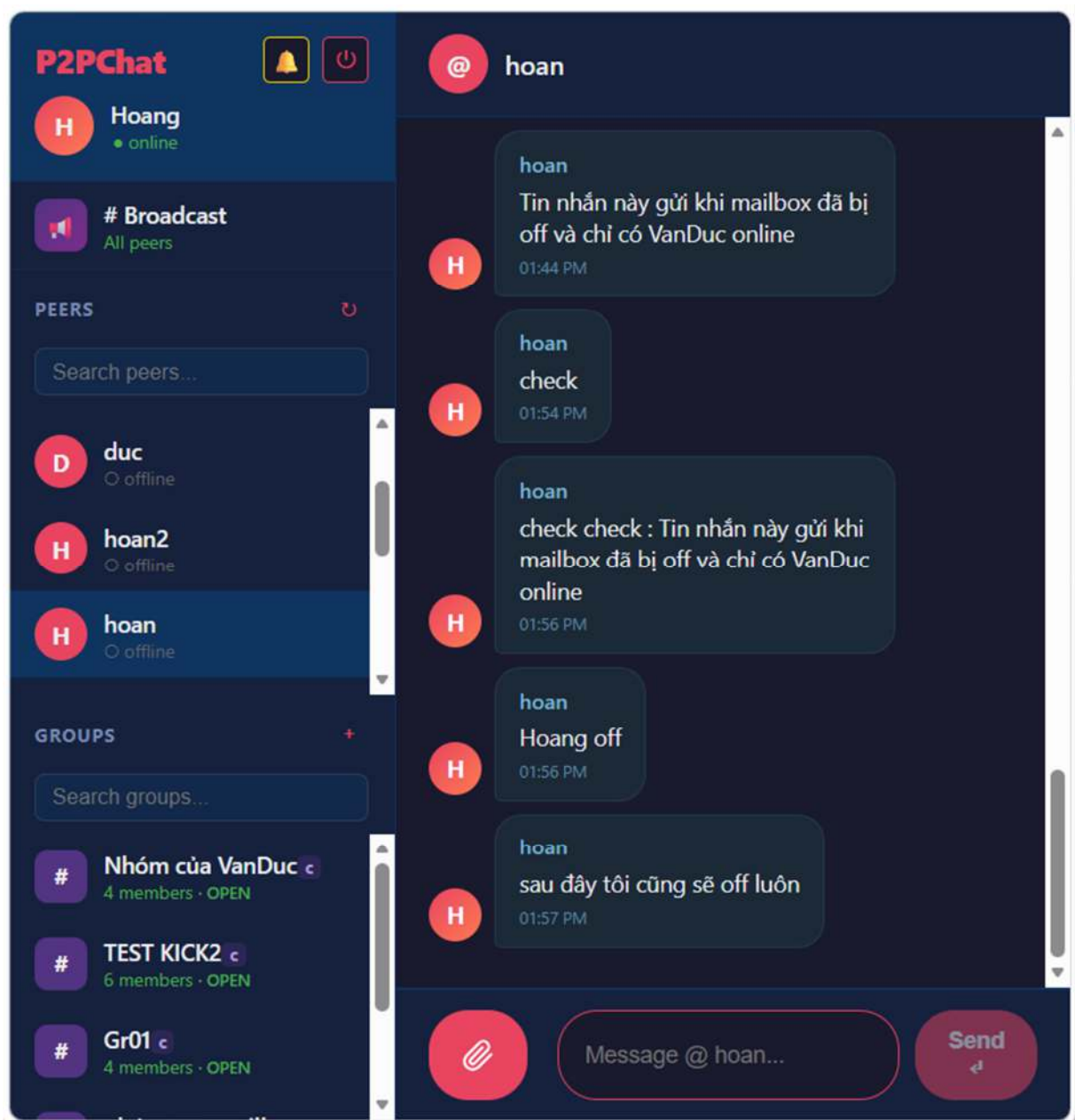
1. Peer B ngoại tuyến và Mailbox Server sập: Peer B (người nhận) đang offline. Mailbox Server (cơ chế Store-and-Forward chính) bị lỗi hoặc sập. Peer A và Peer C (Peer trung gian) đang online.
2. Peer A gửi tin nhắn: Peer A gửi tin nhắn trực tiếp đến Peer B.
3. Thất bại truyền trực tiếp và Mailbox:
 - Việc gửi trực tiếp thất bại vì Peer B đang ngoại tuyến.
 - Peer A cố gắng lưu tin nhắn vào Mailbox Server bằng lệnh STORE_MESSAGE, nhưng thất bại do Mailbox đang sập.
4. Kích hoạt Store-and-Forward qua Peer Trung Gian: Hệ thống chuyển sang cơ chế dự phòng qua Peer Trung Gian.
 - Peer A mã hóa đầu cuối (E2EE) nội dung tin nhắn cho Peer B.
 - Peer A chọn Peer C (một peer đang online) làm Primary Relay và gửi bản tin đã mã hóa đó cho Peer C bằng tin nhắn STORE_MESSAGE để nhờ giữ hộ. Peer C chỉ lưu trữ dữ liệu đã mã hóa và không thể đọc được nội dung.
 - Peer A lưu trạng thái tin nhắn vào Outbox cục bộ (Outbound_Messages) để theo dõi.
5. Peer A ngoại tuyến: Peer A ngắt kết nối khỏi mạng (offline).
6. Peer B kết nối lại: Peer B (người nhận) online trở lại.
7. Peer Trung Gian chuyển tiếp (RELAY): Peer C (Relay Peer) phát hiện Peer B đã online. Peer C chủ động chuyển tiếp bản tin đã mã hóa mà Peer A nhờ giữ hộ đến Peer B.
8. Giao nhận thành công: Peer B nhận tin nhắn, giải mã bằng Private Key RSA cục bộ và khôi phục nội dung gốc. Peer B gửi DELIVERY_ACK đến Peer C và có thể gửi tín hiệu đến Mailbox Server (nếu đã hoạt động lại) để cập nhật trạng thái. Tin nhắn được giao thành công cho Peer B dù cả Peer gửi (A) và Mailbox Server đều đã offline tại thời điểm giao.

Kết luận: Kịch bản này kiểm tra khả năng chịu lỗi và tính bền vững của việc giao nhận tin nhắn (Store-and-Forward) khi một thành phần tập trung (Mailbox Server) gặp

sự cố, khẳng định luồng ưu tiên: Direct P2P -> Mailbox Server -> Relay Peer -> Local Outbox Retry.



Hình 5.8. Peer gửi (hoan) chuyển tiếp tin nhắn qua Peer trung gian (VanDuc) do Peer nhận (Hoang) và Mailbox đều offline



Hình 5.9. Peer nhận (Hoang) nhận tin nhắn thành công từ Peer trung gian (VanDuc) dù Peer gửi (hoan) đã offline

5.1.4 Kịch bản 4

Kịch bản: Mô phỏng churn (peer tham gia/rời mạng liên tục).

Mục đích: Đánh giá khả năng chịu lỗi và độ bền vững của ứng dụng chat P2P trước việc peer tham gia/rời mạng liên tục. Qua đó, tối ưu hóa điểm cân bằng giữa tần suất PING/Heartbeat và chi phí băng thông điều khiển, nhằm đẩy nhanh tốc độ hội tụ bảng định tuyến, đồng thời nâng cao chất lượng dịch vụ thông qua việc tối đa hóa tỷ lệ phát tin thành công và giảm thiểu độ trễ truyền tin.

Mô tả:

1. Hệ thống khởi tạo một mạng lưới P2P gồm N peer. Các peer được định danh và liên kết với nhau theo một cấu trúc liên kết cụ thể.
2. Sử dụng mô hình mô phỏng hướng sự kiện (Event-Driven Simulation) để quản lý hàng đợi sự kiện theo dòng thời gian.
3. Các peer mới tham gia vào hệ thống tuân theo phân phối Poisson. Khi vào mạng, peer sẽ liên hệ với Bootstrap Node để cập nhật bảng.
4. Mỗi peer khi hoạt động sẽ được gán một Session Time ngẫu nhiên tuân theo Phân phối Mũ (Exponential Distribution). Khi hết thời gian sống, peer sẽ lập tức offline mà không báo trước.
5. Trong suốt thời gian mô phỏng, các peer còn sống sẽ liên tục thực hiện hành vi gửi tin nhắn chat ngẫu nhiên cho nhau để giả lập ứng dụng nhắn tin thời gian thực.
6. Song song với đó, các peer thực hiện cơ chế gửi gói tin điều khiển (Ping/Heartbeat) định kỳ để phát hiện các peer lân cận vừa bị sập nhằm cập nhật lại bảng.
7. Sau khi kết thúc kịch bản, hệ thống trích xuất dữ liệu để tính toán 3 chỉ số cốt lõi:
 - *Tỷ lệ phát tin thành công (Message Delivery Ratio - MDR)*: Phần trăm số tin nhắn chat đến được đích.
 - *Thời gian hội tụ bảng định tuyến (Convergence Time)*: Thời gian trung bình để mạng lưới ổn định lại sau mỗi sự kiện một nút bị sập.
 - *Chi phí kiểm soát (Control Overhead)*: Tỷ lệ băng thông tiêu tốn cho các gói tin quản lý (Ping, cập nhật cấu trúc) trên tổng lưu lượng mạng.

Khởi tạo mạng P2P gồm 15 peer, trong đó 5 peer online từ đầu. Mỗi peer online trung bình 60 giây rồi crash, và cứ trung bình 20 giây lại có một peer mới tham gia. Tin nhắn được gửi mỗi 5 giây, heartbeat cũng chạy mỗi 5 giây để phát hiện peer chết, và mỗi peer giữ tối đa 4 láng giềng. Toàn bộ mô phỏng chạy trong 100 giây.

P2PChat Event-Driven Churn Simulation	
Peers	: 15
Init peers	: 5
Topology	: ring
Duration	: 100.0s
Mean session	: 60.0s (Exponential)
Mean downtime	: 30.0s (Exponential)
Mean msg gap	: 5.0s (Exponential)
HB interval	: 5.0s
Lambda join	: 0.05 peer/s
Max neighbors	: 4

Hình 5.10 Khởi tạo P2P gồm 15 peer.

Quá trình mô phỏng:

SIM_T	EVENT	SRC	DST	DETAIL
3.57	REGISTER	peer_07	BOOTSTRAP	online=6 neighbors=4
3.57	ACK	BOOTSTRAP	peer_07	peers=6
3.58	SEND_MSG	peer_01	peer_07	[ONLINE] delivered=True
4.33	SEND_MSG	peer_00	peer_02	[ONLINE] delivered=True
5.00	PING	ALL	NEIGHBORS	online=6 total_neigh=24
8.20	SEND_MSG	peer_00	peer_02	[ONLINE] delivered=True
8.89	REJOIN	peer_06	BOOTSTRAP	offline=3.1s online=7 neighbors=4
8.89	ACK	BOOTSTRAP	peer_06	peers=7
10.00	PING	ALL	NEIGHBORS	online=7 total_neigh=28
11.00	SEND_MSG	peer_02	peer_03	[ONLINE] delivered=True
11.47	SEND_MSG	peer_04	peer_06	[ONLINE] delivered=True
12.30	SEND_MSG	peer_01	peer_00	[ONLINE] delivered=True
12.94	REJOIN	peer_12	BOOTSTRAP	offline=7.9s online=8 neighbors=4
12.94	ACK	BOOTSTRAP	peer_12	peers=8
13.11	REGISTER	peer_08	BOOTSTRAP	online=9 neighbors=4
13.11	ACK	BOOTSTRAP	peer_08	peers=9
14.94	CRASH	peer_03	NETWORK	session=14.9s affected=4
15.00	PING	ALL	NEIGHBORS	online=8 total_neigh=28
15.05	SEND_MSG	peer_02	peer_06	[ONLINE] delivered=True
17.91	CRASH	peer_07	NETWORK	session=14.3s affected=4
19.33	CRASH	peer_12	NETWORK	session=6.4s affected=4
20.00	CONVERGE	NETWORK	peer_12	ct=0.67s
20.00	PING	ALL	NEIGHBORS	online=6 total_neigh=20
20.21	CRASH	peer_08	NETWORK	session=7.1s affected=4
22.22	SEND_MSG	peer_06	peer_04	[ONLINE] delivered=True
25.00	CONVERGE	NETWORK	peer_03	ct=10.06s
25.00	CONVERGE	NETWORK	peer_08	ct=4.79s
25.00	PING	ALL	NEIGHBORS	online=5 total_neigh=16
29.10	REJOIN	peer_12	BOOTSTRAP	offline=9.8s online=6 neighbors=4
29.10	ACK	BOOTSTRAP	peer_12	peers=6
30.00	PING	ALL	NEIGHBORS	online=6 total_neigh=23
30.02	REJOIN	peer_05	BOOTSTRAP	offline=5.1s online=7 neighbors=4
30.02	ACK	BOOTSTRAP	peer_05	peers=7
32.44	SEND_MSG	peer_05	peer_04	[ONLINE] delivered=True
32.88	CRASH	peer_02	NETWORK	session=32.9s affected=4

33.04	SEND_MSG	peer_05	peer_06	[ONLINE] delivered=True
33.79	REJOIN	peer_02	BOOTSTRAP	offline=0.9s online=7 neighbors=4
33.79	ACK	BOOTSTRAP	peer_02	peers=7
34.41	SEND_MSG	peer_06	peer_04	[ONLINE] delivered=True
34.79	REJOIN	peer_13	BOOTSTRAP	offline=7.1s online=8 neighbors=4
34.79	ACK	BOOTSTRAP	peer_13	peers=8
35.00	PING	ALL	NEIGHBORS	online=8 total_neigh=31
37.32	REJOIN	peer_08	BOOTSTRAP	offline=17.1s online=9 neighbors=4
37.32	ACK	BOOTSTRAP	peer_08	peers=9
38.66	REJOIN	peer_07	BOOTSTRAP	offline=20.7s online=10 neighbors=4
38.66	ACK	BOOTSTRAP	peer_07	peers=10
40.00	PING	ALL	NEIGHBORS	online=10 total_neigh=40
41.64	REJOIN	peer_03	BOOTSTRAP	offline=26.7s online=11 neighbors=4
41.64	ACK	BOOTSTRAP	peer_03	peers=11
45.00	PING	ALL	NEIGHBORS	online=11 total_neigh=44
46.10	CRASH	peer_08	NETWORK	session=46.1s affected=4
48.59	CRASH	peer_02	NETWORK	session=14.8s affected=4
50.00	PING	ALL	NEIGHBORS	online=9 total_neigh=32
52.63	REJOIN	peer_09	BOOTSTRAP	offline=24.1s online=10 neighbors=4
52.63	ACK	BOOTSTRAP	peer_09	peers=10
53.43	CRASH	peer_13	NETWORK	session=18.6s affected=4
53.49	SEND_MSG	peer_05	peer_04	[ONLINE] delivered=True
55.00	PING	ALL	NEIGHBORS	online=9 total_neigh=33
57.09	SEND_MSG	peer_03	peer_12	[ONLINE] delivered=True
58.30	SEND_MSG	peer_01	peer_12	[ONLINE] delivered=True
59.69	SEND_MSG	peer_12	peer_00	[ONLINE] delivered=True
60.00	PING	ALL	NEIGHBORS	online=9 total_neigh=33
60.75	SEND_MSG	peer_04	peer_00	[ONLINE] delivered=True
62.94	SEND_MSG	peer_00	peer_12	[ONLINE] delivered=True
65.00	PING	ALL	NEIGHBORS	online=9 total_neigh=33
69.71	REJOIN	peer_02	BOOTSTRAP	offline=21.1s online=10 neighbors=4
69.71	ACK	BOOTSTRAP	peer_02	peers=10
70.00	PING	ALL	NEIGHBORS	online=10 total_neigh=39
71.32	CRASH	peer_02	NETWORK	session=1.6s affected=4
72.90	SEND_MSG	peer_07	peer_12	[ONLINE] delivered=True
73.66	SEND_MSG	peer_01	peer_03	[ONLINE] delivered=True
74.57	REJOIN	peer_11	BOOTSTRAP	offline=1.4s online=10 neighbors=4

```

74.57 ACK          BOOTSTRAP peer_11 peers=10
75.00 CONVERGE     NETWORK peer_02 ct=3.68s
75.00 PING         ALL      NEIGHBORS online=10 total_neigh=37
76.77 SEND_MSG     peer_04 peer_07 [ONLINE] delivered=True
77.67 SEND_MSG     peer_00 peer_06 [ONLINE] delivered=True
77.98 REJOIN       peer_02 BOOTSTRAP offline=6.7s online=11 neighbors=4
77.98 ACK          BOOTSTRAP peer_02 peers=11
78.88 SEND_MSG     peer_05 peer_00 [ONLINE] delivered=True
80.00 PING         ALL      NEIGHBORS online=11 total_neigh=44
80.02 CRASH        peer_00 NETWORK session=80.0s affected=4
84.30 SEND_MSG     peer_11 peer_01 [ONLINE] delivered=True
85.00 PING         ALL      NEIGHBORS online=10 total_neigh=36
85.35 SEND_MSG     peer_07 peer_09 [ONLINE] delivered=True
89.68 SEND_MSG     peer_12 peer_09 [ONLINE] delivered=True
89.68 CRASH        peer_12 NETWORK session=60.6s affected=3
90.00 PING         ALL      NEIGHBORS online=9 total_neigh=31
90.09 CRASH        peer_11 NETWORK session=15.5s affected=4
90.83 REJOIN       peer_14 BOOTSTRAP offline=28.2s online=9 neighbors=4
90.83 ACK          BOOTSTRAP peer_14 peers=9
95.00 CONVERGE     NETWORK peer_11 ct=4.91s
95.00 PING         ALL      NEIGHBORS online=9 total_neigh=34
96.57 SEND_MSG     peer_04 peer_14 [ONLINE] delivered=True
98.34 CRASH        peer_04 NETWORK session=98.3s affected=3
98.74 CRASH        peer_07 NETWORK session=98.7s affected=4
100.00 END                                     simulation complete

```

Hình 5.11 Log vòng đời churn của 15 peer trong 100 giây mô phỏng

```

=====
KET QUA MO PHONG CHURN
=====

[1] THONG KE SU KIEN
Tong su kien xu ly : 84
Loai                      So lan
-----
ACK                        16
CONVERGE                   5
CRASH                      14
END                        1
PING                       19
REGISTER                   2
REJOIN                     14
SEND_MSG                   26
-----
Online dau                 5
Online cuoi                7

```

Hình 5.12 Thống kê mô phỏng churn

Kết quả thống kê cho thấy, có 84 sự kiện được xử lý trong 100s.

```

[2] MESSAGE DELIVERY RATIO (MDR)
Tong tin gui      : 26
Da delivered      : 26
MDR tong          : 100.00%
MDR direct        : 100.00% (26 tin, receiver online)

[3] CONVERGENCE TIME (CT)
So crash phat hien : 5
Trung binh         : 12.51s
Min                : 0.67s
Max                : 42.12s
Ly thuyet (~HB*3)  : 15s

[4] CONTROL OVERHEAD (CO)
Data bytes         : 6,656
Control bytes       : 82,656
Control Overhead    : 92.55%
Pings sent         : 606
Pings lost          : 52

```

Hình 5.13 Kết quả 3 chỉ số

- MDR = 100%: Tất cả 26 tin nhắn đều được delivered thành công, dù mạng liên tục có peer crash. Điều này cho thấy cơ chế mailbox hoạt động tốt — tin nhắn không bị mất khi người nhận tạm thời offline, mà được lưu lại và gửi khi họ quay lại.
- CT trung bình = 12.51s: Rất sát lý thuyết 15s. Chỉ có 5 crash được phát hiện và mạng hồi phục nhanh.
- Bất thường ở chỗ: ở phút 195, có một convergence mất 133.85s; phút 285 mất 84.31s. Điều này xảy ra khi mạng rơi vào trạng thái network partition tạm thời — nhiều peer crash cùng lúc khiến các peer khác bị cô lập, mất nhiều chu kỳ heartbeat mới hồi phục được topology.
- CO = 93.27%: Cao. Đây là trade-off được chấp nhận cho mạng P2P churn-heavy, nhưng nếu cần tối ưu bằng thông thực tế, có thể cân nhắc giảm tần suất heartbeat hoặc dùng cơ chế adaptive heartbeat

Kết luận: Mạng P2P ring với cơ chế heartbeat đạt hiệu quả cao về độ tin cậy truyền tin nhưng phải trả giá bằng băng thông và tốc độ hồi phục không đồng đều theo thời gian.

5.2 Đánh giá

Dự án P2PChat đã xây dựng được một hệ thống chat ngang hàng có đầy đủ các chức năng chính như nhắn tin trực tiếp, chat nhóm, broadcast, gửi file, lưu lịch sử cục bộ và giao tiếp thời gian thực qua WebSocket. Mỗi peer vừa đóng vai trò client gửi tin, vừa là server nhận tin, đúng với định hướng của mô hình P2P.

Một điểm mạnh của hệ thống là khả năng xử lý trường hợp peer không online đồng thời. Khi gửi trực tiếp thất bại, tin nhắn có thể được lưu qua mailbox server. Nếu mailbox không khả dụng, hệ thống tiếp tục fallback sang cơ chế Store-and-Forward qua peer trung gian. Nhờ local outbox, retry nền, relay storage, tombstone và cơ chế kiểm tra trạng thái delivery, tin nhắn có khả năng được giao lại ngay cả trong môi trường mạng không ổn định.

Hệ thống cũng đã áp dụng E2EE cho tin nhắn trực tiếp và tin nhắn offline. Nội dung tin được mã hóa trước khi truyền qua mạng hoặc lưu tại mailbox/relay. Các thành phần trung gian chỉ lưu ciphertext, không đọc được nội dung gốc. Điều này giúp tăng tính riêng tư so với mô hình server trung tâm truyền thống. Tuy nhiên, local database của chính peer vẫn lưu bản rõ sau khi giải mã để phục vụ hiển thị lịch sử chat.

Về mặt lưu trữ, SQLite được sử dụng phù hợp với mô hình mỗi peer có dữ liệu cục bộ riêng. Các bảng như messages, outbound_messages, relay_messages, relay_assignments, known_peers, group_cache giúp hệ thống quản lý lịch sử chat, trạng thái gửi, retry, relay và thông tin peer một cách rõ ràng.

Giao diện Web UI hỗ trợ các thao tác cơ bản như chọn peer, gửi tin, xem trạng thái online/offline, trạng thái gửi tin, nhóm chat và file transfer. Việc sử dụng WebSocket giúp cập nhật tin nhắn và trạng thái outbox gần thời gian thực.

Tuy vậy, hệ thống vẫn còn một số hạn chế. Bootstrap server và mailbox server vẫn là các thành phần quan trọng trong quá trình discovery và offline delivery. Relay peer đã giúp giảm phụ thuộc vào mailbox, nhưng discovery vẫn cần bootstrap hoặc cache cục bộ. Cơ chế E2EE hiện tại mới bảo vệ dữ liệu trên đường truyền và tại trung gian,

chưa mã hóa local history. Ngoài ra, hệ thống chưa có bộ test tự động đầy đủ cho các kịch bản churn phức tạp như nhiều peer offline/online liên tục, relay mất kết nối giữa chừng hoặc network partition.

Nhìn chung, project đã đáp ứng được mục tiêu chính của một hệ thống P2P Chat có khả năng hoạt động trong môi trường mạng không ổn định. Các cơ chế như direct TCP, mailbox, relay Store-and-Forward, durable outbox và E2EE tạo thành một nền tảng tương đối hoàn chỉnh cho việc truyền tin phân tán.

5.3 Kết luận và hướng phát triển

Project P2PChat đã triển khai thành công một hệ thống chat ngang hàng sử dụng Java, React và SQLite. Hệ thống cho phép các peer giao tiếp trực tiếp qua TCP, đồng thời hỗ trợ Web UI để người dùng gửi nhận tin nhắn, quản lý nhóm và truyền file. Bootstrap server được sử dụng để hỗ trợ discovery và heartbeat, còn mailbox server và relay peer giúp xử lý các tình huống peer nhận offline.

Về mặt kỹ thuật, project đã kết hợp nhiều cơ chế quan trọng trong hệ thống phân tán: peer discovery, heartbeat, direct messaging, Store-and-Forward, retry với outbox bền vững, relay qua peer trung gian, E2EE, WebSocket realtime và lưu trữ cục bộ. Điều này giúp hệ thống không chỉ gửi tin trong điều kiện bình thường, mà còn có khả năng phục hồi khi một số thành phần bị lỗi hoặc peer tham gia/rời mạng liên tục.

Cơ chế Store-and-Forward là một điểm nổi bật của project. Hệ thống không chỉ dựa vào mailbox server mà còn bổ sung relay peer trung gian khi mailbox không khả dụng. Nhờ đó, tin nhắn có thể được lưu tạm ở một peer khác và chuyển tiếp lại cho người nhận khi người đó online. Cơ chế này phù hợp với bản chất P2P, nơi các node có thể online/offline bất kỳ lúc nào.

Cơ chế E2EE cũng góp phần bảo vệ tính riêng tư của người dùng. Tin nhắn direct được mã hóa trước khi gửi đi và chỉ được giải mã tại peer nhận. Mailbox server hoặc relay peer chỉ lưu payload đã mã hóa, không đọc được nội dung thật. Đây là nền tảng quan trọng để phát triển hệ thống theo hướng an toàn hơn.

Trong tương lai, project có thể được mở rộng theo các hướng sau:

Bảng 5.1 Hướng phát triển của hệ thống trong tương lai

Hướng phát triển	Mô tả
Mã hóa local database	Mã hóa bảng messages hoặc toàn bộ SQLite DB để bảo vệ lịch sử chat trên thiết bị người dùng
Tăng cường E2EE cho group chat	Bổ sung group key hoặc sender key để group message offline cũng được mã hóa đầy đủ
Cải thiện relay selection	Chọn relay dựa trên uptime, độ ổn định, độ trễ hoặc lịch sử giao tin thành công
DHT discovery	Giảm phụ thuộc vào bootstrap server bằng cơ chế discovery phân tán hơn
NAT traversal	Hỗ trợ hole punching hoặc relay TCP/WebRTC để peer khác mạng dễ kết nối trực tiếp
Test tự động	Xây dựng test cho các kịch bản churn, mailbox down, relay offline, duplicate message và network failure
Quan sát hệ thống	Bổ sung dashboard/metrics cho outbox, relay queue, delivery latency và retry count
Đồng bộ nhiều thiết bị	Cho phép một user dùng nhiều peer/device và đồng bộ lịch sử an toàn
Tối ưu file transfer	Hỗ trợ resume tốt hơn, retry chunk, hoặc transfer qua relay khi direct file TCP thất bại

Tóm lại, P2PChat đã đạt được mục tiêu xây dựng một ứng dụng chat ngang hàng có khả năng truyền tin trực tiếp, lưu và chuyển tiếp tin khi offline, hỗ trợ bảo mật bằng E2EE và hoạt động được trong môi trường Docker nhiều peer. Dù vẫn còn các hướng cải tiến về bảo mật local, kiểm thử và discovery phân tán, project đã tạo được nền tảng vững chắc cho một hệ thống chat P2P có khả năng mở rộng và chịu lỗi tốt hơn.

TÀI LIỆU THAM KHẢO

- [1] Phạm Văn Cường, Nguyễn Xuân Anh, *Giáo trình Các hệ thống phân tán (Dùng cho đề cương INT1405)*, Học viện Công nghệ Bưu chính Viễn thông, Hà Nội, 2024.
- [2] Nancy A. Lynch, *Distributed Algorithms*, Morgan Kaufmann Publishers, San Francisco, California, USA, 1996.
- [3] Gerard Tel, *Introduction to Distributed Algorithms*, 2nd Edition, Cambridge University Press, Cambridge, United Kingdom, 2000.
- [4] Hagit Attiya and Jennifer Welch, *Distributed Computing: Fundamentals, Simulations, and Advanced Topics*, McGraw-Hill, New York, USA, 2004.
- [5] Leslie Lamport, “Time, Clocks, and the Ordering of Events in a Distributed System,” *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, July 1978.
- [6] Hagit Attiya, Sweta Kumari, Archit Somani, and Jennifer L. Welch, “Store-Collect in the Presence of Continuous Churn with Application to Snapshots and Lattice Agreement,” in *Proceedings of the 22nd International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS)*, Lecture Notes in Computer Science, vol. 12514, Springer, 2020, pp. 1–15.

PHỤ LỤC

LINK REPO GITHUB DỰ ÁN

